

INFORMATIQUE • TLE TI

MODULE 1 : PROGRAMMATION

## LEÇON N03

# PROGRAMMATION DYNAMIQUE CÔTÉ SERVEUR AVEC PHP

DURÉE CONSEILLÉE : 15 H

### DANS CETTE LEÇON

Cette leçon initie les élèves de Terminale TI à la programmation dynamique côté serveur avec PHP : insertion de scripts dans HTML, variables, constantes, tableaux, fonctions, opérateurs et superglobales. Elle montre comment récupérer les données d'un formulaire HTML et générer une réponse personnalisée, compétence clé du développement d'applications web au Bac TI.

- À la fin de cette leçon, je sais expliquer le mécanisme d'exécution côté serveur d'un script PHP et le rôle du serveur web.
- À la fin de cette leçon, je sais déclarer et utiliser des variables, des constantes, des tableaux indexés et associatifs.
- À la fin de cette leçon, je sais insérer et tester un script PHP dans une page HTML.
- À la fin de cette leçon, je sais afficher des résultats avec echo, print et print\_r, en formatant le texte avec des balises HTML.
- À la fin de cette leçon, je sais structurer un script PHP : balises, commentaires et inclusion de fichiers (include, require).
- À la fin de cette leçon, je sais déclarer et appeler une fonction PHP avec paramètres et valeur de retour.



### TÉLÉCHARGE ONBUCH

Gratuit • Google Play  
Tuteur IA, annales,  
concours, résultats



### PASSE À ONBUCH+

Tous les cours signés, Tuteur IA  
illimité, fiches PDF — onglet  
OnBuch+ de l'app

### CE COURS SIGNÉ CONTIENT

#### COURS

complet, illustré, pas à pas

#### MÉTHODES

et exercices résolus

#### EXERCICES

type examen + corrigés

#### FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.  
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

## SOMMAIRE

1. Exécution côté serveur et insertion de PHP dans HTML
2. Structure d'un script PHP : commentaires et inclusion de fichiers
3. Affichage, variables, constantes et tableaux
4. Fonctions PHP et opérateurs
5. Variables superglobales et récupération des données d'un formulaire
6. Synthèse : mini-application dynamique complète
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

## OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais expliquer le mécanisme d'exécution côté serveur d'un script PHP et le rôle du serveur web.
- À la fin de cette leçon, je sais insérer et tester un script PHP dans une page HTML.
- À la fin de cette leçon, je sais structurer un script PHP : balises, commentaires et inclusion de fichiers (include, require).
- À la fin de cette leçon, je sais déclarer et utiliser des variables, des constantes, des tableaux indexés et associatifs.
- À la fin de cette leçon, je sais afficher des résultats avec echo, print et print\_r, en formatant le texte avec des balises HTML.
- À la fin de cette leçon, je sais déclarer et appeler une fonction PHP avec paramètres et valeur de retour.
- À la fin de cette leçon, je sais utiliser les opérateurs arithmétiques, de comparaison, logiques et d'assignation.
- À la fin de cette leçon, je sais récupérer et afficher les valeurs d'un formulaire HTML grâce à \$\_GET, \$\_POST et \$\_FILES.
- À la fin de cette leçon, je sais exploiter quelques variables serveur (\$\_SERVER) pour obtenir des informations sur la requête.

## PRÉREQUIS

- Structure d'une page HTML et balises de base (notion de Première TI)
- Création d'un formulaire HTML : balises `<form>`, `<input>`, `<select>`, attributs `method` et `action`
- Notions d'algorithmique : variables, types, instructions conditionnelles et boucles (Première TI)
- Notion de client/serveur et d'URL (réseaux, Première TI)
- Installation et lancement d'un serveur local type WAMP/XAMPP (début d'année de Terminale)

## 1 EXÉCUTION CÔTÉ SERVEUR ET INSERTION DE PHP DANS HTML

Le lycée de Ngaoundéré veut mettre en ligne un formulaire d'inscription : les élèves saisissent leur nom, leur classe et leur moyenne, et le site doit répondre instantanément « Admis en Tle TI » ou « Dossier à revoir ». Une simple page HTML ne sait ni calculer ni décider : elle affiche toujours la même chose, comme un panneau d'affichage. Il faut un programme exécuté sur le serveur, capable de lire ce que l'élève a tapé, de comparer sa moyenne à 10, puis de fabriquer une réponse personnalisée.

**Problématique :** comment une page web peut-elle « réfléchir », récupérer ce que l'utilisateur a saisi et renvoyer une réponse construite à la demande ? C'est exactement le rôle du langage **PHP**, moteur de la plupart des sites dynamiques camerounais : scolarité en ligne, plateformes de mobile money (MTN MoMo, Orange Money), portails d'inscription universitaire. Dans cette section, nous allons comprendre le mécanisme d'exécution côté serveur, puis écrire et tester notre tout premier script PHP.

### 1.1 Page statique ou page dynamique ?

Imaginons d'abord ce que sait faire une page écrite en HTML seul. Le fichier `accueil.html` contient du texte balisé ; le navigateur le lit et l'affiche. Demain, après-demain, dans un an : la page affichera exactement la même chose. C'est une **page statique**.

Or le lycée de Ngaoundéré a besoin que la réponse dépende de la moyenne saisie. Il faut donc qu'un programme s'exécute au moment de la demande et fabrique le HTML de la réponse. C'est une **page dynamique**.

#### DÉFINITION Page statique et page dynamique

Une **page statique** est un fichier HTML dont le contenu est figé : le serveur se contente de l'envoyer tel quel au navigateur. Une **page dynamique** est une page dont le contenu HTML est généré par un programme (ici un script PHP) exécuté sur le serveur à chaque requête. Deux visiteurs, ou le même visiteur à deux instants différents, peuvent donc recevoir des pages différentes.

| Critère                               | Page statique (HTML seul)          | Page dynamique (HTML + PHP)                    |
|---------------------------------------|------------------------------------|------------------------------------------------|
| Contenu                               | Identique pour tous, à tout moment | Généré à la demande, personnalisé              |
| Qui « travaille » ?                   | Le navigateur (affichage)          | Le serveur (calcul) puis le navigateur         |
| Peut traiter un formulaire ?          | Non                                | Oui, via les superglobales                     |
| Peut interroger une base de données ? | Non                                | Oui (MySQL, etc.)                              |
| Exemple camerounais                   | Page de présentation d'un lycée    | Relevé de notes en ligne, portail Mobile Money |

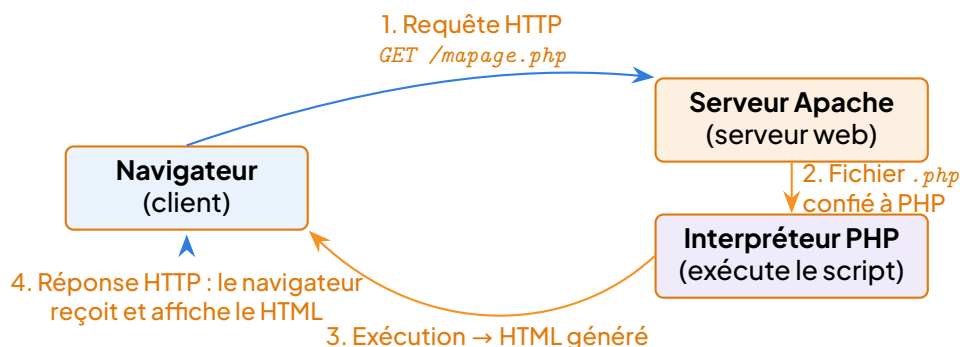
**ATTENTION** Le HTML seul ne décide pas

HTML est un langage de description de documents, pas un langage de programmation : il ne connaît ni les variables, ni les conditions, ni les calculs. Écrire « si moyenne  $\geq 10$  alors afficher Admis » est impossible en HTML pur. C'est précisément le manque que PHP vient combler.

## 1.2 Le dialogue client-serveur : qui fait quoi ?

Pour comprendre PHP, il faut visualiser le voyage d'une requête. Quand tu tapes `http://localhost/mapage` dans ton navigateur, voici ce qui se passe :

1. Le **navigateur** (le **client**) envoie une **requête HTTP** au serveur : « donne-moi la ressource `mapage.php` ».
2. Le **serveur web** (**Apache**) reçoit la requête. Il remarque l'extension `.php` : au lieu d'envoyer le fichier tel quel, il le confie à l'**interpréteur PHP**.
3. L'interpréteur PHP **exécute le script** ligne par ligne : calculs, tests, lecture de données. Tout ce que le script « affiche » (avec `echo`) devient du texte HTML.
4. Apache renvoie au navigateur le **HTML généré** (réponse HTTP). Le navigateur l'affiche.



Le cycle requête-réponse : le PHP est exécuté sur le serveur, le client ne reçoit que du HTML.

#### DÉFINITION Exécution côté serveur

On dit qu'un langage s'exécute **côté serveur** lorsque le code est interprété sur la machine serveur, avant l'envoi de la réponse. Le client ne reçoit que le résultat de l'exécution (du HTML) et ne voit jamais le code source PHP. C'est une différence fondamentale avec JavaScript, qui s'exécute côté client, dans le navigateur.

#### PROPRIÉTÉ Confidentialité du code PHP

Si un visiteur fait « Afficher le code source de la page » dans son navigateur, il voit uniquement le HTML produit, jamais les instructions PHP. C'est ce qui permet de cacher des informations sensibles (identifiants de base de données, règles de calcul, clés d'API) dans les scripts.

#### PROPRIÉTÉ L'extension .php est obligatoire

C'est l'extension du fichier qui déclenche l'interprétation : Apache ne transmet à l'interpréteur PHP que les fichiers dont le nom se termine par `.php` (via la directive `AddType application/x-httpd-php .php`). Un script PHP placé dans un fichier `.html` sera envoyé tel quel au navigateur, sans être exécuté.

#### LESAVAIS-TU ? PHP, un géant discret

PHP signifie « PHP : Hypertext Preprocessor » : c'est un **acronyme récursif**, une clin d'œil des informaticiens (le nom contient son propre nom!). Créé en 1994 par Rasmus Lerdorf,

PHP propulse aujourd'hui la majorité des sites web dynamiques de la planète : WordPress (qui fait tourner des millions de blogs et de sites d'entreprises camerounaises), Wikipédia, et même Facebook à ses débuts. Au Cameroun, la plupart des plateformes de scolarité en ligne, de gestion administrative (impots.cm, portails universitaires) et des services de paiement mobile s'appuient sur PHP côté serveur.

### 1.3 Insérer du PHP dans un document HTML

Un fichier .php est un document HTML ordinaire dans lequel on ouvre des « zones de programme ». On signale au serveur : « attention, ce qui suit est du PHP, exécute-le ».

#### PROPRIÉTÉ Balise canonique d'insertion

Le code PHP s'insère entre la balise ouvrante `<?php` et la balise fermante `?>`. C'est la **balise canonique** : elle fonctionne sur tous les serveurs, quelle que soit leur configuration. Tout ce qui est hors de ces balises est du HTML envoyé tel quel ; tout ce qui est dedans est exécuté.

#### EXEMPLE Premier mélange HTML/PHP

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Ma premiere page dynamique</title>
</head>
<body>
  <h1>Lycée de Ngaoundéré</h1>
  <!-- Ceci est du HTML pur : envoyé tel quel -->
  <p>Bienvenue sur notre portail.</p>
  <?php
    // Ceci est du PHP : exécuté sur le serveur.
    // echo envoie du texte (ici du HTML) vers le navigateur.
    echo "<p>Bonjour le Cameroun !</p>";
  ?>
</body>
</html>
```

Observe bien : le PHP produit du HTML. La chaîne "`<p>Bonjour le Cameroun !</p>`" contient des balises `<p>` : le navigateur recevra un vrai paragraphe HTML, comme si tu l'avais écrit à la main. Le serveur a simplement « rempli les trous » avant l'envoi.

**ATTENTION** Oublier la balise fermante ou se tromper de balise

Si tu écris du PHP hors des balises `<?php ... ?>`, il sera affiché comme du texte brut dans la page : le visiteur verra ton code ! À l'inverse, du HTML écrit à l'intérieur des balises PHP (hors d'un `echo`) provoque une erreur d'analyse (Parse error). Retiens la règle : dedans = PHP, dehors = HTML.

## 1.4 Mise en place pratique : tester un script en local

Pour exécuter du PHP, il faut un serveur web équipé de l'interpréteur PHP. En classe et à la maison, on utilise un paquetage tout-en-un : **WampServer** (Windows) ou **XAMPP** (Windows, Linux, macOS), qui installent ensemble Apache, MySQL et PHP.

**MÉTHODE** Tester un script PHP en local

1. Installer WampServer ou XAMPP, puis lancer le service Apache (l'icône de WAMP doit passer au vert ; dans XAMPP, cliquer sur Start devant Apache).
2. Placer le fichier dans le dossier racine du serveur : `C:\wamp64\www` pour WAMP, `C:\xampp\htdocs` pour XAMPP. Par exemple `www\mapage.php`.
3. Ouvrir le navigateur et taper l'adresse `http://localhost/mapage.php` (localhost désigne « ta propre machine »).
4. Vérifier le résultat affiché, puis modifier le script, enregistrer et recharger la page (touche F5) pour voir l'effet.



Adresse de test : `http://localhost/mapage.php`

Arborescence du dossier `www` : chaque fichier `.php` devient accessible via `http://localhost/`.

**ATTENTION** L'erreur classique du double-clic

Si tu double-cliques sur `mapage.php` dans l'explorateur, le navigateur l'ouvre avec une adresse du type `file:///C:/wamp64/www/mapage.php`. Dans ce cas, le fichier ne passe pas par Apache : le PHP n'est pas interprété et tu vois le code source brut (ou rien du tout). Il

faut toujours passer par `http://localhost/...` pour que la requête transite par le serveur.

#### EXPÉRIENCE TP guidé : ma première page dynamique

Objectif : vérifier l'installation et observer la différence entre HTML statique et PHP dynamique.

Matériel : un poste avec WampServer ou XAMPP installé, un éditeur de texte (Notepad++, VS Code).

Manipulation :

1. Démarre Apache et crée le fichier `www\bonjour.php` avec le contenu suivant :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Premier script PHP</title>
</head>
<body>
  <h1>Bonjour le Cameroun !</h1>
  <?php
    // date() renvoie une chaîne formatée :
    // 'H:i:s' = heures :minutes :secondes du serveur
    echo "<p>Il est exactement " . date('H:i:s') . " sur le serveur.</p>";
  ?>
</body>
</html>
```

2. Ouvre `http://localhost/bonjour.php` dans le navigateur.

3. Recharge la page plusieurs fois (F5) à quelques secondes d'intervalle.

Observations : le titre `<h1>` ne change jamais (HTML statique), mais l'heure affichée est différente à chaque rechargement.

Interprétation : à chaque requête, le serveur ré-exécute le script ; `date('H:i:s')` lit l'horloge du serveur et `echo` l'insère dans le HTML. La page est donc fabriquée à la demande : c'est la définition même du dynamisme côté serveur.

#### EXERCICE RÉSOLU 1 QCM de vérification

Énoncé. Un élève crée le fichier `essai.html` contenant `<?php echo "Salut"; ?>`, le place dans `www` et ouvre `http://localhost/essai.html`. Il voit s'afficher le texte `<?php echo "Salut"; ?>` au lieu de « Salut ». Expliquer la cause de l'erreur et proposer deux corrections.

**SOLUTION**

Cause. Le fichier porte l'extension `.html` : Apache l'envoie directement au navigateur sans le confier à l'interpréteur PHP. Le navigateur reçoit donc le code PHP brut et, ne comprenant pas la balise inconnue `<?php`, l'affiche comme du texte.

Correction 1 (la bonne). Renommer le fichier en `essai.php` puis ouvrir `http://localhost/essai.php` : l'extension `.php` déclenche l'interprétation, et « Salut » s'affiche.

Correction 2 (partielle). Si l'on veut garder un nom en `.html`, il faut reconfigurer Apache pour qu'il passe aussi les `.html` à PHP : c'est possible mais déconseillé à notre niveau, car cela ralentit inutilement toutes les pages statiques.

Leçon à retenir : c'est l'extension `.php` et le passage par `http://localhost` qui garantissent l'exécution du script.

**À RETENIR** L'essentiel de la section

- Une page statique (HTML seul) affiche toujours la même chose ; une page dynamique est générée par un programme à chaque requête.
- PHP s'exécute côté serveur : le navigateur envoie une requête HTTP, Apache confie le fichier `.php` à l'interpréteur PHP, et le client ne reçoit que le HTML généré — jamais le code source.
- Le code PHP s'insère entre `<?php` et `?>` ; dedans c'est du PHP, dehors c'est du HTML. Le PHP produit du HTML grâce à `echo`.
- Pour tester en local : fichier dans `www` (WAMP) ou `htdocs` (XAMPP), Apache démarré, adresse `http://localhost/fichier.php` — jamais de double-clic (`file://`).

**EXERCICE 1** À toi de jouer

1. Crée un fichier `lycee.php` qui affiche le nom de ton lycée dans un titre `<h1>` (HTML pur) et la phrase « Page générée le ... à ... » (date et heure du serveur, avec `date('d/m/Y')` et `date('H:i')`) produite par PHP.
2. Que se passe-t-il si tu enregistres ce fichier sous le nom `lycee.html` ? Justifie.
3. Un camarade affirme : « en faisant Afficher le code source, je peux lire le mot de passe écrit dans le script PHP ». A-t-il raison ? Argumente.

**CORRIGÉ** Exercice — À toi de jouer

1. Fichier `lycee.php` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Mon lycee</title>
</head>
<body>
  <h1>Lycee de Ngaoundéré</h1>
  <?php
    echo "<p>Page générée le " . date('d/m/Y')
      . " à " . date('H:i') . ".</p>";
  ?>
</body>
</html>
```

On enregistre dans `www`, on démarre Apache, puis on ouvre `http://localhost/lycee.php`.

2. Avec `lycee.html`, Apache ne transmet pas le fichier à l'interpréteur PHP : le code `<?php ... ?>` est envoyé tel quel et ne s'exécute pas. La date n'apparaît pas (le navigateur ignore la balise inconnue).

3. Non, il a tort : PHP s'exécute côté serveur, donc « Afficher le code source » ne montre que le HTML généré. Le code PHP — et ses éventuels secrets — reste invisible du client.

## 2 STRUCTURE D'UN SCRIPT PHP : COMMENTAIRES ET INCLUSION DE FICHIERS

Dans la section précédente, tu as découvert qu'un script PHP s'exécute côté serveur et que son résultat (du HTML pur) est envoyé au navigateur. Tu as inséré ton premier bloc `<?php ... ?>` dans une page. Mais écrire un script qui « marche » ne suffit pas : un vrai développeur écrit du code **lisible**, **maintenable** et **réutilisable**. Imagine le site d'une startup doualaïse de 50 pages : si le menu est recopié 50 fois et qu'on ajoute un lien, il faudra modifier 50 fichiers ! Cette section te donne les deux outils qui résolvent ces problèmes : les **commentaires** (pour l'humain qui relit le code) et l'**inclusion de fichiers** (pour factoriser le code commun).

### 2.1 Anatomie d'un script PHP

**DÉFINITION** Script PHP

Un **script PHP** est un fichier texte (d'extension `.php`) contenant une ou plusieurs instructions PHP délimitées par les balises `<?php` et `?>`. Chaque instruction se termine obligatoirement par un **point-virgule** ;.

Trois règles fondamentales gouvernent l'écriture :

1. **Les balises.** Tout code PHP est placé entre `<?php` (ouverture) et `?>` (fermeture). Hors de ces balises, le texte est renvoyé tel quel au navigateur (c'est du HTML).
2. **Le point-virgule.** Chaque instruction se termine par `;`. C'est le signe qui dit à l'interpréteur : « l'instruction est finie, passe à la suivante ».
3. **La casse.** PHP distingue les majuscules des minuscules pour les noms de variables (`$Note` et `$note` sont deux variables différentes), les constantes et les noms de classes, mais pas pour les noms de fonctions (prédéfinies ou utilisateur : `ECHO`, `echo` et `Echo` fonctionnent toutes de la même façon).

#### EXEMPLE Premier script bien formé

```
<?php
    $note = 15;      // déclaration d'une variable
    echo $note;      // affiche 15
    ECHO "points";   // fonction : la casse n'importe pas
?>
```

#### ATTENTION L'erreur classique n°1 : le point-virgule oublié

Oublier le `;` en fin d'instruction provoque une Parse error : syntax error, unexpected ... avec un numéro de ligne. Piège : la ligne indiquée est souvent celle qui suit l'oubli, car PHP ne découvre le problème qu'en rencontrant l'instruction suivante. Si l'erreur est signalée ligne 12, regarde d'abord la ligne 11 !

## 2.2 Les commentaires : écrire pour les humains

Un commentaire est un texte ignoré par l'interpréteur PHP. Il ne s'exécute pas, ne ralentit pas le site : il sert uniquement à expliquer le code à celui qui le relit — y compris toi-même, six mois plus tard, devant ton projet de fin d'année.

#### PROPRIÉTÉ Les trois styles de commentaires en PHP

- `// commentaire` : commentaire sur une seule ligne (style C++);
- `# commentaire` : commentaire sur une seule ligne (style shell, moins utilisé);
- `/* commentaire */` : commentaire sur plusieurs lignes, entre `/*` et `*/`.

```
<?php
/* =====
   Script : moyenne.php
   Auteur : Élève de Tle TI, Lycée de Ngaoundéré
   Rôle   : calcule la moyenne de deux notes
   ===== */

$note1 = 14; // première note sur 20
$note2 = 16; // deuxième note sur 20

$moyenne = ($note1 + $note2) / 2; // formule de la moyenne simple
echo "Moyenne : " . $moyenne;    // affiche : Moyenne : 15
?>
```

Les trois styles de commentaires annotés dans un même script (exemple corrigé)

#### À RETENIR Pourquoi commenter ?

- Lisibilité : un en-tête de fichier (nom, auteur, rôle) permet de comprendre le script en 5 secondes;
- Maintenance : on explique le pourquoi d'un choix, pas le quoi (évident);
- Débogage : on peut « désactiver » temporairement une ligne en la commentant avec `//`, sans la supprimer.

Bonne pratique : commenter chaque bloc logique, jamais chaque ligne de façon évidente (`// on affiche` devant un `echo` n'apporte rien).

#### ATTENTION Ne jamais imbriquer les commentaires multi-lignes

Écrire `/* ... /* ... */ ... */` est une erreur : PHP ferme le commentaire au premier `*/` rencontré, et le reste devient du code exécuté, provoquant une Parse error. Pour désactiver un bloc contenant déjà des `/* */`, utilise plutôt des `//` ligne par ligne.

## 2.3 L'inclusion de fichiers : include et require

**Intuition.** Ton site a 3 pages (accueil, produits, contact) qui partagent le même en-tête et le même pied de page. Plutôt que de copier-coller ce code commun dans chaque page, tu l'écris une seule fois dans un fichier séparé, et chaque page le « branche » au moment de l'exécution.

#### DÉFINITION Inclusion de fichier

L'**inclusion de fichier** est le mécanisme par lequel PHP insère le contenu d'un fichier dans un autre fichier au moment de l'exécution, exactement comme si le code inclus avait été écrit à la place de l'instruction d'inclusion. Quatre instructions existent : `include`, `require`, `include_once` et `require_once`.

#### PROPRIÉTÉ Différence entre `include` et `require`

Si le fichier demandé est absent ou introuvable :

- `include 'fichier.php'` ; produit un avertissement (warning) : le script continue son exécution ;
- `require 'fichier.php'` ; produit une erreur fatale (fatal error) : le script s'arrête immédiatement.

Règle pratique : on utilise `require` pour les fichiers indispensables (connexion à la base de données, fonctions critiques, structure HTML obligatoire) et `include` pour les éléments dont l'absence n'empêche pas la page de fonctionner (bandeau publicitaire, widget météo, pied de page décoratif).

#### PROPRIÉTÉ Les variantes `_once`

`include_once` et `require_once` vérifient si le fichier a déjà été inclus dans le script : si oui, il n'est pas inclus une seconde fois. Cela évite les inclusions multiples qui provoquent des erreurs du type « Cannot redeclare function » lorsqu'un fichier inclus définit des fonctions ou des constantes.

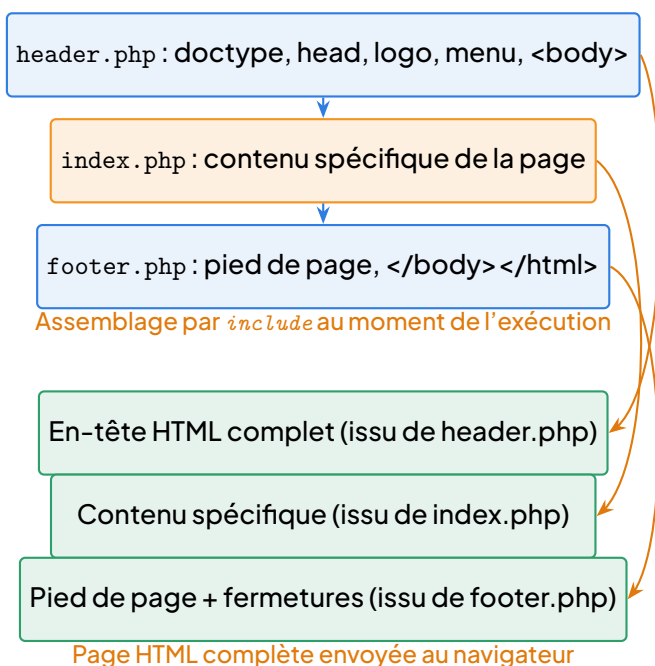
#### EXEMPLE Comparaison des quatre instructions

```
<?php
include 'menu.php';    // warning si absent, le script continue
require 'config.php';  // erreur fatale si absent, arrêt total
include_once 'menu.php'; // inclus une seule fois au maximum
require_once 'config.php'; // idem, avec arrêt si absent
?>
```

## 2.4 Cas d'usage : factoriser l'en-tête et le pied de page

**MÉTHODE** Structurer un site avec header.php et footer.php

1. Créer un fichier `header.php` contenant le début du HTML commun : `<!DOCTYPE html>`, `<head>`, le logo et le menu de navigation. Il ouvre la balise `<body>`.
2. Créer un fichier `footer.php` contenant le pied de page et les balises fermantes `</body></html>`.
3. Dans chaque page du site, écrire `<?php include 'header.php'; ?>` en tout début, le contenu propre à la page au milieu, puis `<?php include 'footer.php'; ?>` à la fin.
4. Pour modifier le menu : éditer uniquement `header.php` — les 3 (ou 50) pages sont mises à jour instantanément.



Un site découpé en blocs : `header.php` et `footer.php` assemblés autour de chaque page par `include`

**EXEMPLE** Les trois fichiers en pratique

Fichier `header.php` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
```

```
<title>Ma boutique – Yaoundé</title>
</head>
<body>
  <nav>
    <a href="index.php">Accueil</a> |
    <a href="produits.php">Produits</a> |
    <a href="contact.php">Contact</a>
  </nav>
```

Fichier `footer.php` :

```
<footer>
  <p>&copy; 2025 Ma boutique – Tous droits réservés</p>
</footer>
</body>
</html>
```

Fichier `index.php` :

```
<?php include 'header.php'; ?>
<h1>Bienvenue sur ma boutique</h1>
<p> Paiement par MTN Mobile Money accepté.</p>
<?php include 'footer.php'; ?>
```

#### EXERCICE RÉSOLU 2

#### Le gain mesurable de la factorisation

Énoncé. Un site vitrine comporte 3 pages. L'en-tête (doctype, `<head>`, menu) occupe 12 lignes et le pied de page 8 lignes, soit 20 lignes communes par page. (a) Combien de lignes dupliquées sans inclusion ? (b) Combien de lignes écrites au total avec inclusion ? (c) Quel est le gain ?

#### SOLUTION

(a) Sans inclusion, les 20 lignes communes sont recopiées dans chacune des 3 pages :

$$20 \times 3 = 60 \text{ lignes dupliquées.}$$

(b) Avec inclusion, on écrit une fois `header.php` (12 lignes) et une fois `footer.php` (8 lignes), plus 2 lignes d'`include` par page :

$$12 + 8 + (2 \times 3) = 26 \text{ lignes.}$$

(c) Le gain est de  $60 - 26 = 34$  lignes écrites en moins, et surtout 60 lignes dupliquées supprimées : toute modification du menu se fait désormais en un seul endroit. Sur un site de 50 pages, le même raisonnement donne  $20 \times 50 = 1000$  lignes dupliquées évitées.

**ATTENTION** Chemins et erreurs d'inclusion

- Le chemin est relatif au script exécuté (le point d'entrée) : si `header.php` est dans un dossier `includes/`, écris `include 'includes/header.php';`.
- Une faute de frappe dans le nom (`header.php`) provoque un warning (`include`) ou une erreur fatale (`require`) : vérifie l'orthographe et la casse, surtout sur un serveur Linux qui distingue majuscules et minuscules dans les noms de fichiers.
- Sécurité : n'inclus jamais un fichier dont le nom provient d'une entrée utilisateur (`$_GET`, `$_POST`) sans contrôle strict : un attaquant pourrait forcer l'inclusion d'un fichier arbitraire (faille d'inclusion locale LFI ou à distance RFI). C'est un réflexe de cybersécurité à acquérir dès maintenant.

**EXPÉRIENCE** TP guidé : découper une page en trois fichiers

Objectif : transformer une page monolithique en site factorisé. Matériel : WampServer (ou XAMPP) démarré, dossier `www/mon_site/`.

1. Crée `header.php` avec le doctype, le `<head>` et un menu de deux liens.
2. Crée `footer.php` avec un pied de page et les balises fermantes.
3. Crée `index.php` qui inclut les deux fichiers et affiche un titre entre les deux.
4. Ouvre `http://localhost/mon_site/index.php` : la page complète s'affiche.
5. Renomme volontairement `footer.php` en `pied.php` et recharge : observe le warning et la page qui s'affiche quand même. Remplace `include` par `require` dans `index.php`, recharge : observe l'erreur fatale et l'arrêt du script.

Interprétation : tu viens de vérifier expérimentalement la différence entre `include` (avertissement, exécution poursuivie) et `require` (erreur fatale, arrêt). Restaure le bon nom de fichier avant de terminer.

**LESAVAIS-TU ?** L'inclusion, fondation des grands sites

Les grands CMS comme WordPress — qui propulse une large part des sites web camerounais, des blogs aux plateformes d'annonces — reposent entièrement sur ce principe : `get_header()` et `get_footer()` ne sont que des habillages autour de `include`. Maîtriser l'inclusion, c'est comprendre l'architecture interne de millions de sites.

**EXERCICE 2 À toi de jouer**

Un site scolaire de 4 pages partage un en-tête de 15 lignes et un pied de page de 10 lignes. (1) Calcule le nombre de lignes dupliquées sans inclusion. (2) Écris le squelette complet de la page `notes.php` utilisant `require_once` pour l'en-tête et le pied de page, en justifiant le choix de `require_once` plutôt qu'`include`. (3) Ajoute un en-tête de commentaire multi-lignes (nom du script, auteur, rôle) au fichier `notes.php`.

**CORRIGÉ Exercice 1**

(1) Lignes communes :  $15 + 10 = 25$  par page, donc  $25 \times 4 = 100$  lignes dupliquées. (2) Squelette :

```
<?php
/* =====
   Script : notes.php
   Auteur : Élève de Tle TI
   Rôle   : page d'affichage des notes
   ===== */
require_once 'header.php'; // indispensable : sans en-tête, pas de page valide
?>
    <h1>Consultation des notes</h1>
<?php
require_once 'footer.php'; // ferme proprement le document HTML
?>
```

On choisit `require_once` car ces fichiers sont indispensables à la validité de la page (sans eux, le HTML est incomplet) : mieux vaut un arrêt net qu'une page cassée ; le suffixe `_once` protège contre une double inclusion accidentelle.

**À RETENIR L'essentiel de la section**

- Un script PHP vit entre `<?php` et `?>` ; chaque instruction se termine par `;` ; les variables et constantes sont sensibles à la casse, pas les fonctions.
- Trois commentaires : `//` et `#` (une ligne), `/* ... */` (multi-lignes, non imbriquables).
- `include` = warning et le script continue ; `require` = erreur fatale et arrêt ; les variantes `_once` évitent les inclusions multiples.
- Factoriser `header.php` et `footer.php` supprime la duplication et simplifie la maintenance : c'est la première règle d'architecture d'un site dynamique.

## 3 AFFICHAGE, VARIABLES, CONSTANTES ET TABLEAUX

Dans la section précédente, tu as appris à structurer un script PHP : les balises `<?php ... ?>`, les commentaires, et l'inclusion de fichiers avec `include` et `require`. Tu sais donc désormais où et comment écrire du code PHP. Mais un script qui ne produit rien et ne mémorise rien ne sert à rien ! Imagine le site de notes de ton lycée : il doit afficher les résultats à l'écran, mémoriser le nom de l'élève, sa classe, ses notes... C'est exactement l'objet de cette section : donner à tes scripts une voix (les fonctions d'affichage) et une mémoire (variables, constantes et tableaux). Ce sont les briques de base de toute application dynamique, du blog personnel à l'API de Mobile Money.

### 3.1 Les fonctions d'affichage : `echo`, `print` et `print_r`

#### Pourquoi afficher ?

Un script PHP s'exécute sur le serveur et son rôle principal est de générer du texte (souvent du HTML) qui sera envoyé au navigateur du client. Sans instruction d'affichage, ton script travaille en silence : il calcule, mais l'utilisateur ne voit rien. PHP propose trois outils complémentaires : `echo`, `print` et `print_r`.

#### DÉFINITION Les fonctions d'affichage `echo` et `print`

**`echo`** et **`print`** sont deux constructions du langage PHP qui envoient une chaîne de caractères vers la sortie (le document HTML généré).

- `echo` accepte plusieurs arguments séparés par des virgules et ne retourne rien. C'est l'instruction la plus utilisée en pratique.
- `print` n'accepte qu'un seul argument et retourne toujours la valeur 1 (ce qui permet, rarement, de l'utiliser dans une expression).

Les parenthèses sont facultatives pour les deux : `echo "Bonjour";` et `echo("Bonjour");` sont équivalents.

#### EXEMPLE Premiers affichages

```
<?php
// echo avec un seul argument
echo "Bienvenue au Lycee de Nkol-Eton !";

// echo avec PLUSIEURS arguments (separés par des virgules)
echo "Nom : ", "Amina", " - Classe : ", "Tle TI";

// print : UN SEUL argument obligatoirement
print "Bonne annee scolaire";

// print retourne 1 : on peut le tester (usage rare)
$retour = print "Texte affiche";
```

```
echo $retour; // affiche 1  
?>
```

**ATTENTION** Piège classique avec print

Écrire `print "a", "b";` provoque une erreur d'analyse (Parse error) car `print` n'accepte qu'un seul argument. Si tu dois afficher plusieurs morceaux, utilise `echo` avec des virgules, ou `print` avec la concaténation (opérateur point, vu plus bas) : `print "a" . "b";`.

**print\_r** : l'outil de débogage des tableaux

`echo` et `print` savent afficher des chaînes et des nombres, mais si tu leur passes un tableau, tu obtiens seulement le mot `Array` (avec un avertissement). Pour inspecter le contenu complet d'une variable complexe, PHP fournit **print\_r** (« print readable », affichage lisible).

**MÉTHODE** Déboguer un tableau avec print\_r

Pour examiner le contenu d'un tableau pendant le développement :

1. Place le tableau dans une variable, par exemple `$notes`.
2. Écris `echo '<pre>';` avant l'affichage : la balise HTML `<pre>` conserve les espaces et retours à la ligne.
3. Appelle `print_r($notes);` : PHP affiche chaque indice/clé avec sa valeur.
4. Ferme avec `echo '</pre>';`.
5. Retire ou commente ces lignes de débogage avant la mise en production : un visiteur ne doit jamais voir la structure interne de tes données.

**EXEMPLE** print\_r en action

```
<?php  
$notes = [12, 15, 9];  
echo '<pre>';  
print_r($notes);  
echo '</pre>';  
/* Affiche :  
Array  
(  
    [0] => 12  
    [1] => 15  
    [2] => 9  
)
```

```
*/
?>
```

| Critère             | echo                                | print               | print_r                                             |
|---------------------|-------------------------------------|---------------------|-----------------------------------------------------|
| Nombre d'arguments  | Plusieurs (virgules)                | Un seul             | Une variable                                        |
| Valeur retournée    | Aucune                              | Toujours 1          | true (ou la chaîne si 2 <sup>e</sup> argument true) |
| Affiche un tableau? | Non (affiche Array)                 | Non (affiche Array) | Oui, en détail                                      |
| Usage principal     | Affichage courant (le plus utilisé) | Affichage simple    | Débogage des tableaux                               |
| Exemple de sortie   | Bonjour Amina                       | Bonjour             | Array ( [0] => 12 )                                 |

### 3.2 Les variables : la mémoire de ton script

#### Intuition et définition

Pour calculer la moyenne d'Amina, ton script doit retenir son nom, ses notes, le total... Ces informations sont stockées dans la mémoire vive du serveur, dans des « cases » auxquelles on donne un nom : ce sont les variables.

#### DÉFINITION Variable PHP

Une **variable** PHP est une zone mémoire nommée, dont le nom commence obligatoirement par le symbole \$, et qui stocke une valeur (texte, nombre, tableau...). PHP est un langage à typage dynamique : on ne déclare pas le type de la variable ; celui-ci est déterminé automatiquement au moment de l'affectation, selon la valeur reçue.

#### EXEMPLE Déclaration par affectation

```
<?php
$nom = 'Amina'; // chaine de caracteres (string)
$age = 17;      // nombre entier (integer)
$moyenne = 14.5; // nombre decimal (float)
$admise = true; // booleen (boolean)

// Typage dynamique : la meme variable peut changer de type
$donnee = 25;    // $donnee est un entier
$donnee = 'Douala'; // $donnee devient une chaine : aucune erreur
?>
```

Contrairement au langage C où l'on écrit `int age = 17;`, en PHP aucun mot-clé de type n'est écrit : le signe = suffit à créer la variable et à la typer en une seule opération.

**PROPRIÉTÉ** Règles de nommage des variables PHP

Le nom d'une variable (après le \$) doit respecter les règles suivantes :

- il commence par une lettre ou un tiret bas (underscore `_`) — jamais par un chiffre ;
- il ne contient que des lettres, chiffres et underscores : pas d'espace, pas d'accent, pas de tiret ;
- il est sensible à la casse : `$moyenne`, `$Moyenne` et `$MOYENNE` sont trois variables différentes.

**Valides :** `$nom`, `$note1`, `$_total`, `$moyenneGenerale`. **Invalides :** `$1note`, `$mon nom`, `$prénom`, `$note-1`.

**Concaténation et interpolation**

Afficher « Amina a 17 ans » à partir de `$nom` et `$age` demande d'assembler des morceaux de texte. PHP offre deux techniques.

**DÉFINITION** Concaténation et interpolation

- La **concaténation** assemble des chaînes avec l'opérateur point `.` : `'Bonjour ' . $nom`.
- L'**interpolation** consiste à écrire la variable directement dans une chaîne entre guillemets doubles : PHP remplace alors `$nom` par sa valeur.

**EXEMPLE** Point de concaténation et guillemets doubles

```
<?php
$nom = 'Amina';
$age = 17;

// Technique 1 : concatenation avec le point (.)
echo 'Bonjour' . $nom . ', tu as ' . $age . ' ans.';

// Technique 2 : interpolation dans les guillemets DOUBLES
echo "Bonjour $nom, tu as $age ans.";

// Les deux lignes produisent exactement :
// Bonjour Amina, tu as 17 ans.
?>
```

**ATTENTION** Guillemets simples contre guillemets doubles : le piège n°1

C'est l'erreur la plus fréquente des débutants (et des candidats au Bac!) :

- Entre guillemets simples, il n'y a aucune interpolation : `echo '$nom'` ; affiche littéralement les quatre caractères `$nom`.
- Entre guillemets doubles, PHP remplace la variable par sa valeur : `echo "$nom"` ; affiche Amina.

Retiens : guillemets doubles = PHP interprète ; guillemets simples = PHP affiche tel quel. En cas de doute, utilise la concaténation avec le point, qui fonctionne dans tous les cas.

### 3.3 Les constantes : des valeurs qui ne changent jamais

Certaines valeurs sont fixées une fois pour toutes : le taux de TVA au Cameroun (19,25 %), le nombre  $\pi$ , le nom de ton établissement. Les stocker dans une variable serait risqué : une ligne malencontreuse pourrait les modifier. PHP propose les **constantes**.

**DÉFINITION** Constante PHP

Une **constante** est une valeur nommée qui, une fois définie, ne peut plus être modifiée ni supprimée pendant toute l'exécution du script. Son nom ne commence pas par \$ et s'écrit par convention en majuscules. Deux syntaxes coexistent :

- `define('TAUX_TVA', 19.25)` ; — utilisable partout, y compris dans un bloc conditionnel ;
- `const PI = 3.14` ; — syntaxe plus courte (PHP 5.3+), au niveau supérieur du script ou dans une classe.

**EXEMPLE** Définition et utilisation de constantes

```
<?php
define('TAUX_TVA', 19.25); // constante définie avec define()
const PI = 3.14;           // constante définie avec const

$prixHT = 10000;           // prix hors taxe en FCFA
$prixTTC = $prixHT * (1 + TAUX_TVA / 100); // pas de $ devant TAUX_TVA !

echo "Prix TTC : " . $prixTTC . " FCFA"; // Prix TTC : 11925 FCFA

// TAUX_TVA = 20 ; // ERREUR : impossible de redéfinir une constante
?>
```

**ATTENTION** Pas de \$ devant une constante

Écrire `$TAUX_TVA` fait croire à PHP qu'il s'agit d'une variable (non déclarée, donc vide, avec un avertissement Warning). Une constante s'utilise sans jamais mettre de \$ : `TAUX_TVA`, `PI`.

### 3.4 Les tableaux : stocker plusieurs valeurs sous un seul nom

#### Le problème à résoudre

Ton lycée veut afficher les 3 notes d'informatique d'Amina. Tu pourrais créer `$note1`, `$note2`, `$note3`... mais que feras-tu avec 30 élèves et 10 matières ? Il faut une structure capable de regrouper plusieurs valeurs dans une seule variable : le **tableau**.

**DÉFINITION** Tableau indexé

Un **tableau indexé** est une variable qui contient une collection ordonnée de valeurs, chacune repérée par un indice numérique entier commençant à 0. Deux syntaxes équivalentes de création :

- `$notes = array(12, 15, 9);` — syntaxe historique;
- `$notes = [12, 15, 9];` — syntaxe courte (PHP 5.4+), la plus utilisée aujourd'hui.

**DÉFINITION** Tableau associatif

Un **tableau associatif** est un tableau dont les éléments sont repérés par des clés (chaînes de caractères choisies par le programmeur) au lieu d'indices numériques. Chaque élément est un couple clé => valeur : `$eleve = ['nom' => 'Amina', 'classe' => 'Tle TI', 'moyenne' => 14.5];`

Tableau indexé : `$notes`

|    |    |   |
|----|----|---|
| 12 | 15 | 9 |
| 0  | 1  | 2 |

Tableau associatif : `$eleve`

|       |          |           |
|-------|----------|-----------|
| Amina | Tle TI   | 14.5      |
| "nom" | "classe" | "moyenne" |

Représentation mémoire d'un tableau indexé (cases numérotées à partir de 0) et d'un tableau associatif (cases étiquetées par des clés).

#### Accès, modification et ajout

**PROPRIÉTÉ** Manipulation des éléments d'un tableau

- **Lecture** : `$notes[0]` vaut 12 (premier élément, indice 0); `$eleve['nom']` vaut 'Amina'.
- **Modification** : `$notes[1] = 17;` remplace 15 par 17.
- **Ajout** : `$notes[] = 18;` ajoute 18 à la fin du tableau (les crochets vides demandent à PHP de calculer le prochain indice libre).
- **Taille** : `count($notes)` retourne le nombre d'éléments.

**EXEMPLE** Lecture, modification, ajout

```
<?php
$notes = [12, 15, 9];
echo $notes[0];      // affiche 12 (attention : indice 0 = 1er element)

$notes[1] = 17;      // la 2e note passe de 15 a 17
$notes[] = 18;       // ajout : $notes vaut [12, 17, 9, 18]
echo count($notes);  // affiche 4

$eleve = ['nom' => 'Amina', 'classe' => 'Tle TI', 'moyenne' => 14.5];
echo $eleve['nom'];   // affiche Amina
$eleve['moyenne'] = 15; // modification via la cle
?>
```

**ATTENTION** L'indice du premier élément est 0, pas 1

Demander `$notes[3]` dans un tableau de 3 éléments (indices 0, 1, 2) provoque un avertissement `Undefined array key` et retourne `NULL`. Le dernier indice vaut toujours `count($notes) - 1`. Autre piège : pour un tableau associatif, la clé doit être entre apostrophes — `$eleve[nom]` (sans apostrophes) fait croire à PHP qu'il s'agit d'une constante nommée `nom`.

### 3.5 Formater en HTML le texte renvoyé par le serveur

Souviens-toi : le navigateur du client ne reçoit **que du HTML**. PHP peut donc glisser des balises HTML directement dans ses affichages pour mettre en forme le résultat : `<strong>` pour le gras, `<br>` pour un saut de ligne, `<em>` pour l'italique.

**EXEMPLE** Calcul de la moyenne affichée en gras

```
<?php
$nom = 'Amina';
$notes = [12, 15, 9]; // tableau indexé des 3 notes

$somme = $notes[0] + $notes[1] + $notes[2];
$moyenne = $somme / count($notes); // 36 / 3 = 12

// Concatenation de balises HTML et de variables avec le point
echo 'La moyenne de <strong>' . $nom . '</strong> est : '
    . '<strong>' . $moyenne . '</strong><br>';
// Le navigateur affiche : La moyenne de Amina (en gras) est : 12 (en gras)
?>
```

**EXERCICE RÉSOLU 3****Moyenne de trois notes avec mise en forme HTML**

Énoncé : on donne les trois notes d'informatique d'Amina : 12, 15 et 9, stockées dans un tableau indexé. Écris un script PHP complet qui calcule sa moyenne et affiche, sur deux lignes, son nom en gras puis sa moyenne en gras suivie de « /20 ».

**SOLUTION**

Solution détaillée :

```
<?php
// 1. Declaration des donnees
$nom = 'Amina';
$notes = [12, 15, 9]; // indices 0, 1 et 2

// 2. Calcul de la moyenne
$somme = $notes[0] + $notes[1] + $notes[2]; // 12 + 15 + 9 = 36
$moyenne = $somme / count($notes); // 36 / 3 = 12

// 3. Affichage formate en HTML (balises <strong> et <br>)
echo 'Eleve : <strong>' . $nom . '</strong><br>';
echo 'Moyenne : <strong>' . $moyenne . '/20</strong>';
?>
```

Étapes du raisonnement : (1) on regroupe les notes dans un tableau indexé, ce qui évite de créer trois variables séparées ; (2) on additionne les éléments par leurs indices 0, 1, 2 et on divise par `count($notes)`, ce qui rend le script réutilisable si le nombre de notes change ; (3) on concatène les balises HTML avec les variables grâce à l'opérateur point. Résultat dans le navigateur : Amina sur la première ligne, 12 /20 sur la seconde.

**EXPÉRIENCE****TP guidé : fiche d'identité dynamique d'un élève**

Objectif : créer une page `fiche.php` qui affiche la fiche d'un élève à partir de variables, d'une constante et d'un tableau associatif.

Matériel / logiciels : WampServer (ou XAMPP) démarré, un éditeur de code, un navigateur.  
Manipulation :

1. Dans le dossier `www` de WampServer, crée le fichier `fiche.php`.
2. Saisis le code suivant :

```
<?php
define('ETABLISSEMENT', 'Lycee de Nkol-Eton');

$eleve = [
    'nom' => 'Amina Mbarga',
    'classe' => 'Terminale TI',
    'moyenne' => 14.5
];

echo '<h1>'. ETABLISSEMENT. '</h1>';
echo 'Nom : <strong>'. $eleve['nom']. '</strong><br>';
echo 'Classe : <em>'. $eleve['classe']. '</em><br>';
echo 'Moyenne : <strong>'. $eleve['moyenne']. '/20</strong><br>';

echo '<pre>';
print_r($eleve); // affichage de debugage
echo '</pre>';
?>
```

3. Enregistre, puis ouvre `http://localhost/fiche.php` dans ton navigateur.

Observations : la page affiche la fiche mise en forme (titre, gras, italique), suivie du contenu détaillé du tableau grâce à `print_r`.

Interprétation : le serveur a exécuté le PHP et n'a envoyé au navigateur que du HTML (vérifie avec « Afficher le code source » : aucune trace de `<?php!`). Modifie une valeur du tableau, recharge : la page change sans toucher au HTML. C'est toute la puissance du dynamisme côté serveur.

#### ATTENTION Erreurs fréquentes à bannir

- Oublier le `$` devant une variable : `nom = 'Amina';` est une erreur fatale.
- Oublier le point de concaténation : `echo 'Bonjour ' $nom;` provoque un Parse error.
- Confondre `'` et `"` : `echo '$nom';` affiche `$nom` au lieu de la valeur.
- Mettre un `$` devant une constante ou oublier les apostrophes autour d'une clé associative.
- Oublier le point-virgule final : chaque instruction PHP se termine par `;`.

**LESAVAIS-TU ?** Au Bac TI, le \$ et le point valent des points

Les correcteurs du Baccalauréat TI le confirment chaque année : dans les questions de code PHP, l'oubli du \$ devant une variable et l'oubli du point de concaténation font partie des erreurs les plus fréquentes et les plus sanctionnées. Une ligne comme `echo "Bonjour"` `nom` peut coûter la totalité des points de la question, même si l'idée est bonne. Entraîne-toi à relire systématiquement chaque ligne en te posant trois questions : ma variable a-t-elle son \$ ? mes chaînes sont-elles reliées par des points ? mon point-virgule est-il là ? Ce réflexe de relecture vaut plusieurs points à l'examen.

**À RETENIR** L'essentiel de la section

- `echo` (plusieurs arguments, le plus utilisé), `print` (un argument, retourne 1) et `print_r` (débogage des tableaux, idéalement entre `<pre>` et `</pre>`).
- Une variable commence par \$, sans déclaration de type : PHP est à typage dynamique. Nommage : lettre ou \_ après \$, pas d'espace ni d'accent, sensible à la casse.
- Concaténation avec le point . ; interpolation uniquement entre guillemets doubles.
- Constante : `define('TAUX_TVA', 19.25);` ou `const PI = 3.14;` — sans \$, en majuscules, non modifiable.
- Tableau indexé : `$notes = [12, 15, 9];`, indices à partir de 0 ; tableau associatif : `$eleve = ['nom' => 'Amina'];` ; ajout avec `$notes[] = 18;` ; taille avec `count()`.
- Le serveur renvoie du HTML : on formate avec des balises concaténées, ex. `echo '<strong>' . $nom . '</strong><br>';`

**EXERCICE 3** À toi de jouer

1. Déclare une constante `FRAIS_SCOLARITE` valant 50000 et une variable `$remise` valant 10 (pourcentage). Affiche le montant à payer après remise, en gras, suivi de « FCFA ».
2. Crée un tableau associatif `$produit` avec les clés `nom` (« Cartable »), `prix` (8500) et `stock` (23). Affiche une phrase complète du type : « Le produit Cartable coûte 8500 FCFA (stock : 23). »
3. Le script suivant contient quatre erreurs ; trouve-les et corrige-les :

```
<?php
nom = 'Amina';
$notes = [12, 15, 9];
echo 'Moyenne de $nom :';
echo $notes[0] + $notes[1] + $notes[3] / 3;
?>
```

**CORRIGÉ Exercices proposés**

```
<?php
define('FRAIS_SCOLARITE', 50000);
$remise = 10;
$montant = FRAIS_SCOLARITE * (1 - $remise / 100); // 45000
echo 'Montant à payer : <strong>' . $montant . '</strong> FCFA';
?>
```

```
<?php
$produit = ['nom' => 'Cartable', 'prix' => 8500, 'stock' => 23];
echo 'Le produit <strong>' . $produit['nom'] . '</strong> coûte '
    . $produit['prix'] . ' FCFA (stock : ' . $produit['stock'] . ')';
?>
```

3. Erreurs corrigées : (a) nom sans \$ → \$nom; (b) 'Moyenne de \$nom : ' en guillemets simples n'interpole pas → utiliser des guillemets doubles ou la concaténation; (c) \$notes[3] n'existe pas (indices 0 à 2) → \$notes[2]; (d) la division est prioritaire sur l'addition → parenthéser : (\$notes[0] + \$notes[1] + \$notes[2]) / 3.

```
<?php
$nom = 'Amina';
$notes = [12, 15, 9];
echo "Moyenne de $nom : ";
echo ($notes[0] + $notes[1] + $notes[2]) / 3; // affiche 12
?>
```

Tu maîtrises désormais l'affichage et les structures de données fondamentales de PHP. Dans la prochaine section, nous donnerons à tes scripts une véritable intelligence : les **fonctions** pour réutiliser ton code, et les **opérateurs** arithmétiques, de comparaison et logiques pour prendre des décisions.

## 4 FONCTIONS PHP ET OPÉRATEURS

Dans la section précédente, tu as appris à afficher des informations avec `echo` et `print`, à stocker des données dans des variables, des constantes et des tableaux. C'est un bon début, mais un vrai site web dynamique ne se contente pas d'afficher : il calcule, il compare, il décide. Imagine le système de gestion des notes du lycée de Nkongsamba que nous développons : pour chaque élève, il faut calculer une moyenne, vérifier si elle atteint 10, et afficher « Admis » ou « Ajourné ». Si le code de calcul de la moyenne est recopié à chaque endroit où on en a besoin, la moindre modification devient un cauchemar. La solution s'appelle une **fonction** : un bloc de code réutilisable, écrit une seule fois. Et pour décider, il faut comparer des valeurs et combiner des conditions : c'est le rôle des **opérateurs**. C'est tout l'objet de cette section.

## 4.1 Déclaration et appel d'une fonction PHP

### Pourquoi une fonction ?

Une fonction résout deux problèmes à la fois : elle évite la duplication de code (on écrit le calcul une seule fois) et elle nomme une action (on comprend le programme en lisant les noms des fonctions). Dans notre application de notes, au lieu de réécrire le calcul de moyenne partout, on écrira une fonction `moyenne()` et on l'appellera autant de fois que nécessaire.

#### DÉFINITION Fonction PHP

Une **fonction** est un bloc d'instructions réutilisable, déclaré avec le mot-clé `function`, pouvant recevoir des **paramètres** en entrée et retourner une valeur avec l'instruction `return`. Une fonction n'est exécutée que lorsqu'elle est appelée par son nom.

#### PROPRIÉTÉ Syntaxe de déclaration et d'appel

La déclaration d'une fonction suit la syntaxe exacte suivante :

```
function nomFonction($param1, $param2){  
    // instructions  
    return $resultat; // valeur renvoyée à l'appelant  
}
```

L'appel s'écrit simplement avec le nom suivi des **arguments** entre parenthèses :

```
$m = moyenne(12, 16); // $m reçoit la valeur retournée
```

Règles à respecter :

- le nom d'une fonction commence par une lettre ou un underscore, sans espace ni accent, et n'est pas sensible à la casse (`Moyenne` et `moyenne` désignent la même fonction);
- les paramètres sont des variables locales précédées de `$`;
- l'instruction `return` termine immédiatement la fonction : tout code placé après un `return` exécuté n'est jamais atteint.

#### EXEMPLE Une fonction de calcul de moyenne

```
<?php  
// Declaration de la fonction : deux parametres  
function moyenne($note1, $note2){  
    $resultat = ($note1 + $note2) / 2;  
    return $resultat; // la valeur est renvoyée à l'appelant  
}  
  
// Appels de la fonction avec des arguments concrets
```

```
$m1 = moyenne(12, 16); // $m1 vaut 14
$m2 = moyenne(8, 9); // $m2 vaut 8.5

echo "Moyenne d'Amina : $m1 <br>";
echo "Moyenne de Brice : $m2";
?>
```

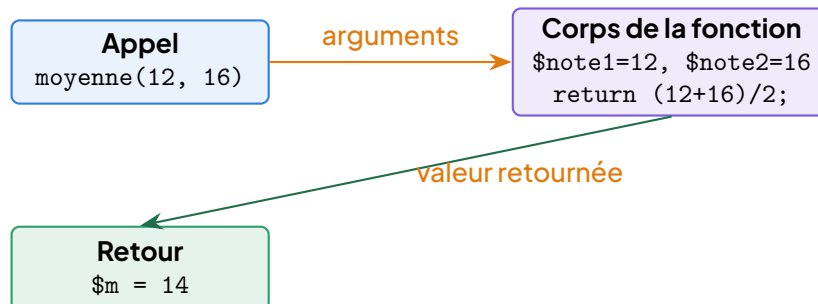
Observe bien la distinction de vocabulaire : `$note1` et `$note2` sont les paramètres (dans la déclaration), tandis que 12 et 16 sont les arguments (dans l'appel). Le passage se fait **par valeur** : PHP recopie la valeur de chaque argument dans le paramètre correspondant ; la fonction travaille sur des copies et ne peut pas modifier les variables de l'appelant.

### Fonction sans valeur de retour.

Une fonction peut aussi se contenter d'effectuer une action (par exemple un affichage) sans rien retourner. On omet alors l'instruction `return` :

```
<?php
function afficherTitre($texte) {
    echo "<h2 style='color:navy'>$texte</h2>"; // affichage direct
}

afficherTitre("Bulletin de Terminale TI"); // appel, rien à récupérer
?>
```



Flux d'appel d'une fonction : les arguments entrent dans les paramètres, le corps calcule, `return` renvoie le résultat à l'appelant.

### Portée des variables (complément utile au Bac).

Une variable déclarée à l'intérieur d'une fonction est **locale** : elle n'existe que pendant l'exécution de la fonction et est invisible à l'extérieur. Réciproquement, une variable déclarée dans le script principal n'est pas accessible directement dans une fonction.

```
<?php
function test() {
    $x = 5; // variable locale à la fonction
}

test();
echo $x; // ERREUR : $x n'existe pas ici (variable indéfinie)
```

| ?&gt;

C'est précisément pour communiquer avec la fonction qu'on utilise les paramètres (en entrée) et `return` (en sortie). Ce point n'est pas exigé en détail par le programme, mais il explique de nombreuses erreurs « Undefined variable » que tu rencontreras en TP.

## 4.2 Les opérateurs arithmétiques et d'assignation

### Opérateurs arithmétiques.

PHP fournit les opérateurs de calcul usuels, plus deux très utiles en programmation : le modulo et la puissance.

| Opérateur | Nom            | Exemple | Résultat |
|-----------|----------------|---------|----------|
| +         | Addition       | 7 + 3   | 10       |
| -         | Soustraction   | 7 - 3   | 4        |
| *         | Multiplication | 7 * 3   | 21       |
| /         | Division       | 7 / 2   | 3,5      |
| %         | Modulo (reste) | 17 % 5  | 2        |
| **        | Puissance      | 2 ** 10 | 1024     |

#### EXEMPLE Le modulo en pratique : parité d'un nombre d'élèves

L'expression `17 % 5` vaut 2, car  $17 = 5 \times 3 + 2$  : le modulo donne le reste de la division euclidienne. Application concrète : le proviseur veut répartir les élèves en rangées de deux pour la visite du rectorat. Un nombre d'élèves est pair si et seulement si son reste dans la division par 2 est nul :

```
<?php
$nbEleves = 47;
if ($nbEleves % 2 == 0) {
    echo "Nombre pair : rangées complètes.";
} else {
    echo "Nombre impair : un élève sera seul sur la dernière rangée.";
}
// Affiche : Nombre impair : un élève sera seul...
?>
```

#### LESAVAIS-TU ? Le modulo, star discrète de l'informatique

L'opérateur `%` est partout : pour alterner les couleurs des lignes paires et impaires d'un tableau de notes affiché en HTML (`$i % 2`), pour faire tourner un compteur en boucle (les minutes d'une horloge : `$total % 60`), ou encore dans les algorithmes de chiffrement qui protègent tes transactions MTN Mobile Money. Retenir « modulo = reste » ouvre la porte à des dizaines d'astuces de programmation.

## Opérateurs d'assignation et incrémentation.

L'opérateur = affecte une valeur à une variable. PHP propose des raccourcis pour modifier une variable à partir de sa propre valeur :

| Écriture raccourcie       | Équivalent                     | Effet                       |
|---------------------------|--------------------------------|-----------------------------|
| <code>\$a += 3;</code>    | <code>\$a = \$a + 3;</code>    | ajoute 3 à \$a              |
| <code>\$a -= 2;</code>    | <code>\$a = \$a - 2;</code>    | retire 2                    |
| <code>\$a *= 4;</code>    | <code>\$a = \$a * 4;</code>    | multiplie par 4             |
| <code>\$a /= 2;</code>    | <code>\$a = \$a / 2;</code>    | divise par 2                |
| <code>\$t .= "cm";</code> | <code>\$t = \$t . "cm";</code> | concatène (ajoute du texte) |
| <code>\$i++;</code>       | <code>\$i = \$i + 1;</code>    | incrémente de 1             |
| <code>\$i--;</code>       | <code>\$i = \$i - 1;</code>    | décrémente de 1             |

```
<?php
$total = 1000; // prix d'un livre en FCFA
$total += 500; // ajout des frais : $total vaut 1500
$total *= 3;   // trois exemplaires : $total vaut 4500
$message = "Total : ";
$message .= $total; // concatenation avec .=
$message .= " FCFA";
echo $message; // Total : 4500 FCFA
?>
```

## 4.3 Opérateurs de comparaison et opérateurs logiques

### Opérateurs de comparaison.

Une comparaison produit un **booléen** : `true` (vrai) ou `false` (faux). C'est le carburant des instructions conditionnelles `if`.

| Opérateur          | Signification                      | Exemple vrai             |
|--------------------|------------------------------------|--------------------------|
| <code>==</code>    | égalité de valeur (sans les types) | <code>10 == "10"</code>  |
| <code>===</code>   | égalité stricte (valeur et type)   | <code>10 === 10</code>   |
| <code>!=</code>    | différent                          | <code>5 != 8</code>      |
| <code>&lt;</code>  | strictement inférieur              | <code>7 &lt; 12</code>   |
| <code>&gt;</code>  | strictement supérieur              | <code>15 &gt; 9</code>   |
| <code>&lt;=</code> | inférieur ou égal                  | <code>10 &lt;= 10</code> |
| <code>&gt;=</code> | supérieur ou égal                  | <code>14 &gt;= 10</code> |

#### ATTENTION Les deux pièges classiques du Bac et des TP

1. Confondre = et ==. L'écriture `if ($moyenne = 10)` affecte 10 à la variable et la condition est toujours vraie : le programme semble « marcher » mais donne des résultats

absurdes. Dans un `if`, on compare avec `==` ou `===`.

- Oublier que `==` ignore les types. En PHP, `'10' == 10` est vrai (la chaîne est convertie en nombre avant comparaison), alors que `'10' === 10` est faux (les types diffèrent : chaîne contre entier). Les données venant d'un formulaire sont toujours des chaînes : méfie-toi, et préfère `===` quand le type compte.

### Opérateurs logiques.

Pour combiner plusieurs conditions — par exemple « admis en TI et moyenne générale suffisante » — on utilise les opérateurs logiques :

| Opérateur               | Nom         | Signification                                                  |
|-------------------------|-------------|----------------------------------------------------------------|
| <code>&amp;&amp;</code> | ET logique  | vrai si les deux conditions sont vraies                        |
| <code>  </code>         | OU logique  | vrai si au moins une condition est vraie                       |
| <code>!</code>          | NON logique | inverse la valeur : <code>!true</code> vaut <code>false</code> |

| ET : <code>\$a &amp;&amp; \$b</code> |                  |             |
|--------------------------------------|------------------|-------------|
| <code>\$a</code>                     | <code>\$b</code> | résultat    |
| vrai                                 | vrai             | <b>vrai</b> |
| vrai                                 | faux             | faux        |
| faux                                 | vrai             | faux        |
| faux                                 | faux             | faux        |

| OU : <code>\$a    \$b</code> |                  |             |
|------------------------------|------------------|-------------|
| <code>\$a</code>             | <code>\$b</code> | résultat    |
| vrai                         | vrai             | <b>vrai</b> |
| vrai                         | faux             | <b>vrai</b> |
| faux                         | vrai             | <b>vrai</b> |
| faux                         | faux             | faux        |

Tables de vérité : le ET exige que les deux conditions soient vraies; le OU n'en exige qu'une seule.

Exemple concret : un élève obtient une bourse s'il a au moins 14 de moyenne et une assiduité d'au moins 90 % :

```
<?php
$moyenne = 15;
$assiduite = 92; // en pourcentage

if ($moyenne >= 14 && $assiduite >= 90) {
    echo "Bourse accordée.";
} else {
    echo "Bourse refusée.";
}
// Affiche : Bourse accordée.
?>
```

## 4.4 Priorité des opérateurs et parenthèses

### PROPRIÉTÉ Priorités des opérateurs (admise)

Les opérateurs arithmétiques de PHP suivent les priorités usuelles des mathématiques : **\*\*** d'abord, puis **\***, **/** et **%**, puis **+** et **-**. Les comparaisons viennent ensuite, puis **&&**, puis **||**. Les **parenthèses** l'emportent toujours sur toutes les priorités.

Ainsi,  $2 + 3 * 4$  vaut 14 (et non 20), car la multiplication est prioritaire. De même :

$$(12 + 16)/2 = 14 \quad \text{mais} \quad 12 + 16/2 = 20.$$

C'est l'erreur classique du calcul de moyenne : sans parenthèses autour de la somme, le résultat est faux. En cas de doute, mets des parenthèses : elles ne coûtent rien et rendent le code plus lisible.

## 4.5 Application : une fonction de décision

### MÉTHODE Écrire une fonction de décision en trois étapes

1. Identifier les paramètres : de quelles données la décision dépend-elle ? Ici, une seule : la moyenne de l'élève.
2. Tester la condition avec un opérateur de comparaison : ici `$moyenne >= 10`.
3. Retourner le résultat avec `return` dans chaque branche du test.

### EXERCICE RÉSOLU 4 Fonction de décision d'admission

Énoncé : écris une fonction PHP `decider($moyenne)` qui retourne la chaîne "Admis" si la moyenne est supérieure ou égale à 10, et "Ajourné" sinon. Teste-la avec les moyennes 13,5 et 8.

#### SOLUTION

Solution détaillée :

```
<?php
// Etape 1 : un seul parametre, la moyenne
function decider($moyenne){
    // Etape 2 : comparaison avec >=
    if ($moyenne >= 10){
        return "Admis"; // Etape 3 : retour dans chaque branche
    } else {
        return "Ajourné";
    }
}
```

```
}  
  
// Appels de test  
echo "Amina (13,5) : " . decider(13.5) . "<br>"; // Admis  
echo "Brice (8) : " . decider(8); // Ajourné  
?>
```

Vérification mentale : pour  $13,5 \geq 10$ , la condition est vraie, la fonction retourne "Admis"; pour 8, la condition est fausse, c'est la branche `else` qui s'exécute. Remarque que `return` quitte immédiatement la fonction : on aurait même pu omettre le `else` et écrire simplement `return "Ajourné";` après le `if`.

#### EXERCICE 4 À toi de jouer

1. Écris une fonction `mention($moyenne)` qui retourne "Passable" si  $10 \leq m < 12$ , "Assez bien" si  $12 \leq m < 14$ , "Bien" si  $14 \leq m < 16$ , et "Très bien" si  $m \geq 16$  (utilise `&&` pour les intervalles).
2. Écris une fonction `estPair($n)` qui retourne `true` si l'entier `$n` est pair, `false` sinon (indice : modulo).
3. Que vaut l'expression  $2 + 3 * 4 ** 2$ ? Justifie avec les priorités.

#### CORRIGÉ Exercice

```
function mention($m) {  
    if ($m >= 16) return "Très bien";  
    if ($m >= 14 && $m < 16) return "Bien";  
    if ($m >= 12 && $m < 14) return "Assez bien";  
    if ($m >= 10 && $m < 12) return "Passable";  
    return "Aucune mention";  
}
```

2. `return ($n % 2 == 0);` — le reste de la division par 2 est nul exactement quand le nombre est pair.
3. La puissance est prioritaire :  $4^2 = 16$ ; puis la multiplication :  $3 \times 16 = 48$ ; puis l'addition :  $2 + 48 = 50$ . L'expression vaut donc 50.

#### À RETENIR L'essentiel de la section

- Une fonction se déclare avec `function nom($param) { ... return $r; }` et s'appelle par son nom avec des arguments ; le passage des paramètres se fait par valeur.
- Une variable déclarée dans une fonction est locale ; on communique avec la fonction via les paramètres et `return`.
- Opérateurs arithmétiques : `+` `-` `*` `/` `%` `**` ; le modulo donne le reste de la division.
- Assignment : `=`, raccourcis `+=` `-=` `*=` `/=` `.=`, incrémentation `$i++`.
- Comparaison : `==` compare les valeurs, `===` compare valeur et type ; ne jamais écrire `=` dans un `if`.
- Logique : `&&` (ET), `||` (OU), `!` (NON).
- Les priorités sont celles des mathématiques ; les parenthèses l'emportent toujours.

Tu sais désormais faire calculer, comparer et décider à tes scripts PHP. Il ne manque plus qu'un ingrédient pour un site vraiment dynamique : recevoir des données de l'utilisateur. C'est exactement l'objet de la prochaine section, consacrée aux variables superglobales `$_GET`, `$_POST` et `$_FILES`.

## 5 VARIABLES SUPERGLOBALES ET RÉCUPÉRATION DES DONNÉES D'UN FORMULAIRE

Dans la section précédente, nous avons appris à écrire des **fonctions** PHP et à manipuler les **opérateurs** (arithmétiques, de comparaison, logiques, d'assignation). Nos scripts savaient donc calculer et décider. Mais il leur manquait quelque chose d'essentiel : ils travaillaient sur des valeurs écrites en dur dans le code. Or une application web réelle — l'inscription à un concours sur un portail du MINESEC, le paiement d'une facture via MTN Mobile Money, la saisie des notes dans un logiciel scolaire — commence toujours par un utilisateur qui **saisit des données dans un formulaire**. Comment ces données voyagent-elles du navigateur jusqu'au script PHP ? C'est précisément le rôle des **variables superglobales**, que nous étudions maintenant.

### 5.1 Qu'est-ce qu'une variable superglobale ?

**Intuition.** Quand un internaute clique sur « Envoyer » dans un formulaire, le navigateur fabrique une requête HTTP contenant les données saisies, puis l'expédie au serveur. Avant même que la première ligne de ton script ne s'exécute, PHP a déjà **décortiqué cette requête** et rangé son contenu dans des tableaux tout prêts. Tu n'as rien à déclarer, rien à importer : ces tableaux existent déjà et sont remplis automatiquement.

#### DÉFINITION Variable superglobale

Une **variable superglobale** est un tableau associatif prédéfini par PHP, automatiquement

rempli à chaque requête HTTP et accessible partout dans le script (y compris à l'intérieur des fonctions, sans le mot-clé `global`). Son nom commence toujours par `$_` en majuscules : `$_GET`, `$_POST`, `$_FILES`, `$_SERVER`, etc.

#### À RETENIR Les superglobales au programme

- `$_GET` : données passées dans l'URL (la query string) ;
- `$_POST` : données envoyées dans le corps de la requête HTTP ;
- `$_FILES` : informations sur les fichiers téléversés ;
- `$_SERVER` : informations fournies par le serveur web (méthode, adresse IP, navigateur...).

## 5.2 Rappel HTML : le formulaire, porte d'entrée des données

Tout commence côté client, avec la balise `<form>`. Deux attributs sont décisifs :

- `action` : l'adresse du script PHP qui recevra les données ;
- `method` : la méthode HTTP d'envoi, `get` ou `post`.

Chaque champ de saisie possède un attribut `name` : c'est ce nom qui deviendra la **clé** dans le tableau superglobal. C'est la correspondance fondamentale à retenir :

`name="moyenne" (HTML)` ↔ `$_POST['moyenne'] (PHP)`

#### EXEMPLE Un formulaire de saisie de moyenne

```
<!-- fichier : saisie.html -->
<form action="decision.php" method="post">
  <label>Nom de l'eleve :
    <input type="text" name="nom">
  </label>
  <label>Moyenne generale :
    <input type="text" name="moyenne">
  </label>
  <input type="submit" value="Envoyer">
</form>
```

Au clic sur « Envoyer », le navigateur transmet les champs `nom` et `moyenne` au script `decision.php` par la méthode POST.

**PROPRIÉTÉ** GET ou POST : où passent les données ? (admise, vérifiable expérimentalement)

Avec `method="get"`, les données sont accrochées à l'URL sous la forme `page.php?nom=Amina&age=17` : elles sont visibles dans la barre d'adresse, mémorisées dans l'historique, et limitées en taille (quelques milliers de caractères selon les navigateurs). Avec `method="post"`, les données transitent dans le corps de la requête HTTP : elles n'apparaissent pas dans l'URL et ne sont pas limitées en taille ; c'est la méthode adaptée aux formulaires sensibles (mots de passe, notes, paiements).

**ATTENTION** GET ne veut pas dire « sécurisé »... ni POST d'ailleurs

POST rend les données invisibles dans l'URL, mais elles circulent quand même en clair sans chiffrement (d'où l'importance du HTTPS, vu dans le module Réseaux). On choisit POST pour les formulaires qui modifient quelque chose (inscription, paiement, note) et GET pour les recherches ou navigations sans danger, dont on veut pouvoir partager le lien.

### 5.3 \$\_GET : lire les données de l'URL

Quand la méthode est `get` (ou quand on écrit soi-même l'URL), PHP remplit le tableau `$_GET` à partir de la query string, c'est-à-dire la partie de l'URL située après le point d'interrogation.

script appelé par le navigateur URL / données

resultat.php?nom=Amina&age=17

clés de \$\_GET

valeurs associées

```
$_GET = [
    'nom' => 'Amina',
    'age' => '17'
]
```

Anatomie d'une URL avec query string : chaque couple clé=valeur, séparé par `&`, remplit une case du tableau `$_GET`.

**EXEMPLE** Lecture de \$\_GET

```
<?php
// URL appelée : resultat.php?nom=Amina&age=17
$nom = $_GET['nom']; // 'Amina'
$age = $_GET['age']; // '17' (attention : toujours une chaîne !)
```

```
echo "Bonjour $nom, tu as $age ans.";
?>
```

Remarque importante : toutes les valeurs reçues sont des chaînes de caractères, même les nombres. Pour calculer avec `$_GET['age']`, PHP la convertira automatiquement, mais il est prudent d'utiliser `(int)` pour forcer la conversion.

## 5.4 \$\_POST : récupérer les données d'un formulaire

Avec `method="post"`, c'est le tableau `$_POST` qui est rempli, selon exactement le même principe de correspondance `name ↔ clé`. Mais avant de lire une clé, il faut **vérifier qu'elle existe** : l'utilisateur a pu laisser un champ vide, ou appeler le script directement sans passer par le formulaire. C'est le rôle de la fonction `isset()`.

### DÉFINITION La fonction `isset()`

`isset($_POST['nom'])` renvoie `true` si la clé 'nom' existe dans le tableau `$_POST` et n'est pas `null`, `false` sinon. C'est le garde-fou obligatoire avant toute lecture d'une donnée de formulaire.

### MÉTHODE Récupérer proprement les données d'un formulaire

1. Vérifier l'existence de chaque champ avec `isset($_POST['champ'])`;
2. Stocker la valeur dans une variable au nom clair : `$nom = $_POST['nom'];`;
3. Traiter : calculs, tests avec les opérateurs de comparaison et logiques;
4. Afficher la réponse avec `echo` en y insérant des balises HTML de formatage (`<strong>`, `<p>`, `<br>`...).

### EXEMPLE Exemple chiffré : le verdict de la moyenne

Le script `decision.php` reçoit le formulaire de saisie vu plus haut. Il répond « Admis en Tle TI » si la moyenne est supérieure ou égale à 10, « Dossier à revoir » sinon.

```
<?php
// fichier : decision.php
// 1) Verification de l'existence des champs
if (isset($_POST['nom']) && isset($_POST['moyenne'])) {
    // 2) Stockage dans des variables
    $nom = $_POST['nom'];
    // Attention : en France/Cameroun on saisit souvent 13,5 (virgule).
```

```
// PHP ne comprend que le point. On remplace la virgule par un point avant conversion.
$moyenne = (float)str_replace(',', '.', $_POST['moyenne']);

// 3) Traitement : comparaison avec la barre de 10
if ($moyenne >= 10) {
    $verdict = "Admis(e) en Tle TI";
    $couleur = "green";
} else {
    $verdict = "Dossier a revoir";
    $couleur = "red";
}

// 4) Reponse HTML formatee
echo "<p>Bonjour <strong>" . htmlspecialchars($nom) . "</strong>,</p>";
echo "<p>Moyenne : $moyenne / 20</p>";
echo "<p style='color:$couleur'><strong>$verdict</strong></p>";
} else {
    echo "<p>Erreur : veuillez remplir le formulaire.</p>";
}
?>
```

Pour `nom = "Amina"` et `moyenne = "13,5"` (saisie 13,5), la page affiche : « Bonjour Amina, Moyenne : 13.5 / 20, **Admis(e) en Tle TI** ».

#### ATTENTION L'erreur classique : se tromper de superglobale

Si le formulaire déclare `method="get"` mais que le script lit `$_POST['nom']`, le tableau `$_POST` reste vide : PHP affiche un avertissement « Undefined array key » et la variable récupérée vaut `null`. Symétriquement, lire `$_GET` pour un formulaire `post` échoue aussi. Règle d'or : la superglobale utilisée doit toujours correspondre à l'attribut `method` du formulaire. Le test `isset()` permet justement de détecter ce problème proprement.

#### ATTENTION Sécurité : ne jamais afficher une saisie brute

Un visiteur malveillant peut taper du code HTML ou JavaScript dans un champ, par exemple `<script>...</script>`. Si ton script fait `echo $_POST['nom'];` sans protection, ce code sera exécuté par le navigateur des autres visiteurs : c'est la faille **XSS** (Cross-Site Scripting), très exploitée par les cybercriminels, y compris contre des sites camerounais. La parade minimale exigée : passer toute donnée affichée par `htmlspecialchars()`, qui transforme `<` en `&lt;` et neutralise le code injecté. Retiens : on ne fait jamais confiance à une saisie utilisateur.

## 5.5 \$\_FILES : récupérer un fichier téléversé

Certaines applications demandent d'envoyer un **fichier** : photo d'identité pour une inscription en ligne, relevé de notes scanné, CV pour une offre d'emploi à Douala. Deux conditions côté

HTML : le formulaire doit être en `method="post"` et porter l'attribut `enctype="multipart/form-data"`, et le champ est un `<input type="file">`.

#### EXEMPLE Formulaire d'envoi de photo

```
<form action="upload.php" method="post"
  enctype="multipart/form-data">
  <label>Votre photo :
    <input type="file" name="photo">
  </label>
  <input type="submit" value="Televerser">
</form>
```

Côté PHP, le fichier n'arrive pas dans `$_POST` mais dans `$_FILES`, un tableau associatif à deux niveaux : la première clé est le `name` du champ, la seconde désigne une caractéristique du fichier.

| Clé de <code>\$_FILES['photo']</code> | Signification                                                                            |
|---------------------------------------|------------------------------------------------------------------------------------------|
| <code>['name']</code>                 | Nom original du fichier sur le poste du client (ex. <code>ma_photo.jpg</code> )          |
| <code>['tmp_name']</code>             | Nom temporaire du fichier sur le serveur, où PHP l'a stocké en attendant le traitement   |
| <code>['size']</code>                 | Taille du fichier en octets                                                              |
| <code>['type']</code>                 | Type MIME déclaré (ex. <code>image/jpeg</code> ) — à ne pas croire aveuglément           |
| <code>['error']</code>                | Code d'erreur : 0 (ou la constante <code>UPLOAD_ERR_OK</code> ) si tout s'est bien passé |

#### EXEMPLE Traitement d'un téléversement

```
<?php
// fichier : upload.php
if (isset($_FILES['photo']) && $_FILES['photo']['error'] == 0) {
    $nomFichier = $_FILES['photo']['name'];
    $temporaire = $_FILES['photo']['tmp_name'];
    $taille     = $_FILES['photo']['size'];

    if ($taille <= 2000000) { // 2 Mo maximum
        // Déplacement du fichier temporaire vers le dossier final
        move_uploaded_file($temporaire, "photos/" . $nomFichier);
        echo "<p>Photo recue : $nomFichier ($taille octets)</p>";
    } else {
        echo "<p>Fichier trop volumineux !</p>";
    }
} else {
    echo "<p>Aucun fichier recu ou erreur d'envoi.</p>";
}
?>
```

Sans `move_uploaded_file()`, le fichier temporaire est supprimé à la fin du script : c'est le piège classique du téléversement.

## 5.6 Quelques variables serveur : `$_SERVER`

Le tableau `$_SERVER` contient des renseignements fabriqués par le serveur web lui-même. Quatre clés à connaître pour le Bac :

| Clé de <code>\$_SERVER</code> | Rôle                                                                                 |
|-------------------------------|--------------------------------------------------------------------------------------|
| <code>REQUEST_METHOD</code>   | Méthode utilisée : 'GET' ou 'POST' — permet de tester comment le script a été appelé |
| <code>PHP_SELF</code>         | Chemin du script en cours — utile pour qu'un formulaire s'appelle lui-même           |
| <code>REMOTE_ADDR</code>      | Adresse IP du client (ex. celle d'un abonné CAMTEL)                                  |
| <code>HTTP_USER_AGENT</code>  | Identité du navigateur du client (Chrome, Firefox...)                                |

### ATTENTION Piège XSS avec `$_SERVER['PHP_SELF']`

Utiliser `action="<?php echo $_SERVER['PHP_SELF']; ?>"` sans protection est dangereux : un attaquant peut injecter du JavaScript via l'URL (ex. `/script.php/"><script>alert(1)</script>`). Toujours écrire : `action="<?php echo htmlspecialchars($_SERVER['PHP_SELF']); ?>"`.

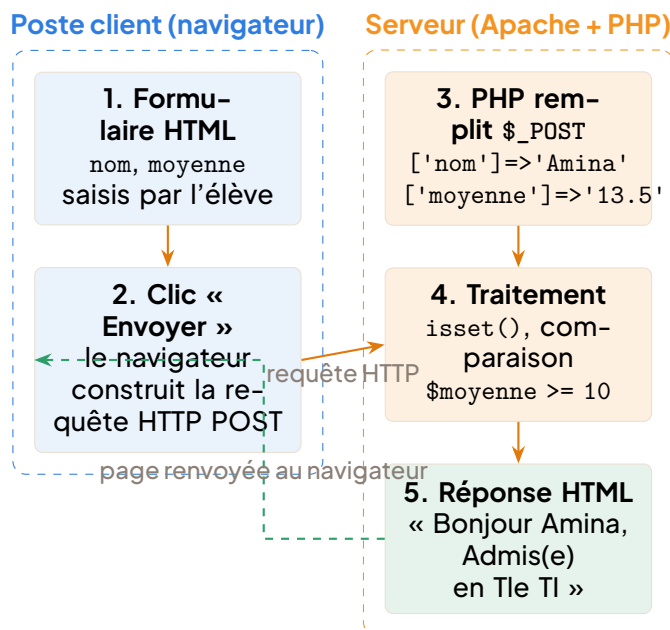
### EXEMPLE Tester la méthode de la requête

```
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST'){
    echo "Le formulaire a bien été envoyé en POST.";
} else {
    echo "Accès direct : affichez le formulaire.";
}
?>
```

C'est la technique standard pour qu'un même fichier affiche le formulaire (première visite, en GET) puis traite les données (envoi, en POST).

## 5.7 La chaîne complète : du formulaire à la réponse

Récapitulons visuellement le voyage des données, du clic de l'élève jusqu'à la page de réponse personnalisée.



Flux complet d'un formulaire : saisie côté client, transport par la requête HTTP, remplissage de \$\_POST, traitement PHP, puis génération de la page de réponse personnalisée.

## EXERCICE RÉSOLU 5

## Formulaire d'inscription à un club informatique

Énoncé. Un lycée de Yaoundé veut inscrire les élèves au club informatique via une page web. Le formulaire (method="post", action="inscription.php") contient deux champs : nom (texte) et classe (texte). Écris le script inscription.php qui vérifie la présence des deux champs, puis affiche « Bienvenue au club, nom de classe ! » de façon sécurisée, ou un message d'erreur sinon.

## SOLUTION

Solution détaillée.

```

<?php
// fichier : inscription.php
// Etape 1 : verifier que le formulaire a ete envoye en POST
if ($_SERVER['REQUEST_METHOD'] == 'POST'){

    // Etape 2 : verifier l'existence des deux champs
    if (isset($_POST['nom']) && isset($_POST['classe'])){

        // Etape 3 : stocker dans des variables
        $nom = $_POST['nom'];
        $classe = $_POST['classe'];

        // Etape 4 : afficher la reponse, protegee par htmlspecialchars
        echo "<p>Bienvenue au club, <strong>"
  
```

```

        . htmlspecialchars($nom) . "</strong> de <em>"
        . htmlspecialchars($classe) . "</em> !</p>";
    } else {
        echo "<p>Veuillez remplir tous les champs.</p>";
    }
} else {
    echo "<p>Acces interdit : passez par le formulaire.</p>";
}
?>

```

Commentaires. Le test sur `$_SERVER['REQUEST_METHOD']` empêche l'accès direct au script par simple saisie de l'URL. Le double `isset()` garantit qu'aucune clé manquante ne provoque d'avertissement. Enfin, `htmlspecialchars()` neutralise toute tentative d'injection de code (XSS) dans les champs texte. Pour `nom = "Ngo Bell"` et `classe = "Tle TI"`, la page affiche : « Bienvenue au club, Ngo Bell de Tle TI! ».

#### EXPÉRIENCE TP guidé : observer `$_GET` et `$_POST` en vrai

Objectif : vérifier expérimentalement la propriété sur le passage des données et le contenu des superglobales. Matériel : un poste avec WampServer (ou XAMPP) démarré, un navigateur.

1. Crée dans `www` un fichier `form.html` contenant un formulaire avec deux champs `nom` et `ville`, d'abord en `method="get"`, `action="espion.php"`.
2. Crée `espion.php` avec le code suivant :

```

<?php
echo "<h3>Contenu de \$_GET</h3>";
echo "<pre>"; print_r($_GET); echo "</pre>";
echo "<h3>Contenu de \$_POST</h3>";
echo "<pre>"; print_r($_POST); echo "</pre>";
echo "<p>Methode : " . $_SERVER['REQUEST_METHOD'] . "</p>";
?>

```

3. Remplis le formulaire et envoie. Observe : les données apparaissent dans l'URL et dans `$_GET`, tandis que `$_POST` est vide (`Array ( )`).
4. Remplace `method="get"` par `method="post"` et recommence. Observe : l'URL reste propre, `$_GET` est vide et `$_POST` contient les données.

Interprétation : la fonction `print_r()` affiche le contenu brut d'un tableau — c'est l'outil de débogage idéal pour inspecter une superglobale. Les balises `<pre>` conservent l'indentation pour une lecture aisée. L'expérience confirme la propriété : GET expose les données dans l'URL, POST les cache dans le corps de la requête.

**EXERCICE 5 À toi de jouer**

1. Écris le formulaire HTML (méthode POST) et le script PHP qui reçoivent le montant d'un achat en FCFA et affichent la remise de 5 % accordée si le montant dépasse 10 000 FCFA (opérateurs arithmétiques et de comparaison exigés).
2. Un élève écrit un formulaire en `method="get"` puis lit `$_POST['note']` dans son script. Explique précisément ce qui se produit et corrige de deux façons différentes.
3. Que contient `$_GET` après l'appel de `notes.php?matiere=TI&coef=4` ? Écris le code qui affiche « Matière : TI, coefficient : 4 » de façon sécurisée.

**CORRIGÉ Exercice**

1. **Formulaire** : `<form action="remise.php" method="post">` avec `<input type="text" name="montant">`. **Script** :

```
<?php
if (isset($_POST['montant'])) {
    $montant = (float)str_replace(',', '.', $_POST['montant']);
    if ($montant > 10000) {
        $remise = $montant * 5 / 100;
        echo "<p>Remise : $remise FCFA</p>";
    } else {
        echo "<p>Aucune remise.</p>";
    }
}
?>
```

2. Le tableau `$_POST` est vide car les données sont parties dans l'URL : PHP signale « Undefined array key » et la variable vaut `null`. Correction 1 : changer le formulaire en `method="post"`. Correction 2 : lire `$_GET['note']` dans le script. 3. `$_GET` contient `['matiere'] => 'TI'` et `['coef'] => '4'`. Affichage sécurisé :

```
<?php
if (isset($_GET['matiere']) && isset($_GET['coef'])) {
    echo "Matiere : " . htmlspecialchars($_GET['matiere'])
        . ", coefficient : " . htmlspecialchars($_GET['coef']);
}
?>
```

**À RETENIR L'essentiel de la section**

- Les superglobales sont des tableaux associatifs prédéfinis, remplis automatiquement par PHP à chaque requête et accessibles partout.

- `$_GET` lit l'URL (visible, limité) ; `$_POST` lit le corps de la requête (discret, sans limite de taille) ; la clé est toujours le `name` du champ HTML.
- On vérifie toujours avec `isset()` avant de lire, et on protège tout affichage avec `htmlspecialchars()`.
- `$_FILES` (avec `enctype="multipart/form-data"`) gère les fichiers téléversés ; `$_SERVER` renseigne sur la requête et le client.
- Attention à `$_SERVER['PHP_SELF']` : toujours l'échapper avec `htmlspecialchars()` dans l'attribut `action`.

Tu maîtrises désormais la réception des données côté serveur. Dans la dernière section, nous assemblerons tout — formulaires, superglobales, opérateurs, fonctions — pour construire une mini-application web dynamique complète, digne d'une vraie startup camerounaise.

## 6 SYNTHÈSE : MINI-APPLICATION DYNAMIQUE COMPLÈTE

Dans la section précédente, tu as appris à récupérer les données d'un formulaire grâce aux superglobales `$_POST`, `$_GET` et `$_FILES`, et à les sécuriser avec `isset()`. Chaque notion — variables, constantes, tableaux, fonctions, opérateurs, inclusions — a été étudiée séparément, comme on apprend les gestes techniques au football. Il est temps de jouer le **match** : nous allons assembler toutes ces compétences dans une véritable mini-application web dynamique, de la maquette jusqu'aux tests. C'est exactement la démarche que tu devras reproduire au Baccalauréat et, plus tard, dans un projet professionnel.

### 6.1 Cahier des charges : le concours d'entrée en Terminale TI

**Situation.** Le lycée bilingue de Nkol-Eton organise un concours d'entrée en Terminale TI. Jusqu'ici, les candidatures sont déposées sur papier : erreurs de saisie, dossiers perdus, délais interminables. Le proviseur te confie la mission de créer une page web d'inscription en ligne.

#### DÉFINITION Cahier des charges

Le **cahier des charges** est le document qui décrit précisément ce que l'application doit faire, avant d'écrire la moindre ligne de code. Il fixe les données à saisir, les règles de traitement et les résultats attendus.

Voici le nôtre, volontairement simple mais complet :

- le candidat saisit son **nom**, sa **classe d'origine**, son **âge**, sa **moyenne annuelle** et téléverse sa **photo** d'identité ;

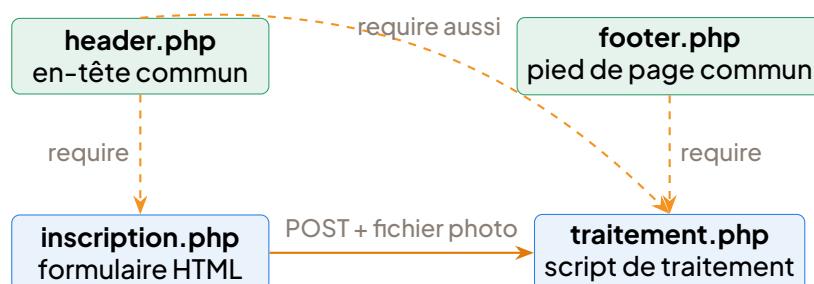
- le serveur applique le barème : **admis si moyenne  $\geq 12$  ET âge  $\leq 20$**  ;
- le serveur affiche une **fiche candidat** formatée en HTML : photo, informations, et décision colorée (vert si admis, rouge sinon) ;
- le code est organisé en fichiers réutilisables (en-tête et pied de page communs).

**LESAVAIS-TU ? Et dans la vraie vie ?**

Cette architecture formulaire → script de traitement → page de réponse est exactement celle des plateformes réelles d'inscription en ligne utilisées dans les universités camerounaises (préinscriptions à l'Université de Yaoundé I, concours de l'ENSPD, plateformes de l'ENS ou de l'IUT). Les développeurs qui les ont construites ont commencé, comme toi aujourd'hui, par un formulaire HTML et un script PHP de quelques dizaines de lignes.

## 6.2 Architecture en fichiers du mini-site

Plutôt qu'un seul fichier fourre-tout, nous découpons l'application en quatre fichiers, reliés par `require` et par la soumission du formulaire :



Architecture du mini-site : les fichiers communs (vert) sont inclus par les pages principales (bleu) ; le formulaire envoie ses données au script de traitement.

L'intérêt est double : si le proviseur veut changer le bandeau du site, on modifie **un seul** fichier (`header.php`) au lieu de chaque page ; et le code de traitement reste séparé de la présentation, ce qui facilite la relecture.

**MÉTHODE Construire une application PHP en quatre étapes**

1. Maquetter les pages : dessiner sur papier (ou en HTML statique) le formulaire et la page de réponse, avec les noms des champs et les couleurs de décision.
2. Écrire le formulaire : choisir `method="post"`, prévoir `enctype` si un fichier est envoyé, et donner à chaque champ un attribut `name` clair.

3. Écrire le traitement pas à pas : vérifier avec `isset()`, récupérer les valeurs, appliquer les règles métier (fonction de décision), puis afficher la réponse.
4. Tester avec des cas limites : moyenne exactement égale à 12, âge exactement égal à 20, champ vide, aucun fichier choisi. Un programme qui marche « en général » mais échoue sur les bornes est un programme faux.

### 6.3 Étape 1 et 2 : les fichiers communs et le formulaire

**header.php** — l'en-tête commun, inclus en haut de chaque page :

```
<?php
// header.php : debut commun de toutes les pages du site
const NOM_SITE = "Concours Tle TI - Lycee de Nkol-Eton";
?>
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title><?php echo NOM_SITE; ?></title>
</head>
<body>
  <h1><?php echo NOM_SITE; ?></h1>
  <hr>
```

**footer.php** — le pied de page commun :

```
<?php
// footer.php : fin commune de toutes les pages
$annee = date("Y"); // annee courante fournie par le serveur
?>
  <hr>
  <p><em>&copy; ; <?php echo $annee; ?> - Lycee bilingue de Nkol-Eton</em></p>
</body>
</html>
```

**inscription.php** — la page du formulaire. Remarque la cohérence des attributs `name` : ce sont eux qui deviendront les clés de `$_POST` et `$_FILES` dans le script de traitement.

```
<?php
// inscription.php : page du formulaire de candidature
require "header.php"; // inclusion de l'en-tete commun
?>
<h2>Formulaire d'inscription au concours</h2>
<form action="traitement.php" method="post"
  enctype="multipart/form-data">
  <p>
    <label for="nom">Nom complet :</label>
    <input type="text" name="nom" id="nom" required>
  </p>
  <p>
    <label for="classe">Classe d'origine :</label>
    <select name="classe" id="classe">
      <option value="Premiere C">Premiere C</option>
```

```

        <option value="Premiere D">Premiere D</option>
        <option value="Premiere TI">Premiere TI</option>
    </select>
</p>
<p>
    <label for="age">Age :</label>
    <input type="number" name="age" id="age" min="14" max="30" required>
</p>
<p>
    <label for="moyenne">Moyenne annuelle (sur 20) :</label>
    <input type="number" name="moyenne" id="moyenne"
        min="0" max="20" step="0.25" required>
</p>
<p>
    <label for="photo">Photo d'identite</label>
    <input type="file" name="photo" id="photo" accept="image/*">
</p>
<p>
    <input type="submit" value="Envoyer ma candidature">
</p>
</form>
<?php
require "footer.php" ; // inclusion du pied de page commun
?>

```

#### ATTENTION L'erreur classique : oublier enctype

Si tu oublies `enctype="multipart/form-data"` dans la balise `<form>`, le navigateur envoie le formulaire sans le contenu du fichier : le tableau `$_FILES` restera vide, même si l'utilisateur a bien choisi une photo. C'est l'une des erreurs les plus fréquentes (et les plus difficiles à diagnostiquer, car aucun message d'erreur ne s'affiche). Règle d'or : dès qu'un formulaire contient un `<input type="file">`, l'attribut `enctype` est obligatoire, et la méthode doit être `post` (un fichier ne peut pas passer par l'URL en `get`).

## 6.4 Étape 3 : le script de traitement

C'est le cœur de l'application. Il vérifie la réception des données, applique le barème grâce à une fonction réutilisable, mémorise le candidat dans un tableau associatif, puis affiche la fiche.

#### EXEMPLE Barème du concours : combiner `&&` et `>=`

Le jury a fixé la règle suivante : admis si moyenne  $\geq 12$  ET âge  $\leq 20$ . En PHP, cela se traduit naturellement par une fonction qui combine un opérateur de comparaison (`>=`, `<=`) et un opérateur logique (`&&`) :

```

function decider($moyenne, $age) {
    // Les DEUX conditions doivent etre vraies : operateur &&
    // Acces aux constantes globales SEUIL_ADMISSION et AGE_MAX
    if ($moyenne >= SEUIL_ADMISSION && $age <= AGE_MAX) {

```

```

        return "ADMIS";
    } else {
        return "REFUSE";
    }
}

```

Exemples chiffrés : moyenne 14 et âge 19 → ADMIS ; moyenne 12,00 et âge 20 → ADMIS (cas limites, les inégalités sont larges) ; moyenne 15 et âge 21 → REFUSE (l'âge élimine malgré la bonne moyenne) ; moyenne 11,75 et âge 18 → REFUSE.

Voici maintenant le script complet, **traitement.php**, avec une gestion sécurisée du nom de fichier uploadé :

```

<?php
// traitement.php : reception et traitement de la candidature
require "header.php";

// ---- 1. Constantes du bareme ----
const SEUIL_ADMISSION = 12; // moyenne minimale exigee
const AGE_MAX = 20; // age maximal autorise
const DOSSIER_PHOTOS = "photos/"; // dossier de destination (doit exister et etre accessible en ecriture)

// ---- 2. Fonction de decision (reutilisable) ----
function decider($moyenne, $age) {
    if ($moyenne >= SEUIL_ADMISSION && $age <= AGE_MAX) {
        return "ADMIS";
    }
    return "REFUSE";
}

// ---- 3. Verification de la reception du formulaire ----
if (isset($_POST["nom"], $_POST["classe"], $_POST["age"],
    $_POST["moyenne"])) {

    // ---- 4. Recuperation et conversion des valeurs ----
    $nom = trim($_POST["nom"]);
    $classe = $_POST["classe"];
    $age = (int) $_POST["age"]; // entier
    $moyenne = (float) $_POST["moyenne"]; // decimal

    // ---- 5. Traitement securise du fichier photo ----
    $photoOK = false;
    $destination = "";
    if (isset($_FILES["photo"]) &&
        $_FILES["photo"]["error"] === UPLOAD_ERR_OK) {

        // Verification de la taille (ex : max 2 Mo)
        if ($_FILES["photo"]["size"] > 2 * 1024 * 1024) {
            echo "<p style='color:red'><strong>Erreur :</strong> le fichier depasse 2 Mo.</p>";
        } else {
            // Securisation du nom de fichier : on garde l'extension, on genere un nom unique
            $extension = strtolower(pathinfo($_FILES["photo"]["name"], PATHINFO_EXTENSION));
            $extensionsAutorisees = ["jpg", "jpeg", "png", "gif", "webp"];
            if (in_array($extension, $extensionsAutorisees)) {
                $nomFichierSecurise = uniqid("candidat_") . "." . $extension;
                $destination = DOSSIER_PHOTOS . $nomFichierSecurise;
            }
        }
    }
}

```

```

        if (move_uploaded_file($_FILES["photo"]["tmp_name"], $destination)) {
            $photoOK = true;
        } else {
            echo "<p style='color:red'><strong>Erreur :</strong> echec de l'enregistrement de la photo.</p>";
        }
    } else {
        echo "<p style='color:red'><strong>Erreur :</strong> format d'image non autorise (jpg, png, gif, webp uniquement).</p>";
    }
}

// ---- 6. Decision et memorisation dans un tableau ----
$decision = decider($moyenne, $age);
$candidat = [
    "nom" => $nom,
    "classe" => $classe,
    "age" => $age,
    "moyenne" => $moyenne,
    "avis" => $decision
];

// ---- 7. Couleur selon la decision ----
if ($decision === "ADMIS") {
    $couleur = "green";
} else {
    $couleur = "red";
}

// ---- 8. Affichage formate de la fiche candidat ----
echo "<h2>Fiche du candidat</h2>";
if ($photoOK) {
    // htmlspecialchars sur le chemin par prudence, bien que genere par nous
    echo "<p><img src='\" . htmlspecialchars($destination) .
        \"' alt='Photo du candidat' width='120'></p>";
}
echo "<table border='1' cellpadding='8'>";
echo "<tr><th>Nom</th><td><strong>\" . htmlspecialchars($candidat[\"nom\"]) .
    \"</strong></td></tr>";
echo "<tr><th>Classe</th><td>\" . htmlspecialchars($candidat[\"classe\"]) .
    \"</td></tr>";
echo "<tr><th>Age</th><td>\" . $candidat[\"age\"] .
    \" ans</td></tr>";
echo "<tr><th>Moyenne</th><td>\" . $candidat[\"moyenne\"] .
    \" / 20</td></tr>";
echo "<tr><th>Decision</th><td style='color : \" . $couleur .
    \"'><strong>\" . $candidat[\"avis\"] .
    \"</strong></td></tr>";
echo "</table>";
} else {
    // Au moins un champ obligatoire n'a pas ete transmis
    echo "<p style='color:red'><strong>Erreur :</strong>
        formulaire incomplet. Veuillez recommencer.</p>";
}

require "footer.php";
?>

```

Observe la logique d'ensemble : vérifier(`isset()`) → récupérer(`$_POST`, `$_FILES`) → sécuriser (trim, cast, validation fichier) → calculer (fonction `decider`) → mémoriser (tableau associatif) → afficher (HTML généré par `echo` avec `htmlspecialchars`). Cette séquence en six temps est la colonne vertébrale de tout script de traitement de formulaire robuste.

**ATTENTION** Parse error et Undefined index : les deux messages à connaître

- Parse error : syntax error, unexpected ... : PHP n'a pas pu lire ton script. Cause n°1 : un point-virgule manquant à la fin d'une ligne, ou une parenthèse/accolade non fermée. Le numéro de ligne indiqué est souvent celui où PHP s'en rend compte : l'oubli est parfois à la ligne précédente.
- Warning : Undefined array key "moyenne" (anciennement Undefined index) : tu lis `$_POST["moyenne"]` alors que cette clé n'existe pas. Causes typiques : l'attribut `name` du champ HTML ne correspond pas exactement à la clé (attention à la casse : "Moyenne" ≠ "moyenne"), ou tu as ouvert `traitement.php` directement sans passer par le formulaire. La parade systématique : tester avec `isset()` avant toute lecture.

**ATTENTION** Sécurité de l'upload : ne jamais faire confiance au client

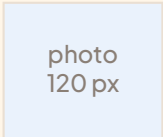
Le code ci-dessus illustre trois protections indispensables :

1. Taille : `$_FILES["photo"]["size"]` comparée à une limite (2 Mo ici).
2. Extension/Type MIME : on extrait l'extension réelle avec `pathinfo()` et on la compare à une liste blanche (`in_array`). Ne te fie jamais à `$_FILES["photo"]["type"]` (fourni par le navigateur, falsifiable).
3. Nom de fichier : on ne conserve jamais le nom original (`$_FILES["photo"]["name"]`) qui peut contenir des caractères dangereux, des espaces, ou permettre d'écraser un fichier existant. On génère un nom unique avec `uniqid()` ou `bin2hex(random_bytes())`.

Sans ces garde-fous, un attaquant pourrait uploader un script `.php` et l'exécuter sur ton serveur.

## 6.5 Étape 4 : maquette de la page de réponse et tests en local

Voici à quoi doit ressembler la fiche affichée par le serveur après un envoi réussi :

| Concours Tle TI – Fiche du candidat                                               |                            |
|-----------------------------------------------------------------------------------|----------------------------|
|  | Nom : MBARGA ESSOMBA Aline |
|                                                                                   | Classe : Première TI       |
|                                                                                   | Âge : 18 ans               |
|                                                                                   | Moyenne : 14,25 / 20       |
|                                                                                   | Décision : ADMIS           |
| © 2025 – Lycée bilingue de Nkol-Eton                                              |                            |

Maquette de la page de réponse : la fiche candidat générée par `traitement.php`, avec la décision colorée en vert.

#### EXPÉRIENCE TP guidé : tester l'application en local

Objectif : vérifier le bon fonctionnement complet du mini-site sous WampServer (ou XAMPP).

Matériel : un poste avec WampServer démarré (icône verte), un navigateur.

1. Crée le dossier `C:\wamp64\www\concours\` et, à l'intérieur, un sous-dossier `photos` (nécessaire pour l'enregistrement des images). Vérifie que le serveur a les droits d'écriture sur `photos/`.
2. Place-y les quatre fichiers : `header.php`, `footer.php`, `inscription.php`, `traitement.php`.
3. Ouvre `http://localhost/concours/inscription.php` dans le navigateur. Le formulaire doit s'afficher avec le bandeau commun.
4. Cas nominal : saisir « MBARGA ESSOMBA Aline », Première TI, 18 ans, 14,25, une photo JPG/PNG. Vérifier que la fiche s'affiche avec ADMIS en vert, que l'image apparaît et qu'un fichier `candidat_...jpg` existe dans le dossier `photos`.
5. Cas limites : moyenne 12,00 et âge 20 → ADMIS ; moyenne 11,75 → REFUSE en rouge ; âge 21 avec moyenne 16 → REFUSE.
6. Cas d'erreur : ouvrir directement `traitement.php` sans passer par le formulaire → le message « formulaire incomplet » doit s'afficher (grâce à `isset`).
7. Test de robustesse : retirer volontairement `enctype` du formulaire, renvoyer → observer que la photo n'apparaît plus, puis corriger.
8. Test sécurité : tenter d'uploader un fichier `test.php` ou une image de 5 Mo → vérifier le message d'erreur rouge.

Interprétation : chaque test correspond à un chemin d'exécution du script. Un développeur professionnel ne dit jamais « ça marche » après un seul essai : il valide les cas nominaux, les cas limites et les cas d'erreur.

## 6.6 Variante : le formulaire auto-soumis

Notre architecture utilise deux pages distinctes. Il existe un pattern très répandu qui n'en utilise qu'une seule.

### PROPRIÉTÉ Formulaire auto-soumis (admise)

Un même script PHP peut à la fois afficher le formulaire et traiter la réponse : il suffit de mettre `action="<?php echo htmlspecialchars($_SERVER['PHP_SELF']); ?>"` (ou un `action` vide) et de tester en début de script si le formulaire a été soumis (`if ($_SERVER["REQUEST_METHOD"] === "POST")` ou `isset($_POST["envoyer"])`). Au premier chargement, la condition est fausse : le formulaire s'affiche. Après soumission, elle est vraie : le même fichier traite les données. Ce pattern, appelé **formulaire auto-soumis**, simplifie la réaffichage des valeurs saisies en cas d'erreur.

## 6.7 Grille de relecture du code

Avant de déclarer ton application terminée, relis chaque fichier avec cette grille — elle couvre 90 % des bugs de débutant :

| Point de contrôle     | Vérification                                                                                                                                               |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Point-virgules        | Chaque instruction PHP se termine par ;                                                                                                                    |
| Dollar des variables  | Toute variable commence par \$ (\$moyenne, jamais moyenne)                                                                                                 |
| Cohérence name / clés | <code>name="moyenne"</code> dans le HTML $\Leftrightarrow$ <code>\$_POST["moyenne"]</code> dans le PHP (même orthographe, même casse)                      |
| Balises fermées       | Chaque <code>&lt;table&gt;</code> a son <code>&lt;/table&gt;</code> , chaque <code>&lt;td&gt;</code> son <code>&lt;/td&gt;</code> , chaque accolade { sa } |
| enctype               | Présent dès qu'un <code>&lt;input type="file"&gt;</code> existe                                                                                            |
| isset()               | Toute lecture de <code>\$_POST</code> / <code>\$_FILES</code> est protégée                                                                                 |
| Chemins d'inclusion   | Les fichiers passés à <code>require</code> existent et sont dans le bon dossier                                                                            |
| Sécurité affichage    | <code>htmlspecialchars()</code> utilisé sur toute variable insérée dans le HTML                                                                            |
| Sécurité upload       | Taille, extension (liste blanche), nom unique généré côté serveur                                                                                          |

### EXERCICE RÉSOLU 6 Ajouter une mention à la décision

**Énoncé.** Le jury souhaite affiner le résultat : un candidat admis avec une moyenne  $\geq 14$  obtient la mention « Bien ». Écris une fonction `mention($moyenne, $age)` qui retourne "ADMIS (mention Bien)", "ADMIS" ou "REFUSE", puis indique la ligne du script `traitement.php` à modifier.

**SOLUTION**

**Solution.** On réutilise la fonction `decider()` existante pour ne pas dupliquer le barème :

```
function mention($moyenne, $age) {  
    $avis = decider($moyenne, $age); // reutilisation  
    if ($avis == "ADMIS" && $moyenne >= 14) {  
        return "ADMIS (mention Bien)";  
    }  
    return $avis; // "ADMIS" simple ou "REFUSE"  
}
```

Dans `traitement.php`, il suffit de remplacer la ligne d'appel :

```
$decision = mention($moyenne, $age); // au lieu de decider(...)
```

Vérification : moyenne 15, âge 18 → « ADMIS (mention Bien) » ; moyenne 13, âge 18 → « ADMIS » ; moyenne 16, âge 22 → « REFUSE » (l'âge prime). Cet exercice illustre un principe capital : une fonction bien conçue se réutilise au lieu d'être recopiée.

**EXERCICE 6 Pour aller plus loin**

1. Modifie le script pour enregistrer chaque candidat dans un tableau indexé `$candidats[]` (plusieurs candidats), puis affiche le nombre total de candidatures reçues avec `count()`. Pense à utiliser la session (`session_start()`) pour persister le tableau entre les requêtes.
2. Ajoute un contrôle qui refuse les fichiers de plus de 2 Mo en testant `$_FILES["photo"]["size"]` (déjà présent dans le code corrigé ci-dessus, vérifie son emplacement).
3. Réécrit l'application en formulaire auto-soumis (un seul fichier `inscription.php`) en utilisant `$_SERVER["PHP_SELF"]` protégé par `htmlspecialchars()`, et en réaffichant les valeurs saisies dans les champs `value="..."` en cas d'erreur de validation.

**À RETENIR L'essentiel de la synthèse**

- Une application PHP dynamique s'organise en fichiers : pages principales, fichiers communs inclus par `require`, script de traitement séparé.

- La séquence de tout traitement robuste : vérifier (`isset`) → récupérer (`$_POST`, `$_FILES`) → sécuriser (`trim`, `cast`, `validation upload`) → calculer (fonctions, opérateurs `>=`, `&&`) → afficher (HTML généré par `echo` avec `htmlspecialchars`).
- Un formulaire contenant un fichier exige `method="post"` et `enctype="multipart/form-data"`.
- On teste toujours les cas nominaux, les cas limites (bornes exactes) et les cas d'erreur.
- La relecture systématique (point-virgules, `$`, cohérence `name/clés`, balises fermées, `htmlspecialchars`, sécurisation `upload`) élimine l'immense majorité des bugs et failles.

Tu es désormais capable de concevoir, coder, tester et relire une application web dynamique complète en PHP — de la maquette au déploiement local. C'est précisément la compétence « créer le dynamisme des pages web » du programme de Terminale TI, et la fondation sur laquelle s'appuieront les leçons suivantes, notamment la connexion à une base de données MySQL pour mémoriser durablement les candidatures.

## 7 ACTIVITÉ D'INTÉGRATION

## 8 PROGRAMMATION DYNAMIQUE CÔTÉ SERVEUR AVEC PHP

### 8.1 Objectifs de l'activité

Cette activité d'intégration vise à mobiliser l'ensemble des savoir-faire acquis au cours de la leçon « Programmation dynamique côté serveur avec PHP » pour concevoir, développer et tester une mini-application web complète. À l'issue de cette activité, vous serez capable de :

- Structurer un site PHP modulaire grâce à l'inclusion de fichiers (`include`).
- Créer un formulaire HTML5 valide envoyant des données par la méthode `POST` et gérant l'envoi de fichiers (`enctype="multipart/form-data"`).
- Exploiter les tableaux superglobaux `$_POST` et `$_FILES` de manière sécurisée (test `isset`, filtrage).
- Manipuler des variables, constantes, tableaux associatifs et fonctions personnalisées.
- Appliquer les opérateurs de comparaison, logiques et d'assignation pour implémenter une règle de gestion métier.
- Générer dynamiquement une réponse HTML structurée (tableau récapitulatif, mise en forme conditionnelle).
- Déboguer les cas limites (seuil exact, champs vides, méthode HTTP erronée).

### 8.2 Situation

Le Lycée Bilingue de Nkolbisson (Yaoundé) souhaite automatiser la pré-sélection des élèves candidats à la classe de Terminale TI (Technologies de l'Information) pour la rentrée prochaine. Actuellement, le secrétariat traite les dossiers manuellement : vérification de la moyenne de Première (seuil fixé à 12/20), contrôle de l'âge (maximum 20 ans au 31 décembre de l'année en cours), et classement des dossiers complets (photo d'identité fournie).

Vous êtes mandatés, par groupes de 3 à 4 élèves, pour développer la page d'admission en ligne qui permettra à un candidat de saisir ses informations, de téléverser sa photo, et d'obtenir instantanément une fiche de décision (Admis / Ajourné) générée par le serveur.

**Données techniques imposées par le cahier des charges :**

- **Seuil d'admission (constante) :** `SEUIL_MOYENNE = 12.0` (type `float`).
- **Âge maximal autorisé :** 20 ans (né après le 31/12/2004 pour la session 2025).
- **Règle de décision (opérateurs logiques) :** Un candidat est **Admis SI ET SEULEMENT SI** (`&&`) sa moyenne est `>= SEUIL_MOYENNE` **ET** son âge est `<= 20`. Sinon, il est **Ajourné**.

- **Champs du formulaire :**

1. `nom_complet` (texte, obligatoire)
2. `classe_origine` (liste déroulante : 1ère TI, 1ère S, 1ère STI2D, Autre, obligatoire)
3. `moyenne_premiere` (nombre, `step="0.01"`, `min="0"`, `max="20"`, obligatoire)
4. `date_naissance` (type date, obligatoire pour calculer l'âge)
5. `photo_identite` (fichier image : JPG, PNG, max 2 Mo, obligatoire)

- **Architecture du site (3 fichiers PHP + 1 CSS) :**

- `header.php` : En-tête commun (doctype, `<head>`, logo lycée, barre de navigation).
- `footer.php` : Pied de page commun (mentions légales, contact MTN/Orange Money pour frais de dossier).
- `formulaire.php` : Affiche le formulaire HTML. Inclut `header.php` et `footer.php`.
- `traitement.php` : Reçoit les données POST, valide, calcule, décide, affiche la fiche. Inclut `header.php` et `footer.php`.
- `style.css` : Feuille de style commune (vert pour Admis, rouge pour Ajourné, tableau lisible).

### 8.3 Consignes

Travaillez en binôme ou trinôme sur un poste équipé d'un serveur local (WampServer, XAMPP ou Laragon). Suivez les étapes ci-dessous dans l'ordre. Chaque étape doit être validée par l'enseignant avant de passer à la suivante.

1. **Préparation de l'environnement.** Créez un dossier `admission_ti/` dans `www/` (ou `htdocs/`). Créez les 5 fichiers vides cités ci-dessus. Démarrez Apache. Vérifiez l'accès à `http://localhost/admission`.
2. **Structure commune** (`header.php` / `footer.php`). Écrivez le squelette HTML5 dans `header.php` (jusqu'à `<main>`) et la fermeture dans `footer.php` (à partir de `</main>`). Incluez `style.css`. Testez l'inclusion dans `formulaire.php` (page blanche mais structure valide W3C).
3. **Formulaire HTML** (`formulaire.php`). Implémentez le formulaire complet avec `method="POST"`, `action="traitement.php"`, `enctype="multipart/form-data"`. Utilisez les attributs HTML5 de validation (`required`, `min`, `max`, `accept="image/*"`). Ajoutez un bouton `submit` libellé « Valider ma candidature ».
4. **Constantes et fonction métier** (`traitement.php`). Au tout début du script (avant tout HTML) :
  - (a) Déclarez la constante `SEUIL_MOYENNE`.
  - (b) Écrivez la fonction `calculerAge($dateNaissance)` qui retourne l'âge entier (années révolues) à la date du jour.
  - (c) Écrivez la fonction `decider($moyenne, $age)` qui retourne la chaîne "Admis" ou "Ajourné" selon la règle métier.

5. **Récupération et sécurisation des entrées.** Dans `traitement.php`, après les fonctions :
- (a) Vérifiez que la requête est bien en POST (`$_SERVER['REQUEST_METHOD'] === 'POST'`). Si non, redirigez vers `formulaire.php` avec `header('Location: ...')`.
  - (b) Pour chaque champ texte/nombre, utilisez `isset($_POST['champ'])` et `trim()` pour récupérer la valeur dans une variable locale. Pour le fichier, vérifiez `isset($_FILES['photo_identite'])` && `$_FILES['photo_identite']['error'] === UPLOAD_ERR_OK`.
  - (c) Validez la moyenne : `is_numeric()` et intervalle `[0, 20]`. Validez la date : `strtotime()` ne retourne pas `false`.
  - (d) En cas d'erreur de validation, affichez un message d'erreur clair dans la page (incluant `header/footer`) et arrêtez le script (`exit`).
6. **Tableau associatif candidat et décision.** Construisez un tableau associatif `$candidat` regroupant : `nom`, `classe`, `moyenne`, `age`, `photo_tmp_name`, `decision`. Appelez `decider()` pour remplir `decision`.
7. **Affichage de la fiche récapitulative.** Toujours dans `traitement.php`, après l'inclusion du `header.php`, générez un tableau HTML `<table>` affichant toutes les infos du candidat. La ligne « Décision » doit avoir une classe CSS `admis` (fond vert, texte blanc) ou `ajourne` (fond rouge, texte blanc). Affichez la photo via `` après lecture binaire du fichier temporaire (`file_get_contents + base64_encode`) — pas de stockage disque pour cet exercice.
8. **Tests de non-régression (cas limites).** Testez systématiquement :
- (a) Candidat type : Moyenne 14.5, âge 18 → Admis.
  - (b) Seuil exact : Moyenne 12.0, âge 20 → Admis (opérateur `>=`).
  - (c) Juste en dessous : Moyenne 11.99, âge 18 → Ajourné.
  - (d) Âge limite : Moyenne 15, âge 21 → Ajourné.
  - (e) Champ vide (nom) → Message d'erreur, pas de crash.
  - (f) Accès direct GET sur `traitement.php` → Redirection vers `formulaire`.
  - (g) Fichier non image (ex : `.pdf`) ou `> 2 Mo` → Erreur gérée.
9. **Présentation.** Chaque groupe projette sa page, explique le code clé (fonction `decider`, gestion `$_FILES`, inclusion), et montre les 3 cas de test les plus significatifs.

## 8.4 Ressources à mobiliser

- **Syntaxe d'inclusion** : `include 'header.php'`; (chemin relatif au script exécuté).
- **Variables superglobales** : `$_POST` (champs `application/x-www-form-urlencoded`), `$_FILES` (fichiers `multipart/form-data`), `$_SERVER['REQUEST_METHOD']`.

- **Fonctions PHP utiles** : `isset()`, `empty()`, `trim()`, `htmlspecialchars()` (pour l’affichage sûr), `is_numeric()`, `strtotime()`, `date_diff()`, `date_create()`, `file_get_contents()`, `base64_encode()`, `header()`, `exit()`.
- **Opérateurs** : Arithmétiques (+, -, \*, /), Comparaison (==, ===, !=, >, <, >=, <=), Logiques (&&, ||, !), Assignment (=, +=, -=).
- **Structure de contrôle** : `if` / `elseif` / `else`, `return` dans les fonctions.
- **HTML5** : `<form method="POST" enctype="multipart/form-data">`, `<input type="date" name="date_na">`, `<input type="file" name="photo_identite" accept="image/*">`, `<select><option value="...">`.
- **CSS minimal** : `.admis { background: #28a745; color: white; }`, `.ajourne { background: #dc3545; color: white; }`, `table { border-collapse: collapse; width: 100%; }`, `td, th { border: 1px solid #ddd; padding: 8px; }`.

## 8.5 Démarche et corrigé commenté

### EXERCICE RÉSOLU 7 Corrigé complet de l’activité d’intégration – Admission Tle TI

L’énoncé complet figure dans la section « Situation » ci-dessus.

#### SOLUTION

#### Étape 1 – Arborescence et fichiers

```
www/
├── admission_ti/
│   ├── header.php
│   ├── footer.php
│   ├── style.css
│   ├── formulaire.php
│   └── traitement.php
```

Démarrez Apache (WampServer : icône verte). Accès : `http://localhost/admission_ti/formulaire.php`.

#### Étape 2 – header.php et footer.php

```
<?php
// header.php – En-tête commun à toutes les pages
// Aucune sortie HTML avant cette balise PHP d’ouverture
?>
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admission Terminale TI – Lycée Bilingue Nkolbisson</title>
  <link rel="stylesheet" href="style.css">
  <!-- Favicon optionnel : <link rel="icon" href="favicon.ico"> -->
</head>
<body>
```

```

<header class="site-header">
  <div class="container">
    
    <h1>Lycée Bilingue de Nkolbisson</h1>
    <p class="slogan">Excellence – Discipline – Innovation</p>
  </div>
</header>
<nav class="main-nav">
  <div class="container">
    <a href="formulaire.php">Nouvelle candidature</a>
    <a href="#">Résultats</a>
    <a href="#">Contact</a>
  </div>
</nav>
<main class="container">
  <!-- Contenu spécifique injecté ici par les pages filles -->

```

```

<?php
// footer.php – Pied de page commun
?>

</main> <!-- Fermeture de <main class="container"> ouvert dans header.php -->
<footer class="site-footer">
  <div class="container">
    <p>&copy; ; 2025 Lycée Bilingue Nkolbisson – Tous droits réservés.</p>
    <p>Frais de dossier payables via MTN Mobile Money ou Orange Money au <strong>+237 6XX XX
XX XX</strong>.</p>
    <p>Développé par les élèves de Terminale TI – Session 2025</p>
  </div>
</footer>
</body>
</html>

```

### Étape 3 – style.css (extrait essentiel)

```

/* style.css */
* { box-sizing : border-box; margin : 0; padding : 0; font-family : 'Segoe UI', sans-serif; }
body { background : #f4f6f9; color : #333; line-height : 1.6; }
.container { max-width : 800px; margin : 0 auto; padding : 20px; }
.site-header { background : #003366; color : white; padding : 20px 0; text-align : center; }
.site-header h1 { margin-top : 10px; font-size : 2rem; }
.main-nav { background : #0055aa; padding : 10px 0; }
.main-nav a { color : white; margin-right : 20px; text-decoration : none; font-weight : bold; }
.main-nav a :hover { text-decoration : underline; }

form { background : white; padding : 30px; border-radius : 8px; box-shadow : 0 2px 10px rgba(0,0,0,0.1)
; }
.form-group { margin-bottom : 20px; }
label { display : block; margin-bottom : 5px; font-weight : 600; }
input[type="text"], input[type="number"], input[type="date"], select {
  width : 100%; padding : 10px; border : 1px solid #ddd; border-radius : 4px; font-size : 1rem;
}
input[type="file"] { padding : 10px 0; }
button[type="submit"] {
  background : #003366; color : white; border : none; padding : 12px 30px;
  font-size : 1.1rem; border-radius : 4px; cursor : pointer; width : 100%;
  transition : background 0.3s;
}

```

```

}
button[type="submit"]:hover { background : #0055aa; }

/* Tableau récapitulatif */
table.recap { width : 100%; border-collapse : collapse; margin-top : 20px; background : white; box-
    shadow : 0 2px 10px rgba(0,0,0,0.1); }
table.recap th, table.recap td { padding : 12px; border : 1px solid #e0e0e0; text-align : left; }
table.recap th { background : #003366; color : white; width : 30%; }
table.recap tr:nth-child(even) { background : #f9f9f9; }
.decision-row { font-weight : bold; font-size : 1.1rem; }
.admis { background : #28a745 !important; color : white !important; }
.ajourne { background : #dc3545 !important; color : white !important; }
.photo-preview { max-width : 150px; border : 1px solid #ddd; border-radius : 4px; }

/* Messages d'erreur / alerte */
.alert { padding : 15px; border-radius : 4px; margin-bottom : 20px; text-align : center; font-weight :
    bold; }
.alert-danger { background : #f8d7da; color : #721c24; border : 1px solid #f5c6cb; }
.alert-info { background : #d1ecf1; color : #0c5460; border : 1px solid #bee5eb; }

```

#### Étape 4 – formulaire.php

```

<?php
// formulaire.php – Page d'accueil avec le formulaire de candidature
// Inclusion de l'en-tête commun
include 'header.php';
?>

<section class="form-section">
    <h2>Candidature Terminale TI – Session 2025</h2>
    <p class="lead">Remplissez soigneusement ce formulaire. Les champs marqués * sont obligatoires.</p>

    <form action="traitement.php" method="POST" enctype="multipart/form-data" novalidate>
        <div class="form-group">
            <label for="nom_complet">Nom complet *</label>
            <input type="text" id="nom_complet" name="nom_complet" required
                placeholder="Ex : NGOUMOU Jean-Pierre" maxlength="100">
        </div>

        <div class="form-group">
            <label for="classe_origine">Classe d'origine *</label>
            <select id="classe_origine" name="classe_origine" required>
                <option value="">-- Choisissez votre classe --</option>
                <option value="1ere_TI">1ère TI (Technologies de l'Information)</option>
                <option value="1ere_S">1ère S (Scientifique)</option>
                <option value="1ere_STI2D">1ère STI2D</option>
                <option value="Autre">Autre (préciser en remarque)</option>
            </select>
        </div>

        <div class="form-group">
            <label for="moyenne_premiere">Moyenne annuelle de Première (/20) *</label>
            <input type="number" id="moyenne_premiere" name="moyenne_premiere" required
                step="0.01" min="0" max="20" placeholder="Ex : 13.75">
        </div>
    </form>

```

```

<div class="form-group">
  <label for="date_naissance">Date de naissance *</label>
  <input type="date" id="date_naissance" name="date_naissance" required
    max="<?= date('Y-m-d') ?>" > <!-- Ne pas autoriser dates futures -->
</div>

<div class="form-group">
  <label for="photo_identite">Photo d'identité (JPG/PNG, max 2 Mo) *</label>
  <input type="file" id="photo_identite" name="photo_identite" required
    accept="image/jpeg,image/png">
  <small>Format passeport, fond clair recommandé.</small>
</div>

<button type="submit">Valider ma candidature</button>
</form>
</section>

<?php
// Inclusion du pied de page commun
include 'footer.php';
?>

```

### Étape 5 – traitement.php (cœur métier)

```

<?php
// traitement.php – Traitement du formulaire et affichage de la décision

// 1. CONSTANTES MÉTIER (accessibles globalement dans ce script)
define('SEUIL_MOYENNE', 12.0);
define('AGE_MAX', 20);
define('MAX_FILE_SIZE', 2 * 1024 * 1024); // 2 Mo en octets
define('ALLOWED_MIME', ['image/jpeg', 'image/png']);

// 2. FONCTIONS MÉTIER (déclarées avant tout appel)
/**
 * Calcule l'âge en années révolues à partir d'une date Y-m-d
 * @param string $dateNaissance Format 'YYYY-MM-DD'
 * @return int Âge en années
 */
function calculerAge(string $dateNaissance) : int {
    $aujourd'hui = new DateTime(); // Date du serveur
    $naissance = new DateTime($dateNaissance);
    $diff = $aujourd'hui->diff($naissance);
    return $diff->y; // Propriété 'y' = années entières
}

/**
 * Décide l'admission selon la règle : Moyenne >= SEUIL ET Age <= AGE_MAX
 * @param float $moyenne
 * @param int $age
 * @return string 'Admis' ou 'Ajourné'
 */
function decider(float $moyenne, int $age) : string {
    // Opérateurs de comparaison (>=, <=) et logique (&&)
    if ($moyenne >= SEUIL_MOYENNE && $age <= AGE_MAX) {
        return 'Admis';
    }
    return 'Ajourné';
}

```

```
}

// 3. VÉRIFICATION MÉTHODE HTTP (Sécurité : on ne traite que du POST)
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    // Redirection propre vers le formulaire si accès direct (GET)
    header('Location : formulaire.php');
    exit; // Arrêt impératif après header()
}

// 4. RÉCUPÉRATION ET NETTOYAGE DES DONNÉES $_POST
// On initialise un tableau d'erreurs pour collecte
$erreurs = [];

// Nom complet
$nom = isset($_POST['nom_complet']) ? trim($_POST['nom_complet']) : '';
if ($nom === '') {
    $erreurs[] = "Le nom complet est obligatoire.";
}

// Classe d'origine
$classe = isset($_POST['classe_origine']) ? trim($_POST['classe_origine']) : '';
if ($classe === '') {
    $erreurs[] = "Veuillez sélectionner une classe d'origine.";
}

// Moyenne : validation numérique + intervalle
$moyenneBrute = isset($_POST['moyenne_premiere']) ? trim($_POST['moyenne_premiere']) : '';
if ($moyenneBrute === '' || !is_numeric($moyenneBrute)) {
    $erreurs[] = "La moyenne doit être un nombre valide.";
} else {
    $moyenne = (float)$moyenneBrute; // Transtypage explicite
    if ($moyenne < 0 || $moyenne > 20) {
        $erreurs[] = "La moyenne doit être comprise entre 0 et 20.";
    }
}

// Date de naissance
$dateNaissance = isset($_POST['date_naissance']) ? trim($_POST['date_naissance']) : '';
$age = null;
if ($dateNaissance === '') {
    $erreurs[] = "La date de naissance est obligatoire.";
} else {
    // Vérification format valide et date passée
    $ts = strtotime($dateNaissance);
    if ($ts === false || $ts > time()) {
        $erreurs[] = "Date de naissance invalide ou future.";
    } else {
        $age = calculerAge($dateNaissance);
    }
}

// 5. GESTION DU FICHIER $_FILES (photo)
$photoBase64 = ''; // Contendra l'image encodée pour affichage direct
$photoMime = '';

if (!isset($_FILES['photo_identite']) || $_FILES['photo_identite']['error'] !== UPLOAD_ERR_OK) {
    $erreurs[] = "La photo d'identité est obligatoire et doit être envoyée correctement.";
}
```

```

} else {
    $fichier = $_FILES['photo_identite'];
    // Vérification taille
    if ($fichier['size'] > MAX_FILE_SIZE) {
        $erreurs[] = "La photo dépasse 2 Mo (taille max autorisée).";
    }
    // Vérification type MIME réel (plus sûr que l'extension)
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mimeReel = finfo_file($finfo, $fichier['tmp_name']);
    finfo_close($finfo);

    if (!in_array($mimeReel, ALLOWED_MIME)){
        $erreurs[] = "Format d'image non autorisé. Utilisez JPG ou PNG uniquement.";
    } else {
        // Lecture binaire et encodage Base64 pour balise 
        $contenuBinaire = file_get_contents($fichier['tmp_name']);
        if ($contenuBinaire !== false) {
            $photoBase64 = base64_encode($contenuBinaire);
            $photoMime = $mimeReel;
        } else {
            $erreurs[] = "Impossible de lire le fichier temporaire.";
        }
    }
}
}

```

```
// 6. SI ERREURS : AFFICHAGE ET ARRÊT
```

```

if (!empty($erreurs)) {
    include 'header.php';
    echo '<div class="alert alert-danger">';
    echo '<strong>Erreurs de validation :</strong><ul style="text-align:left;margin:10px 0;">';
    foreach ($erreurs as $err) {
        echo "<li>" . htmlspecialchars($err) . "</li>";
    }
    echo '</ul><a href="formulaire.php" class="btn-retour">&laquo; Retour au formulaire</a></div>';
    include 'footer.php';
    exit;
}

```

```
// 7. TOUT EST VALIDE : CONSTRUCTION TABLEAU ASSOCIATIF CANDIDAT
```

```

$candidat = [
    'nom' => $nom,
    'classe' => $classe,
    'moyenne' => $moyenne,
    'date_naissance' => $dateNaissance,
    'age' => $age,
    'photo_base64' => $photoBase64,
    'photo_mime' => $photoMime,
    'decision' => decider($moyenne, $age) // Appel fonction métier
];

```

```
// 8. AFFICHAGE FICHE RÉCAPITULATIVE
```

```

include 'header.php';
?>

<section class="result-section">
    <h2>Fiche de décision – Admission Terminale T1</h2>
    <p class="lead">Candidat : <strong><? = htmlspecialchars($candidat['nom']) ?></strong></p>

```

```

<table class="recap">
  <tbody>
    <tr>
      <th>Nom complet</th>
      <td><?= htmlspecialchars($candidat['nom']) ?></td>
    </tr>
    <tr>
      <th>Classe d'origine</th>
      <td><?= htmlspecialchars($candidat['classe']) ?></td>
    </tr>
    <tr>
      <th>Moyenne de Première</th>
      <td><?= number_format($candidat['moyenne'], 2, ',', ' ') ?> / 20</td>
    </tr>
    <tr>
      <th>Date de naissance</th>
      <td><?= date('d/m/Y', strtotime($candidat['date_naissance'])) ?></td>
    </tr>
    <tr>
      <th>Âge (aujourd'hui)</th>
      <td><?= $candidat['age'] ?> an(s)</td>
    </tr>
    <tr>
      <th>Photo d'identité</th>
      <td>
        <?php if ($candidat['photo_base64']) : ?>
          
          <?php else : ?>
            <em>Non fournie</em>
          <?php endif; ?>
        </td>
      </tr>
    <tr class="decision-row <?= strtolower($candidat['decision']) ?>">
      <th>DÉCISION</th>
      <td><?= $candidat['decision'] ?></td>
    </tr>
  </tbody>
</table>

<div class="actions" style="margin-top : 30px; text-align : center;">
  <a href="formulaire.php" class="btn-retour" style="display : inline-block; background : #6c757d; color : white; padding : 10px 20px; border-radius : 4px; text-decoration : none;">Nouvelle candidature</a>
</div>
</section>

<?php
include 'footer.php';
?>

```

Justifications techniques clés (pour l'oral) :

1. **Séparation responsabilités** : header/footer factorisent le HTML commun (DRY – Don't Repeat Yourself). traitement.php ne contient que la logique PHP + l'affichage résul-

tat.

2. **Sécurité entrées** : `trim()` supprime espaces parasites; `htmlspecialchars()` à l'affichage prévient les failles XSS (injection HTML/JS); `isset()` évite les `Notice: Undefined index`.
3. **Validation fichier robuste** : On ne fait pas confiance à `$_FILES['...']['type']` (fourni par le client, falsifiable). On utilise `finfo_file()` sur le fichier temporaire pour détecter le vrai type MIME. Taille vérifiée côté serveur (`MAX_FILE_SIZE` côté client est indicatif seulement).
4. **Constantes vs variables** : `SEUIL_MOYENNE` et `AGE_MAX` sont des `define()` : immuables, globales, lisibles. Pas de "magic numbers" dans le code.
5. **Fonction pure `decider`** : Ne fait qu'un calcul, sans effet de bord (pas d'écho, pas de `$_POST`). Facile à tester unitairement.
6. **Encodage Base64 image** : Permet d'afficher la photo immédiatement sans écrire sur le disque (pas de gestion droits, pas de nettoyage, pas de risque d'écrasement). Adapté pour un TP ; en production on stockerait en BDD ou dossier sécurisé.
7. **Redirection `header()` + `exit()`** : Indispensable après `header('Location:...')` pour arrêter l'exécution du script courant.

Tableau de tests de non-régression (à présenter) :

| Cas de test      | Entrées                                                                 | Résultat attendu                                 | Observation                                                  |
|------------------|-------------------------------------------------------------------------|--------------------------------------------------|--------------------------------------------------------------|
| Nominal Admis    | Nom : "A", Classe : 1ère TI, Moy : 14.5, Naiss : 2007-01-01, Photo : OK | Admis (Vert)                                     | Règle métier respectée                                       |
| Seuil exact      | Moy : 12.0, Age : 20                                                    | Admis                                            | Opérateur <code>&gt;=</code> inclusif                        |
| Seuil manqué     | Moy : 11.99, Age : 18                                                   | Ajourné (Rouge)                                  | Précision float respectée                                    |
| Âge dépassé      | Moy : 15, Age : 21                                                      | Ajourné                                          | Condition <code>age &lt;= 20</code> stricte                  |
| Champ vide (Nom) | Nom : "", autres OK                                                     | Erreur validation + retour formulaire            | <code>isset + trim + empty</code>                            |
| Accès GET direct | URL : <code>/traitement.php</code>                                      | Redirection 302 vers <code>formulaire.php</code> | <code>\$_SERVER['REQUEST_METHOD']</code>                     |
| Fichier PDF      | Photo : <code>test.pdf</code>                                           | Erreur "Format non autorisé"                     | <code>finfo_file</code> détecte <code>application/pdf</code> |
| Fichier > 2 Mo   | Photo : 3 Mo                                                            | Erreur "Dépasse 2 Mo"                            | <code>\$_FILES['size']</code>                                |

## 8.6 Bilan de l'activité

Cette activité d'intégration a permis de mettre en évidence les points suivants, essentiels pour le Baccalauréat et la pratique professionnelle :

1. **Architecture modulaire** : L'utilisation de `include` pour `header.php` et `footer.php` illustre le principe de séparation des responsabilités (structure vs logique) et la réutilisabilité du code, fondements de tout framework MVC moderne (Laravel, Symfony, CodeIgniter).
2. **Cycle requête/réponse HTTP** : Vous avez manipulé concrètement la différence entre GET (affichage formulaire) et POST (traitement données), l'importance de `enctype="multipart/form-data"` pour les fichiers, et la redirection `header()` pour éviter la resoumission au rafraîchissement (pattern PRG – Post/Redirect/Get).
3. **Sécurité des entrées (Sanitization/Validation)** : L'usage systématique de `isset()`, `trim()`, `htmlspecialchars()`, `is_numeric()`, et la vérification du type MIME réel via `finfo_file()` sont des réflexes indispensables pour prévenir les failles XSS, l'injection de code, et l'upload de fichiers malveillants (webshells).
4. **Algorithmique appliquée** : La fonction `decider()` traduit une règle de gestion métier en opérateurs de comparaison (`>=`, `<=`) et logiques (`&&`). La gestion du seuil inclusif (12.0 exactement) et du calcul d'âge précis (`DateTime::diff`) montre la rigueur attendue sur les conditions aux limites.
5. **Structures de données PHP** : Le tableau associatif `$candidat` sert de DTO (Data Transfer Object) entre la couche traitement et la couche vue, organisant les données hétérogènes (chaînes, float, int, binaire encodé) de façon lisible.
6. **Gestion des fichiers uploadés** : Vous avez compris que `$_FILES['...']['tmp_name']` pointe vers un fichier temporaire effacé à la fin du script, d'où la nécessité de le lire (`file_get_contents`) ou déplacer (`move_uploaded_file`) immédiatement. L'encodage Base64 pour affichage direct est une astuce pédagogique ; en production, on stockerait le fichier hors racine web et on enregistrerait son chemin en base de données.
7. **Débogage et tests** : La grille de tests de non-régression (cas nominaux, limites, erreurs) est la marque d'un développeur professionnel. Elle valide la robustesse du code face aux entrées utilisateur imprévisibles.

### Compétences certifiées (référentiel MINESEC) :

- **Savoir-faire** : Insérer et tester un script PHP dans une page web ; Utiliser les variables superglobales `$_GET`, `$_POST`, `$_FILES` ; Déclarer une variable, une constante, un tableau et une fonction PHP ; Récupérer et afficher les valeurs d'un formulaire HTML depuis un fichier PHP ; Utiliser les opérateurs arithmétiques, de comparaison, logiques et d'assignation.
- **Savoir** : Exécution côté serveur ; Structure d'un script PHP ; Variables superglobales ; Fonctions d'affichage ; Opérateurs.
- **Savoir-être** : Rigueur dans la validation des données ; Esprit critique face aux cas limites ; Travail collaboratif (binôme/trinôme) ; Communication technique lors de la présentation.

### Pistes d'extension (projet personnel / club coding) :

- Stocker les candidatures dans une base de données MySQL (table `candidats`) via PDO (préparation requêtes, transactions).
- Ajouter une authentification administrateur (session PHP, `password_hash/verify`) pour lister/télécharger les dossiers.
- Générer un PDF de convocation pour les admis (bibliothèque `tcpdf` ou `dompdf`).
- Déployer sur un VPS (DigitalOcean, OVH) avec Nginx + PHP-FPM + HTTPS (Let's Encrypt) et configurer un nom de domaine `admission.lyceenkolbisson.cm`.

Félicitations ! Vous avez maintenant une base solide pour aborder les frameworks PHP modernes et les API REST en BTS/DUT/Université. Continuez à coder, à lire la documentation officielle (`php.net`), et à sécuriser chaque entrée utilisateur. **Le code propre est du code qui protège l'utilisateur et le système.**

## 9 MÉTHODES ET EXERCICES RÉSOLUS

## 10 PROGRAMMATION DYNAMIQUE CÔTÉ SERVEUR AVEC PHP

### 10.1 Exécution côté serveur et insertion de PHP dans HTML

#### DÉFINITION Exécution côté serveur (PHP)

Contrairement aux langages côté client (HTML, CSS, JavaScript) interprétés par le navigateur, un script PHP (Hypertext Preprocessor) est exécuté exclusivement sur le serveur web (Apache, Nginx, etc.) avant l'envoi de la réponse HTTP. Le serveur repère l'extension `.php`, transmet le fichier à l'interpréteur PHP (Zend Engine), qui exécute le code entre les balises `<?php ... ?>` et remplace ce bloc par le résultat (généralement du HTML). Le navigateur ne reçoit jamais le code source PHP, seulement le HTML généré.

#### PROPRIÉTÉ Insertion de code PHP dans un document HTML

- Extension obligatoire : le fichier doit porter l'extension `.php` (ex. `index.php`), sans quoi le serveur ne délègue pas l'interprétation à PHP.
- Balises de délimitation : `<?php ... ?>` (forme canonique, toujours activée). La forme courte `<?= ... ?>` (équivalent à `<?php echo ... ?>`) est autorisée pour l'affichage direct si `short_open_tag=On`.
- Emplacement : le bloc PHP peut être inséré n'importe où dans le flux HTML (dans `<head>`, `<body>`, au milieu d'un attribut, etc.).
- Sortie : les instructions `echo`, `print`, `print_r` envoient du texte au tampon de sortie ; ce texte peut contenir des balises HTML qui seront interprétées par le navigateur.

#### MÉTHODE Insérer et tester un script PHP dans une page web

1. Vérifier l'environnement : un script PHP ne s'exécute que sur un serveur. Installer un serveur local (WampServer, XAMPP, MAMP) et placer le fichier dans le dossier racine (`www` pour WampServer, `htdocs` pour XAMPP).
2. Nommer le fichier avec l'extension `.php` (jamais `.html`), sinon le serveur n'interprète

pas le code PHP.

3. Délimiter le code PHP avec les balises `<?php ... ?>`, insérées n'importe où dans le document HTML.
4. Produire l'affichage avec `echo` (ou `print`) ; le texte renvoyé peut contenir des balises HTML de formatage (`<b>`, `<br>`, `<span>`, etc.).
5. Tester via le serveur : taper dans le navigateur `http://localhost/mapage.php` (ou `http://127.0.0.1/mapage.php`) et jamais ouvrir le fichier par double-clic (protocole `file:///` : le PHP n'est pas exécuté, le code source s'affiche en clair).
6. Observer le code source reçu (clic droit → « Afficher le code source de la page ») : on ne doit voir que du HTML, preuve que le PHP a été exécuté côté serveur.

Réflexes : chaque instruction PHP se termine par un point-virgule ; ; PHP est sensible à la casse pour les noms de variables (`$var` ≠ `$Var`) mais insensible pour les noms de fonctions (`echo` ≡ `ECHO`) ; une erreur de syntaxe produit une page blanche ou un message `Parse error` (activer `display_errors=On` en développement).

**EXERCICE RÉSOLU 8****Première page dynamique — Lycée de Nkongsamba**

Énoncé. Le site du lycée de Nkongsamba doit afficher un message de bienvenue contenant la date du jour. Écrire une page `accueil.php` qui affiche « Bienvenue au Lycée de Nkongsamba » en gras, suivi de la date courante au format `jj/mm/aaaa` (fonction `date("d/m/Y")`). Expliquer pourquoi le code PHP n'apparaît pas dans le code source reçu par le navigateur.

**SOLUTION**

Solution. Le fichier est enregistré sous `accueil.php` dans le dossier `www` du serveur local, puis appelé par `http://localhost/accueil.php`.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Accueil – Lycée de Nkongsamba</title>
</head>
<body>
  <h1>Portail du lycée</h1>
  <?php
    // echo renvoie du texte (ici avec balises HTML) au navigateur
    echo "<b>Bienvenue au Lycée de Nkongsamba</b><br>";
    echo "Nous sommes le " . date("d/m/Y");
  ?>
</body>
</html>
```

Raisonnement. À la réception de la requête HTTP, le serveur repère l'extension `.php` et transmet le fichier à l'interpréteur PHP. Celui-ci exécute uniquement le code situé entre `<?php` et `?>` : les deux `echo` produisent du texte HTML qui remplace le bloc PHP dans la réponse HTTP. Le navigateur reçoit donc une page 100 % HTML :

```
<h1>Portail du lycée</h1>
<b>Bienvenue au Lycée de Nkongsamba</b><br>
Nous sommes le 14/05/2025
```

Conclusion. Le code PHP n'apparaît jamais dans le code source reçu car il est exécuté côté serveur : seul son résultat (du HTML) est envoyé au client. C'est le principe même de la programmation dynamique côté serveur.

#### À RETENIR Point clé : cycle de vie d'une page PHP

1. Requête HTTP du navigateur vers `http://localhost/page.php`.
2. Serveur web (Apache) détecte `.php` → appelle l'interpréteur PHP.
3. Interpréteur exécute le code PHP, génère du HTML (sortie standard).
4. Serveur renvoie la réponse HTTP (HTML pur) au navigateur.
5. Navigateur affiche le HTML (rendement CSS, exécution JS).

## 10.2 Structure d'un script PHP : commentaires et inclusion de fichiers

#### PROPRIÉTÉ Commentaires en PHP

Trois syntaxes coexistent :

- `//` Commentaire sur une ligne (style C++).
- `#` Commentaire sur une ligne (style Shell/Perl, moins courant).
- `/*` Commentaire sur plusieurs lignes `*/` (style C, ne s'imbrique pas).

Les commentaires sont ignorés par l'interpréteur et ne figurent pas dans le HTML envoyé au client. Ils sont indispensables pour documenter la logique métier, les algorithmes complexes et les paramètres de fonctions.

#### PROPRIÉTÉ Inclusion de fichiers PHP

Permet de factoriser le code (en-têtes, menus, pied de page, configuration, biblio-

thèques de fonctions) et d'éviter la duplication.

| Instruction                              | Comportement si fichier absent                      | Cas d'usage typique                                            |
|------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------|
| <code>include "fichier.php";</code>      | Avertissement (E_WARNING), le script continue       | Éléments d'interface (header, footer, menu)                    |
| <code>require "fichier.php";</code>      | Erreur fatale (E_COMPILE_ERROR), le script s'arrête | Fichiers critiques (connexion BDD, config, fonctions vitales)  |
| <code>include_once / require_once</code> | Identique mais n'inclut qu'une seule fois           | Évite les redéclarations de fonctions/classes (ex. config.php) |

**Syntaxe :** le chemin peut être relatif (`include "inc/header.php";`) ou absolu (`require __DIR__ . "/config.php";`). La constante magique `__DIR__` donne le dossier du script en cours d'exécution, fiable quel que soit le script appelant.

#### EXEMPLE Structure modulaire typique (projet camerounais)

Arborescence :

```

projet/≡≡≡
index.php≡≡≡
config.php      // Connexion BDD, constantes globales≡≡≡
functions.php   // Bibliothèque de fonctions métier≡≡≡
includes/≡
  ≡≡≡ header.php    // <head>, <nav>, ouverture <main>≡
  ≡≡≡ footer.php    // Fermeture </main>, <footer>, scripts JS≡≡≡
uploads/        // Dossier d'upload (droits écriture)

```

`index.php` :

```

<?php
require_once __DIR__ . "/config.php"; // 1. Config critique (arrête si absent)
require_once __DIR__ . "/functions.php"; // 2. Fonctions dispo partout
include __DIR__ . "/includes/header.php"; // 3. En-tête HTML (avertissement si absent)

// Contenu dynamique spécifique à la page d'accueil
echo "<h1>Bienvenue sur la plateforme</h1>";
echo "<p>Date du jour : " . date("d/m/Y") . "</p>";

include __DIR__ . "/includes/footer.php"; // 4. Pied de page
?>

```

### 10.3 Affichage, variables, constantes et tableaux

#### DÉFINITION Variables en PHP

Une variable commence obligatoirement par le symbole \$, suivi d'une lettre (A-Z, a-z) ou d'un underscore \_, puis de lettres, chiffres ou underscores. Pas d'espace, pas de chiffre en première position. PHP est faiblement typé : le type (entier `int`, flottant `float`, chaîne `string`, booléen `bool`, tableau `array`, objet, `null`) est déduit de la valeur affectée et peut changer au cours de l'exécution.

```
$nom = "Amina";    // string
$age = 18;         // int
$moyenne = 14.5;   // float
$inscrit = true;   // bool
$notes = [12, 15, 9]; // array (indice 0, 1, 2)
$var = null;       // null
```

#### DÉFINITION Constantes en PHP

Une constante est un identifiant (sans \$) associé à une valeur immuable pendant toute l'exécution du script. Par convention, son nom est en majuscules (SNAKE\_CASE). Deux syntaxes :

- `define("NOM", valeur);` — fonction, évaluée à l'exécution, portée globale, sensible à la casse (par défaut).
- `const NOM = valeur;` — structure de langage, évaluée à la compilation, portée selon le contexte (globale ou classe), toujours sensible à la casse.

Utilisation : `echo TAUX_TVA;` (sans \$). Une fois définie, toute tentative de redéfinition ou modification provoque une erreur.

#### PROPRIÉTÉ Tableaux en PHP

Un tableau PHP est une map ordonnée (table de hachage) associant des clés (entiers ou chaînes) à des valeurs (tout type). Deux syntaxes équivalentes : `array(...)` et `[...]` (courte, PHP 5.4+).

- **Tableau indexé** : clés entières automatiques à partir de 0. `$notes = [12, 15, 9];` → `$notes[0]==12, $notes[1]==15.`
- **Tableau associatif** : clés chaînes explicites (paires `cle => valeur`). `$eleve = ["nom" => "Kanga", "classe" => "Tle TI", "notes" => [14, 16]];` **Accès** : `$eleve["nom"], $eleve["notes"][0].`
- **Parcours** : `foreach ($tab as $cle => $val) { ... }` (recommandé) ou `for` sur `in-`

dices.

#### PROPRIÉTÉ Fonctions d’affichage et d’inspection

- `echo $var1, $var2, ...;` — structure de langage, pas de retour, accepte plusieurs arguments séparés par des virgules (plus rapide), ne formate pas les tableaux/objets.
- `print $var;` — fonction, retourne toujours 1, un seul argument.
- `print_r($var, $return=false);` — affiche une représentation lisible d’un tableau/objet (clés/valeurs). Si `$return=true`, renvoie la chaîne au lieu d’afficher.
- `var_dump($var);` — affiche type + valeur + taille (débugage complet).

**Piège :** `echo $tableau;` lève une notice Array to string conversion et affiche « Array ». Utiliser `print_r` ou `var_dump`.

#### MÉTHODE Déclarer une variable, une constante, un tableau PHP

1. **Variable :** `$nom = "valeur";` (nom commence par lettre/\_, sensible à la casse).
2. **Constante :** `define("NOM_CONSTANTE", valeur);` ou `const NOM_CONSTANTE = valeur;` (majuscules, sans \$, immuable).
3. **Tableau indexé :** `$tab = [val1, val2, ...];` — indices 0, 1, 2...
4. **Tableau associatif :** `$tab = ["cle1" => val1, "cle2" => val2];` — accès `$tab["cle1"]`.
5. **Affichage :** `echo` pour scalaires; `print_r($tab)` ou `var_dump($tab)` pour inspecter un tableau.

#### EXERCICE RÉSOLU 9 Gestion des frais de scolarité — École privée de Bafoussam

**Énoncé.** Une école privée de Bafoussam fixe la scolarité annuelle à 150 000 FCFA, majorée de 10 % de frais d’inscription. Écrire un script PHP qui : déclare la scolarité comme constante ; calcule le montant total dans une variable ; stocke dans un tableau associatif le nom d’un élève (« Kamga Brice »), sa classe (« Tle TI ») et le total ; affiche une phrase récapitulative.

#### SOLUTION

**Solution.** La scolarité étant fixe pour tous, on la déclare en constante avec `define`. Le total

est un calcul : il va dans une variable. Les informations hétérogènes de l'élève se prêtent à un tableau associatif.

```
<?php
define("SCOLARITE", 150000); // constante, sans $
$total = SCOLARITE * 1.10; // majoration de 10 % (float)
$eleve = [
    "nom" => "Kamga Brice",
    "classe" => "Tle TI",
    "total" => $total
];
// Concaténation avec le point .
echo $eleve["nom"] . " (" . $eleve["classe"] . ") doit payer "
    . $eleve["total"] . " FCFA.";
?>
```

Calcul.  $total = 150\,000 \times 1,10 = 165\,000$  FCFA. L'affichage produit : « Kamga Brice (Tle TI) doit payer 165000 FCFA. »

Conclusion. On utilise `define` pour une valeur fixe (constante sans \$), une variable préfixée de \$ pour un résultat de calcul, et un tableau associatif `cle => valeur` pour regrouper des données nommées.

**À RETENIR****Bonnes pratiques variables / constantes**

- Noms explicites en `camelCase` pour variables (`$montantTotal`), `SNAKE_CASE` majuscules pour constantes (`TAUX_TVA`).
- Initialiser toujours une variable avant utilisation (évite Undefined variable).
- Préférer `const` pour les constantes globales simples, `define` si le nom est dynamique ou conditionnel.
- Pour l'argent : stocker en centimes (entier) ou utiliser `BCMath` pour éviter les erreurs de virgule flottante (ex.  $0.1 + 0.2 \neq 0.3$ ).

## 10.4 Fonctions PHP et opérateurs

**PROPRIÉTÉ****Opérateurs en PHP**

| Catégorie     | Opérateurs                                             | Remarques                                                                                                   |
|---------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Arithmétiques | + - * / Comparaison & \verb== === != !== < > <= >= <=> | == valeur seule,<br>=== va-<br>leur et<br>type;<br><=> opé-<br>rateur<br>vaisseau<br>spatial<br>(PHP 7+)    |
| Logiques      | && ! and or xor                                        | && et<br>   ont<br>priorité<br>haute;<br>and/or<br>priorité<br>basse<br>(éviter<br>dans<br>condi-<br>tions) |
| Assignation   | = += -= *= /= Chaîne & \verb.=                         | Concaténation<br>(point),<br>pas +                                                                          |
| Tableau       | + == === != !==                                        | + = union<br>(clés de<br>droite<br>ignorées<br>si déjà<br>pré-<br>sentes à<br>gauche)                       |

Priorités utiles : ! > arithmétiques > comparaison > && > || > = / +=... Parenthèses ( ) pour forcer l'ordre.

**MÉTHODE**

Utiliser les opérateurs arithmétiques, de comparaison, logiques et d'assignation

1. Arithmétiques : +, -, \*, /, % (modulo), \*\* (puissance). Attention : division par zéro →

DivisionByZeroError.

2. Comparaison : `==` (égalité valeur après conversion), `===` (égalité valeur et type — recommandé), `!=`, `!==`, `<`, `>`, `<=`, `>=`. Résultat : booléen.
3. Logiques : `&&` (ET), `||` (OU), `!` (NON). Court-circuit : `&&` s'arrête au premier faux, `||` au premier vrai.
4. Assignment : `=` puis raccourcis `+=`, `-=`, `*=`, `/=`, `.=` (concaténation).
5. Réflexes Bac : ne jamais confondre `=` (affectation) et `==` (comparaison); utiliser `% 2 === 0` pour tester la parité; concaténer avec `.` (et non `+`); privilégier `===` et `!==` pour éviter les conversions implicites pièges (ex. `"0" == false` est `true`).

## EXERCICE RÉSOLU 10

## Admissibilité au concours de l'ENSET Douala

Énoncé. Un candidat est admissible au concours de l'ENSET Douala si sa moyenne est supérieure ou égale à 12 et son âge inférieur ou égal à 25 ans. Écrire un script qui, à partir de `$moyenne = 13.5` et `$age = 19`, affiche « Admissible » ou « Non admissible ». On utilisera un opérateur logique et un opérateur de comparaison.

## SOLUTION

Solution. La condition combine deux exigences simultanées : c'est un ET logique (`&&`) entre deux comparaisons.

```
<?php
    $moyenne = 13.5;
    $age = 19;
    if ($moyenne >= 12 && $age <= 25) {
        echo "Admissible";
    } else {
        echo "Non admissible";
    }
?>
```

Vérification pas à pas.  $13,5 \geq 12$  est vrai;  $19 \leq 25$  est vrai; donc `vrai && vrai = vrai` : le bloc `if` s'exécute et affiche « Admissible ».

Contre-exemple. Avec `$moyenne = 11`, la première comparaison est fausse, donc toute la condition `&&` est fausse (court-circuit : le second terme n'est même pas évalué).

Conclusion. Le script affiche « Admissible ». La combinaison `>= + && + <=` traduit fidèlement l'énoncé « au moins 12 ET au plus 25 ans ».

## PROPRIÉTÉ

## Déclaration et appel de fonction en PHP

- **Syntaxe** : `function nomFonction($param1, $param2 = valeurDefaut) { ... return $resultat; }`.
- **Portée des variables** : les variables déclarées dans la fonction sont locales (invisibles à l'extérieur). Les variables extérieures ne sont pas accessibles à l'intérieur sauf : passage en paramètre, mot-clé `global $var` (déconseillé), ou tableau `$GLOBALS['var']`.
- **Retour** : `return` termine la fonction immédiatement et renvoie la valeur. Sans `return` (ou `return;`), la fonction renvoie `null`.
- **Arguments** : passage par valeur par défaut (copie). Passage par référence avec `&` dans la définition : `function modifier(& $x) { $x = 10; }`.
- **Nommage** : insensible à la casse (`maFonction()`  $\equiv$  `mafonction()`), convention `camelCase`.

#### MÉTHODE Déclarer et appeler une fonction

1. Déclarer avec le mot-clé `function`, un nom explicite, des paramètres entre parenthèses : `function nomFonction($p1, $p2) { ... }`.
2. Renvoyer un résultat avec `return` (interrompt la fonction); sans `return`, renvoie `null`.
3. Appeler la fonction par son nom suivi des arguments : `$r = nomFonction(3, 4);`.
4. Portée : variable locale à la fonction; variables globales inaccessibles sans `global` (à éviter).
5. Réflexe Bac : une fonction = une seule tâche; testée avec plusieurs jeux de valeurs; l'appelant décide de l'affichage (`echo` hors fonction si possible).

#### EXERCICE RÉSOLU 11 Conversion FCFA vers euros — Startup Yaoundé

Énoncé. Une startup de Yaoundé paie ses prestataires en FCFA mais facture un client français en euros (1 euro = 655,957 FCFA). Écrire une fonction `fcfaVersEuro($montant)` qui renvoie l'équivalent en euros arrondi à 2 décimales (fonction `round($x, 2)`), puis afficher la conversion de 100 000 FCFA.

#### SOLUTION

Solution. Le taux étant fixe, on le déclare en constante; la fonction isole le calcul pour être réutilisable.

```
<?php
define("TAUX_FCFA_EUR", 655.957); // 1 euro = 655,957 FCFA

function fcfaVersEuro($montant){
```

```
$euros = $montant / TAUX_FCFA_EUR; // variable locale
return round($euros, 2);           // arrondi à 2 décimales
}

$montant = 100000;
echo $montant . " FCFA = " . fcfaVersEuro($montant) . " euros";
?>
```

Calcul.  $100\,000 \div 655,957 \approx 152,4493\dots$  puis `round(..., 2)` donne 152,45.

Conclusion. Le script affiche « 100000 FCFA = 152.45 euros ». La fonction encapsule le calcul : il suffit de changer la constante `TAUX_FCFA_EUR` pour mettre à jour toute l'application.

#### LESAVAIS-TU ? PHP dans l'écosystème camerounais

PHP alimente une grande partie du web camerounais : sites d'administration (`impots.cm`, `cnps.cm`, portails universitaires), plateformes e-commerce (Jumia CM, Cdiscount CM), API de paiement mobile (MTN MoMo API, Orange Money API — SDK PHP officiels), systèmes de gestion scolaire (notes, inscriptions). La maîtrise de PHP côté serveur est une compétence directement employable dans les ESN (Entreprises de Services du Numérique) de Douala et Yaoundé.

## 10.5 Variables superglobales et récupération des données d'un formulaire

### DÉFINITION Variables superglobales PHP

Ce sont des tableaux associatifs disponibles partout (portée globale automatique) sans déclaration `global`. Les principales pour le web :

- `$_GET` : paramètres de l'URL (`?cle=valeur`), méthode GET du formulaire. Visibles, historisables, limités en longueur ( $\approx 2000$  car.), jamais pour données sensibles.
- `$_POST` : corps de la requête HTTP, méthode POST du formulaire. Non visibles dans l'URL, pas de limite pratique (config `post_max_size`), obligatoire pour mots de passe, uploads, données personnelles.
- `$_FILES` : fichiers envoyés via `<input type="file">` avec `enctype="multipart/form-data"`. Tableau structuré (voir méthode 6).
- `$_SERVER` : informations serveur/exécution (`REQUEST_METHOD`, `HTTP_USER_AGENT`, `REMOTE_ADDR`, `SCRIPT_NAME`, `QUERY_STRING`, etc.).
- `$_REQUEST` : fusion `$_GET` + `$_POST` + `$_COOKIE` (ordre configurable). Déconseillé : ambiguïté sur l'origine de la donnée.
- `$_COOKIE`, `$_SESSION` : gestion d'état (cours suivant).

**Sécurité :** Toutes les valeurs reçues sont des chaînes (`string`) et non fiables (contrôlées par l'utilisateur). Toujours valider, filtrer, convertir avant usage.

#### MÉTHODE Récupérer et afficher les valeurs d'un formulaire HTML (`$_GET` / `$_POST`)

1. Côté HTML : chaque champ `<input>`, `<select>`, `<textarea>` doit avoir un attribut `name` (c'est la clé d'accès dans le tableau superglobal). La balise `<form>` précise `action` (fichier PHP cible, vide = même page) et `method` (`get` ou `post`).
2. Choisir la méthode : `get` pour recherche, filtres, pagination (données non sensibles, partageables via URL) ; `post` pour authentification, inscription, modification de données, upload (confidentiel, volumineux, non idempotent).
3. Côté PHP : lire `$_GET["name"]` ou `$_POST["name"]` selon la méthode du formulaire.
4. Toujours tester l'existence avec `isset($_POST["champ"])` (ou `!empty(...)`) avant d'utiliser la valeur, pour éviter l'erreur `Undefined array key` (PHP 8+) / `Undefined index` (PHP < 8).
5. Sécuriser l'affichage (anti-XSS) : `htmlspecialchars($chaine, ENT_QUOTES, 'UTF-8')` neutralise `<`, `>`, `"`, `'`, `&` en entités HTML. Systématique pour toute donnée utilisateur ré-affichée.
6. Convertir si besoin : `(int)$val`, `(float)$val`, `filter_var($val, FILTER_VALIDATE_EMAIL)` etc. pour les calculs ou comparaisons.

#### EXERCICE RÉSOLU 12 Bulletin en ligne — Moyenne d'un élève (Collège de Buea)

Énoncé. Le formulaire suivant est proposé aux élèves :

```
<form action="moyenne.php" method="post">
  Note 1 : <input type="number" step="0.01" name="n1" required>
  Note 2 : <input type="number" step="0.01" name="n2" required>
  <input type="submit" value="Calculer">
</form>
```

Écrire le script `moyenne.php` qui récupère les deux notes, calcule la moyenne et affiche « Moyenne :  $X / 20$  », ou un message d'erreur si un champ est manquant ou invalide.

#### SOLUTION

Solution. La méthode est `post`, donc on lit `$_POST`. On teste l'existence et le remplissage avec `isset` et `!== ""`, puis on convertit en nombres avant le calcul. On sécurise l'affichage.

```

<?php
// moyenne.php
if (isset($_POST["n1"], $_POST["n2"])
    && $_POST["n1"] !== "" && $_POST["n2"] !== ""){

    // Conversion explicite chaîne -> float
    $n1 = (float) $_POST["n1"];
    $n2 = (float) $_POST["n2"];

    // Validation métier optionnelle : notes entre 0 et 20
    if ($n1 < 0 || $n1 > 20 || $n2 < 0 || $n2 > 20){
        echo "Erreur : les notes doivent être comprises entre 0 et 20.";
    } else {
        $moyenne = ($n1 + $n2) / 2;
        // htmlspecialchars sur la moyenne (float -> string auto) par rigueur
        echo "Moyenne : " . htmlspecialchars((string)$moyenne) . " / 20";
    }
} else {
    echo "Erreur : veuillez remplir les deux notes.";
}
?>

```

Raisonnement. Sans le test `isset`, un accès direct à `moyenne.php` (ou soumission partielle) déclencherait une erreur `Undefined array key`. Sans la conversion `(float)`, les valeurs resteraient des chaînes (l'addition + force la conversion numérique en PHP, mais c'est un réflexe indispensable pour la clarté et les comparaisons). Exemple : `n1 = 14` et `n2 = 16` donnent  $(14 + 16) / 2 = 15$ , affichage « Moyenne : 15 / 20 ».

Conclusion. La récupération suit toujours le même schéma : `method` du formulaire → `superglobale` correspondante → test `isset` / `empty` → `conversion` / `validation` → `traitement` → affichage sécurisé.

#### MÉTHODE Exploiter `$_FILES` pour l'upload de fichiers

1. Formulaire HTML : obligatoirement `method="post" + enctype="multipart/form-data" + champ <input type="file" name="monFichier">`. Sans `enctype`, `$_FILES` reste vide.
2. Structure de `$_FILES["monFichier"]` (tableau associatif) :
  - `name` : nom original du fichier côté client (ex. `photo.jpg`).
  - `type` : type MIME déclaré par le navigateur (non fiable, côté client).
  - `size` : taille en octets (entier).
  - `tmp_name` : chemin complet du fichier temporaire sur le serveur (ex. `/tmp/phpA1B2C3`). Supprimé à la fin du script s'il n'est pas déplacé.
  - `error` : code d'erreur (0 = `UPLOAD_ERR_OK`, 1 = `UPLOAD_ERR_INI_SIZE` > `upload_max_filesize`, 2 = `UPLOAD_ERR_FORM_SIZE` > `MAX_FILE_SIZE`, 3 = partiel, 4 = aucun fichier).

### 3. Validation côté serveur (indispensable) :

- `$_FILES["f"]["error"] === UPLOAD_ERR_OK (ou === 0).`
  - **Taille :** `$_FILES["f"]["size"] <= TAILLE_MAX.`
  - **Type/Extension :** vérifier l'extension (`pathinfo($name, PATHINFO_EXTENSION)`) et/ou le type MIME réel avec `finfo_file(finfo_open(FILEINFO_MIME_TYPE), $tmp_name)` (ne pas faire confiance à type).
  - **Nom de fichier :** sanitizer (supprimer caractères spéciaux, éviter ..) ou renommer (`ex.uniqid() . ".ext"`).
4. **Déplacer le fichier :** `move_uploaded_file($_FILES["f"]["tmp_name"], $destination)` retourne `true` si succès. Le dossier destination doit exister et être accessible en écriture par l'utilisateur du serveur web (souvent `www-data`).
5. **Commentaires :** `//` ou `#` (ligne), `/* ... */` (bloc). Documenter chaque bloc logique.
6. **Inclusion :** `include` (warning), `require` (fatal), `_once` (anti-double inclusion). Factoriser config, header, footer, fonctions.

#### EXERCICE RÉSOLU 13 Dépôt d'une photo de profil — Réseau social anciens élèves (Garoua)

**Énoncé.** Le formulaire d'upload : `<form action="upload.php" method="post" enctype="multipart/form-data"> <input type="file" name="photo"> <button>Envoyer</button> </form>`. Écrire `upload.php` qui : vérifie l'envoi sans erreur; refuse les fichiers > 500 000 octets; n'accepte que JPEG/PNG; déplace vers `uploads/`; affiche nom et taille du fichier reçu.

#### SOLUTION

**Solution.** On suit la séquence : vérification erreur → contrôle taille → contrôle type/extension → déplacement sécurisé → confirmation.

```
<?php
/* upload.php – Dépôt photo de profil */
const TAILLE_MAX = 500000; // 500 Ko
const EXT AUTORISEES = ['jpg', 'jpeg', 'png', 'webp'];
const DOSSIER_UPLOAD = __DIR__ . "/uploads/";

// 1. Vérification présence et absence d'erreur HTTP/PHP
if (!isset($_FILES["photo"]) || $_FILES["photo"]["error"] !== UPLOAD_ERR_OK) {
    $msg = "Aucun fichier reçu ou erreur de transfert (code : "
        . ($_FILES["photo"]["error"] ?? 'inconnu') . ").";
    echo htmlspecialchars($msg);
    exit;
}

$fichier = $_FILES["photo"];
```

```
// 2. Contrôle taille
if ($fichier["size"] > TAILLE_MAX) {
    echo "Fichier trop volumineux (max " . TAILLE_MAX . " octets).";
    exit;
}

// 3. Contrôle extension (basique) + type MIME réel (robuste)
$ext = strtolower(pathinfo($fichier["name"], PATHINFO_EXTENSION));
if (!in_array($ext, EXT AUTORISEES, true)) {
    echo "Extension non autorisée (jpg, png, webp uniquement).";
    exit;
}

// Vérification MIME réel via finfo (nécessite extension fileinfo, active par défaut)
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $fichier["tmp_name"]);
finfo_close($finfo);
$mimesOk = ['image/jpeg', 'image/png', 'image/webp'];
if (!in_array($mime, $mimesOk, true)) {
    echo "Type de fichier non reconnu (contenu invalide).";
    exit;
}

// 4. Nom de fichier sûr + déplacement
$nomSecurise = uniqid("profil_", true) . "." . $ext; // ex : profil_662f...jpg
$destination = DOSSIER_UPLOAD . $nomSecurise;

if (move_uploaded_file($fichier["tmp_name"], $destination)) {
    echo "Fichier " . htmlspecialchars($nomSecurise)
        . " (" . $fichier["size"] . " octets) reçu et stocké.";
} else {
    echo "Échec du déplacement (droits dossier uploads ?).";
}
?>
```

Raisonnement. Le fichier téléversé est d'abord stocké dans une zone temporaire (tmp\_name) : sans `move_uploaded_file`, il est détruit à la fin du script. Le test `error === UPLOAD_ERR_OK` garantit un transfert complet ; le contrôle de `size` protège le disque ; la vérification `finfo` empêche un fichier `.php` renommé en `.jpg` d'être exécuté si le dossier est mal configuré. `htmlspecialchars` sur le nom (même généré) est une habitude de défense en profondeur.

Conclusion. `$_FILES` centralise toutes les métadonnées ; la séquence « vérifier → contrôler (taille, type réel) → renommer → déplacer → confirmer » est le réflexe professionnel à reproduire au Bac et en entreprise.

#### ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier le `$` devant une variable ou le mettre devant une constante : on écrit `$note = 15` ; mais `define("TAUX", 18)` ; puis `TAUX` (sans `$`). C'est l'erreur de syntaxe la plus sanc-

tionnée.

2. Confondre `et ==` : `if ($age = 18)` affecte 18 et la condition est toujours vraie (valeur affectée = 18 = `truthy`). Une condition exige `==` (ou mieux `===`).
3. Lire `$_POST` alors que le formulaire est en `get` (ou l'inverse), et oublier `isset()` : la superglobale doit correspondre à l'attribut `method`, et tout accès non testé provoque un `Undefined array key (PHP 8)` / `Undefined index`.
4. Oublier le point-virgule ou le point de concaténation : chaque instruction se termine par `;`, et on assemble les chaînes avec `.` et non `+` ("`a`" + "`b`" vaut 0 en PHP avec alerte, pas "`ab`").
5. Ouvrir la page en `file://` au lieu de `http://localhost/...` ou nommer le fichier `.html` : le PHP n'est alors pas exécuté et le code source apparaît en clair. Toujours passer par le serveur et l'extension `.php`. Pour un upload, oublier `enctype="multipart/form-data"` rend `$_FILES` vide : vérifier ce trio d'attributs (`method="post"`, `enctype`, `name` sur l'input file).

## 10.6 Synthèse : mini-application dynamique complète

### EXPÉRIENCE

#### Mini-projet guidé : « Gestion des notes d'une classe »

**Objectif.** Réaliser une application web complète permettant de saisir les notes de 3 matières pour un élève, calculer sa moyenne, déterminer sa mention et afficher un bulletin récapitulatif. Mettre en œuvre : formulaire HTML5, traitement PHP (superglobales, variables, constantes, tableaux, fonctions, opérateurs, conditions), inclusion de fichiers, sécurisation basique (XSS).

**Matériel / Logiciels.** Serveur local (WampServer/XAMPP), éditeur de code (VS Code, Notepad++), navigateur web.

Arborescence attendue.

```
bulletin/===
index.php      // Formulaire de saisie===
config.php     // Constantes (coeff, seuils mentions)===
functions.php  // Fonctions métier (moyenne, mention)===
includes/
  === header.php // HTML head + nav
  === footer.php // Footer + scripts===
traitement.php // Traitement POST + affichage bulletin
```

#### Étape 1 — Configuration (`config.php`).

```
<?php
// config.php
define("COEFF_MATHS", 4);
define("COEFF_PHYSIQUE", 3);
define("COEFF_FRANCAIS", 3);
```

```
define("SEUIL_TRES_BIEN", 16);
define("SEUIL_BIEN", 14);
define("SEUIL_ASSEZ_BIEN", 12);
define("SEUIL_PASSABLE", 10);
?>
```

## Étape 2 — Fonctions métier (functions.php).

```
<?php
// functions.php
require_once __DIR__ . "/config.php";

/* *
 * Calcule la moyenne pondérée de 3 notes.
 * @param float $n1 Note Maths
 * @param float $n2 Note Physique
 * @param float $n3 Note Français
 * @return float Moyenne arrondie 2 décimales
 */
function calculerMoyenne($n1, $n2, $n3) {
    $sommeCoeff = COEFF_MATHS + COEFF_PHYSIQUE + COEFF_FRANCAIS;
    $sommePonderee = $n1 * COEFF_MATHS + $n2 * COEFF_PHYSIQUE + $n3 * COEFF_FRANCAIS;
    return round($sommePonderee / $sommeCoeff, 2);
}

/* *
 * Détermine la mention d'après la moyenne.
 * @param float $moyenne
 * @return string Mention
 */
function determinerMention($moyenne) {
    if ($moyenne >= SEUIL_TRES_BIEN) return "Très Bien";
    if ($moyenne >= SEUIL_BIEN) return "Bien";
    if ($moyenne >= SEUIL_ASSEZ_BIEN) return "Assez Bien";
    if ($moyenne >= SEUIL_PASSABLE) return "Passable";
    return "Insuffisant";
}

/* *
 * Affiche le bulletin HTML (séparation traitement/affichage).
 */
function afficherBulletin($eleve, $notes, $moyenne, $mention) {
    echo "<div class='bulletin'>";
    echo "<h2>Bulletin de " . htmlspecialchars($eleve["nom"]) . " " . htmlspecialchars($eleve["prenom"]) .
        "</h2>";
    echo "<p>Classe : " . htmlspecialchars($eleve["classe"]) . " | Matricule : " . htmlspecialchars($eleve["matricule"]) . "</p>";
    echo "<table border='1' cellpadding='5'><tr><th>Matière</th><th>Note</th><th>Coeff</th></tr>";
    ;
    echo "<tr><td>Mathématiques</td><td>{$notes['maths']}

```

```

    echo "</div>";
}
?>

```

### Étape 3 — Inclusions communes (includes/header.php, footer.php).

```

<!-- includes/header.php -->
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Gestion Notes – Lycée Bilingue</title>
    <style>
        body { font-family : Arial, sans-serif; margin : 2rem; background : #f4f6f9; }
        .container { max-width : 700px; margin : auto; background : white; padding : 2rem; border-radius : 8px; box-shadow : 0 2px 6px rgba(0,0,0,.1); }
        label { display : block; margin-top : 1rem; font-weight : bold; }
        input[type=number], input[type=text] { width : 100%; padding : .5rem; margin-top : .25rem; border : 1px solid #ccc; border-radius : 4px; }
        button { margin-top : 1.5rem; padding : .7rem 1.5rem; background : #2c3e50; color : white; border : none; border-radius : 4px; cursor : pointer; }
        button:hover { background : #1a252f; }
        .bulletin { margin-top : 2rem; padding : 1rem; border : 1px solid #ddd; border-radius : 4px; background : #fafafa; }
        .mention-Tres\ Bien { color : #27ae60; font-weight : bold; }
        .mention-Bien { color : #2980b9; font-weight : bold; }
        .mention-Assez\ Bien { color : #f39c12; font-weight : bold; }
        .mention-Passable { color : #2980b9; }
        .mention-Insuffisant { color : #c0392b; font-weight : bold; }
        .error { color : #c0392b; background : #fadbd8; padding : 1rem; border-radius : 4px; margin-bottom : 1rem; }
    </style>
</head>
<body>
<div class="container">
    <header><h1> Gestion des Notes – Lycée Bilingue de Yaoundé</h1></header>
    <main>

```

```

<!-- includes/footer.php -->
    </main>
    <footer style="margin-top : 2rem; text-align : center; color : #777; font-size : .9rem;">
        <hr><p>&copy; 2025 – Application pédagogique Terminale TI – MINESEC</p>
    </footer>
</div>
</body>
</html>

```

### Étape 4 — Formulaire de saisie (index.php).

```

<?php
// index.php
include __DIR__ . "/includes/header.php";

// Affichage d'erreurs éventuelles passées en GET (après redirection PRG)
if (isset($_GET['error'])) {
    echo "<div class='error'>" . htmlspecialchars($_GET['error']) . "</div>";
}

```

```

}
?>
<form action="traitement.php" method="post" novalidate>
  <fieldset>
    <legend>Informations élève</legend>
    <label>Nom : <input type="text" name="nom" required maxlength="50"></label>
    <label>Prénom : <input type="text" name="prenom" required maxlength="50"></label>
    <label>Classe : <input type="text" name="classe" value="Tle TI" required></label>
    <label>Matricule : <input type="text" name="matricule" required pattern="[A-Z0-9]{6,}"></label>
  </fieldset>
  <fieldset>
    <legend>Notes (sur 20)</legend>
    <label>Mathématiques (coeff 4) : <input type="number" step="0.01" min="0" max="20" name="maths" required></label>
    <label>Physique (coeff 3) : <input type="number" step="0.01" min="0" max="20" name="physique" required></label>
    <label>Français (coeff 3) : <input type="number" step="0.01" min="0" max="20" name="francais" required></label>
  </fieldset>
  <button type="submit">Générer le bulletin</button>
</form>
<?php include __DIR__ . "/includes/footer.php"; ?>

```

### Étape 5 — Traitement et affichage (traitement.php).

```

<?php
// traitement.php
require_once __DIR__ . "/functions.php";
include __DIR__ . "/includes/header.php";

// 1. Récupération et validation des champs obligatoires
$champsRequis = ["nom", "prenom", "classe", "matricule", "maths", "physique", "francais"];
$manquants = [];
foreach ($champsRequis as $champ) {
    if (!isset($_POST[$champ]) || trim($_POST[$champ]) == "") {
        $manquants[] = $champ;
    }
}

if ($manquants) {
    // Redirection PRG (Post-Redirect-Get) pour éviter resoumission au F5
    header("Location : index.php?error=" . urlencode("Champs manquants : " . implode(", ", $manquants)));
    exit;
}

// 2. Nettoyage et conversion
$eleve = [
    "nom" => trim($_POST["nom"]),
    "prenom" => trim($_POST["prenom"]),
    "classe" => trim($_POST["classe"]),
    "matricule" => strtoupper(trim($_POST["matricule"]))
];
$notes = [
    "maths" => (float) $_POST["maths"],
    "physique" => (float) $_POST["physique"],

```

```

    "français" => (float) $_POST["français"]
];

// 3. Validation métier (bornes 0-20)
foreach($notes as $matiere => $note){
    if ($note < 0 || $note > 20){
        header("Location : index.php?error=" . urlencode("Note $matiere invalide (0-20)."));
        exit;
    }
}

// 4. Traitement métier via fonctions
$moyenne = calculerMoyenne($notes["maths"], $notes["physique"], $notes["français"]);
$mention = determinerMention($moyenne);

// 5. Affichage
afficherBulletin($eleve, $notes, $moyenne, $mention);

// Lien retour
echo "<p style='text-align:center;margin-top:1.5rem;'><a href='index.php'< ' > Nouvelle saisie</a></p>";

include __DIR__ . "/includes/footer.php";
?>

```

### Manipulation & Observations.

1. Placer le dossier `bulletin/` dans `www/` (Wamp) ou `htdocs/` (XAMPP).
2. Démarrer les services (Apache + MySQL si BDD plus tard).
3. Ouvrir `http://localhost/bulletin/index.php`.
4. Tester : saisie valide → bulletin affiché ; champ vide → message d'erreur rouge ; note 25 → erreur ; actualiser (F5) sur le bulletin → pas de resoumission (grâce à PRG).
5. Vérifier le code source (Ctrl+U) : aucun code PHP visible, HTML propre et indenté.

Interprétation. Cette mini-application met en œuvre tous les savoir-faire de la leçon : exécution serveur, structure modulaire (inclusions), variables/constants/tableaux, fonctions réutilisables, opérateurs (arithmétiques pour moyenne, comparatifs/logiques pour mention), superglobales `$_POST`, validation/sanitization, sécurisation XSS (`htmlspecialchars`), redirection PRG. C'est un modèle type pour l'épreuve pratique du Baccalauréat.

#### À RETENIR

#### Check-list Bac — PHP Côté Serveur

- Environnement : Fichier `.php` dans `www/`, accès `http://localhost/...`
- Balises : `<?php ... ?>`, point-virgule ; à chaque instruction.
- Variables/Constantes : `$var` (casse), `define("CONST", val) / const CONST = val` (sans \$).

- **Tableaux** : [...] indexé (0-based) ou ["cle"=>val] associatif; print\_r pour debug.
- **Fonctions** : function f(\$p) { return \$r; }, portée locale, appel f(arg).
- **Opérateurs** : === / !== (strict), && / || (court-circuit), . (concaténation).
- **Formulaires** : name sur chaque input, method="post" (sensible) ou get (public), action cible.
- **Récupération** : isset(\$\_POST["k"]) → (float)\$\_POST["k"] → traitement.
- **Sécurité** : htmlspecialchars(\$x, ENT\_QUOTES, 'UTF-8') toujours à l'affichage.
- **Upload** : enctype="multipart/form-data", \$\_FILES, error===0, size, finfo\_file(), move\_uploaded\_file(), renommage sûr.
- **Structure** : require\_once (config/fonctions), include (vues), \_\_DIR\_\_ pour chemins fiables.

## 11 EXERCICES

### 11.1 Je vérifie mes connaissances

#### EXERCICE 7 QCM : Exécution et insertion PHP

Parmi les affirmations suivantes, une seule est exacte. Laquelle ?

1. Un script PHP s'exécute dans le navigateur du client (côté client) comme du JavaScript.
2. L'extension d'un fichier contenant du code PHP doit obligatoirement être `.html` pour être interprété.
3. Le serveur web (Apache, Nginx) transmet le code source PHP brut au navigateur si le module PHP n'est pas installé.
4. Le code PHP est inséré dans un document HTML à l'aide des balises `<?php ... ?>` et s'exécute sur le serveur avant l'envoi de la réponse HTTP.

#### EXERCICE 8 Vrai ou Faux : Structure et inclusion

Pour chaque affirmation, indiquez si elle est Vraie ou Fausse et justifiez brièvement.

1. L'instruction `include 'fichier.php'` ; provoque une erreur fatale (Fatal error) et arrête le script si le fichier est introuvable.
2. L'instruction `require_once 'config.php'` ; garantit que le fichier n'est inclus qu'une seule fois, même si l'instruction est rencontrée plusieurs fois.
3. Un commentaire sur une seule ligne en PHP s'écrit obligatoirement avec `/* ... */`.
4. La balise de fermeture `?>` est obligatoire à la fin de tout fichier PHP pur (ne contenant que du PHP).

#### EXERCICE 9 Définitions et variables superglobales

Complétez les définitions suivantes :

1. La variable superglobale `_____` est un tableau associatif contenant les données transmises par la méthode GET (paramètres d'URL).
2. La variable superglobale `_____` contient les données envoyées par un formulaire HTML avec `method="POST"`.
3. La variable superglobale `_____` permet d'accéder aux informations sur les fichiers

envoyés via un formulaire (`enctype="multipart/form-data"`).

4. Donnez un exemple de variable serveur (autre que les trois ci-dessus) permettant de connaître l'adresse IP du client.

#### EXERCICE 10 Calculs directs : Opérateurs

Évaluez les expressions PHP suivantes (donnez la valeur finale et le type si pertinent) :

1. `$a = 10; $b = 3; $resultat = $a % $b;`
2. `$x = 5; $y = "5"; var_dump($x === $y);`
3. `$p = true; $q = false; $r = $p && $q;`
4. `$montant = 150000; $tva = 19.25; $total = $montant * (1 + $tva / 100);`

## 11.2 Je m'entraîne

#### EXERCICE 11 Script PHP complet : Affichage et variables

Écrivez un script PHP complet (balises d'ouverture/fermeture incluses) qui :

1. Déclare une constante `TAUX_CONVERSION` valant 655.957 (taux FCFA/Euro).
2. Déclare une variable `$montantFCFA = 50000`.
3. Calcule l'équivalent en euros dans une variable `$montantEUR`.
4. Affiche le résultat dans une phrase HTML formatée (ex : `<p>50 000 FCFA = 76,22 EUR</p>`) en utilisant `echo` et l'interpolation de variables (guillemets doubles). Arrondissez à 2 décimales avec `number_format`.

#### EXERCICE 12 Tableaux et boucles : Bulletin simplifié

Soit le tableau associatif `$eleves` suivant :

```
$eleves = [
    ['nom' => 'Mbarga', 'prenom' => 'Jean', 'moyenne' => 14.5],
    ['nom' => 'Kouam', 'prenom' => 'Marie', 'moyenne' => 9.2],
    ['nom' => 'Njoya', 'prenom' => 'Paul', 'moyenne' => 12.0],
    ['nom' => 'Abena', 'prenom' => 'Grace', 'moyenne' => 8.5]
];
```

Écrivez le code PHP qui parcourt ce tableau avec une boucle `foreach` et affiche un tableau HTML (`<table>`) ne contenant que les élèves ayant une moyenne supérieure ou égale à 10.

Les colonnes affichées seront : Nom, Prénom, Moyenne. Utilisez `print_r($eleves)` au début du script pour le débogage (commenté ou non).

#### EXERCICE 13 Fonction utilisateur : Calcul de remise

Écrivez une fonction PHP nommée `calculerRemise($montant, $seuil = 100000, $taux = 0.10)` qui :

1. Retourne le montant de la remise (10% du montant) si `$montant` est strictement supérieur à `$seuil`.
2. Retourne 0 sinon.
3. Les paramètres `$seuil` et `$taux` ont des valeurs par défaut.

Ensuite, appelez cette fonction avec 150000 puis avec 50000 et affichez les résultats dans une liste HTML `<ul>`.

#### EXERCICE 14 Formulaire HTML + Traitement PHP : Conversion FCFA/Euro

Créez deux fichiers distincts :

1. `formulaire.html` : Un formulaire avec un champ `<input type="number" name="montant_fcfa" step="1" min="0" required>`, un bouton soumettre, et l'attribut `action="traitement.php" method="POST"`.
2. `traitement.php` : Le script qui récupère la valeur via `$_POST`, vérifie qu'elle est définie et numérique (`isset, is_numeric`), effectue la conversion en utilisant la constante `TAUX = 655.957`, et affiche le résultat ou un message d'erreur clair.

## 11.3 Je me prépare au Bac

#### EXERCICE 15 Sujet Bac Type : QCM/QR et Analyse de code

Contexte : L'administration d'un lycée de Yaoundé souhaite afficher la liste des élèves admis au Baccalauréat sur son site web. Le développeur a écrit le script `admis.php` ci-dessous.

```
<?php
// Connexion simulée à une base de données
$eleves = [
    ['matricule' => 'TI2024001', 'nom' => 'FOUDA', 'note' => 12.5],
    ['matricule' => 'TI2024002', 'nom' => 'MESSI', 'note' => 9.8],
    ['matricule' => 'TI2024003', 'nom' => 'NGONO', 'note' => 10.0],
```

```

];

echo "<h1>Liste des admis</h1>";
echo "<table border='1'>";
echo "<tr><th>Matricule</th><th>Nom</th><th>Note</th></tr>";

foreach ($eleves as $eleve) {
    if ($eleve['note'] >= 10); { // Erreur 1
        echo "<tr>";
        echo "<td>". $eleve['matricule']. "</td>";
        echo "<td>". $eleve['nom']. "</td>";
        echo "<td>". $eleve['note']. "</td>"; // Erreur 2
        echo "</tr>";
    }
}
echo "</table>";

// Affichage du nombre total d'admis
$nb_admis = count($eleves); // Erreur 3 logique
echo "<p>Total admis : $nb_admis</p>"; // Erreur 4 guillemets
?>

```

Listing 1 – Script admis.php (contient des erreurs volontaires)

## Questions :

1. QCM : Quelle est la différence fondamentale entre `include` et `require` en cas d'échec d'inclusion ?
2. QCM : Quelle superglobale utiliser pour récupérer un `matricule` passé dans l'URL (ex : `admis.php?matricule=TI2024001`) ?
3. Analyse : Identifiez et corrigez les 4 erreurs (syntaxe, logique, affichage) présentes dans le script ci-dessus. Recopiez les lignes corrigées.
4. Algorithme : Proposez l'algorithme (pseudo-code) pour calculer correctement `$nb_admis` (compteur incrémenté dans la boucle).

**EXERCICE 16** Sujet Bac Type : Développement complet – Gestion de bibliothèque

Contexte : Le club informatique du Lycée Bilingue d'Ebolowa développe une mini-application de gestion de livres. On souhaite une page `recherche.php` qui affiche un formulaire de recherche par titre et traite la soumission sur la même page (auto-soumission). Données initiales (à placer au début du script) :

```

$livres = [
    ['isbn' => '978-2-7460-1234-5', 'titre' => 'Algorithmique et Programmation', 'auteur' => 'M. TCHOUA',
     'annee' => 2022, 'dispo' => true],
    ['isbn' => '978-2-212-12345-6', 'titre' => 'Reseaux Informatiques', 'auteur' => 'P. KAMGA', 'annee' =>
     2021, 'dispo' => false],
    ['isbn' => '978-2-100-78901-2', 'titre' => 'Bases de Donnees SQL', 'auteur' => 'A. MBARGA', 'annee'
     => 2023, 'dispo' => true],

```

```
];  
define('NOM_BIBLIO', 'Bibliothèque Centrale – Lycée Bilingue Ebolowa');
```

Travail demandé :

1. Écrivez le code HTML/PHP complet du fichier `recherche.php`.
2. Le formulaire (méthode `GET`) contient un champ texte `recherche_titre` et un bouton Chercher.
3. À la soumission : récupérez le terme saisi (`$_GET`), filtrez le tableau `$livres` pour ne garder que les livres dont le titre contient le terme (insensible à la casse : `stripos`).
4. Affichez les résultats dans un tableau HTML (colonnes : ISBN, Titre, Auteur, Année, Disponibilité Oui/Non).
5. Si aucun résultat, affichez `<p class="erreur">Aucun livre trouvé pour "... "</p>`.
6. Affichez le nom de la bibliothèque (constante) dans un `<header>`.

#### EXERCICE 17 Sujet Bac Type : Architecture Client–Serveur et Sécurité

Contexte : Un élève de Terminale TI, après avoir codé son fichier `inscription.php` dans VS Code, double-clique dessus dans l'explorateur de fichiers Windows. Son navigateur s'ouvre mais affiche le code source PHP brut (ex : `<?php echo "Bonjour"; ?>`) au lieu de Bonjour.

Questions :

1. Expliquez précisément pourquoi le double-clic ne fonctionne pas pour exécuter du PHP. Décrivez le rôle du serveur web (Apache/Nginx) et du module PHP (PHP-FPM/mod\_php).
2. Décrivez la chaîne complète de traitement d'une requête HTTP `POST` sur un formulaire d'inscription, depuis le clic sur "Valider" dans le navigateur jusqu'à l'affichage de la page de confirmation. Utilisez les termes : Client, DNS, Requête HTTP, Serveur Web, Interpréteur PHP, Moteur de base de données (optionnel), Réponse HTTP, Navigateur (Rendu HTML).
3. Sécurité : Le script `inscription.php` récupère un pseudo via `$_POST['pseudo']` et l'affiche directement avec `echo $pseudo;`. Nommez la faille de sécurité critique ainsi introduite et donnez la fonction PHP native pour s'en protéger lors de l'affichage HTML.
4. Bonnes pratiques : Pourquoi est-il recommandé de séparer le fichier de configuration (constantes de BDD, identifiants) dans un fichier `config.php` inclus via `require_once`, plutôt que de l'écrire directement dans chaque script ?

## 12 CORRIGÉS DES EXERCICES

### **CORRIGÉ** Exercice 1 — QCM : Exécution et insertion PHP

L'unique affirmation exacte est la proposition 4.

- Proposition 1 : Fausse. PHP est un langage côté serveur (server-side). Le script est exécuté par l'interpréteur PHP sur le serveur web (Apache, Nginx, etc.) avant l'envoi de la réponse HTTP. Le navigateur ne reçoit que le résultat (généralement du HTML, CSS, JS), jamais le code source PHP.
- Proposition 2 : Fausse. L'extension standard et obligatoire pour qu'un fichier soit traité par le module PHP est `.php` (ou `.phtml`, `.php8`, etc.). Un fichier `.html` n'est pas interprété par PHP par défaut (sauf configuration spécifique du serveur via `AddHandler` ou `SetHandler`).
- Proposition 3 : Fausse. Si le module PHP n'est pas installé ou mal configuré, le serveur web ne sait pas interpréter le code. Selon la configuration, il peut proposer le téléchargement du fichier source (risque majeur de fuite de code) ou afficher une erreur 500 / 403, mais il ne l'exécute pas. Il ne transmet jamais le code source brut "tel quel" dans la réponse HTTP standard s'il est configuré pour le traiter.
- Proposition 4 : Vraie. C'est la définition exacte du fonctionnement de PHP : insertion via les balises `<?php ... ?>` (balises canoniques, toujours actives), exécution sur le serveur, envoi du flux de sortie (HTML) au client.

### **ATTENTION** Point de vigilance Bac

Bien distinguer côté serveur (PHP, Python, Java, Node.js backend) et côté client (HTML, CSS, JavaScript navigateur). Le Baccalauréat teste souvent cette distinction fondamentale : « Où le code est-il exécuté ? » → Serveur pour PHP, Navigateur pour JS/HTML/CSS.

### **CORRIGÉ** Exercice 2 — Vrai ou Faux : Structure et inclusion

1. Faux. `include` émet un `Warning` (avertissement) si le fichier est introuvable, mais ne stoppe pas l'exécution du script. C'est `require` qui provoque une `Fatal error` et arrête le script immédiatement.
2. Vrai. `require_once` (et `include_once`) vérifie si le fichier a déjà été inclus lors de l'exécution courante. Si c'est le cas, il ignore l'instruction suivante, évitant ainsi les erreurs de redéclaration de fonctions, classes ou constantes.

### 3. Faux. PHP supporte trois syntaxes de commentaires :

- `//` Commentaire sur une ligne (style C++),
- `#` Commentaire sur une ligne (style Shell/Perl),
- `/*` Commentaire sur plusieurs lignes `*/` (style C).

Le `//` est le plus courant pour une seule ligne ; `#` est rare mais valide.

### 4. Faux (et déconseillé). Dans un fichier pur PHP (ne contenant que du code PHP, pas de HTML hors balises), la balise de fermeture `?>` est omise volontairement (recommandation PSR-12 et PHP-FIG). Cela évite l'envoi accidentel d'espaces blancs ou de sauts de ligne après la balise, ce qui provoquerait l'erreur "Cannot modify header information - headers already sent" lors de l'utilisation de `header()`, `session_start()`, ou `setcookie()`.

#### ATTENTION Classiques du Bac

- La distinction `include` (Warning, continuation) vs `require` (Fatal Error, arrêt) est un incontournable.
- L'omission de `?>` en fin de fichier pur PHP est une marque de professionnalisme (norme PSR-12).

#### CORRIGÉ Exercice 3 — Variables superglobales et variables serveur

1. `$_GET` : paramètres d'URL (query string).
2. `$_POST` : données envoyées par formulaire `method="POST"` (corps de la requête HTTP).
3. `$_FILES` : fichiers uploadés via formulaire `enctype="multipart/form-data"`.
4. `$_SERVER['REMOTE_ADDR']` : adresse IP du client effectuant la requête.

#### EXEMPLE Autres variables `$_SERVER` utiles

- `$_SERVER['HTTP_USER_AGENT']` : identifiant du navigateur/client.
- `$_SERVER['REQUEST_METHOD']` : GET, POST, PUT, etc.
- `$_SERVER['PHP_SELF']` : nom du script en cours d'exécution (ex : `/index.php`).
- `$_SERVER['HTTP_REFERER']` : page d'origine (non fiable, peut être absent ou falsifié).

**ATTENTION** Faille XSS sur `PHP_SELF`

N'utilisez jamais `echo $_SERVER['PHP_SELF'];` directement dans un attribut `action` de formulaire sans `htmlspecialchars()`. Un attaquant pourrait injecter du JavaScript via l'URL (ex : `/script.php/"><script>alert(1)</script>`). Toujours écrire : `action="<?= htmlspecialchars($_SERVER['PHP_SELF']); ?>"`.

Rappel : Les variables superglobales sont accessibles partout dans le script (portée globale automatique) sans avoir besoin du mot-clé `global`.

**CORRIGÉ** Exercice 4 — Calculs directs : Opérateurs

- `$a = 10; $b = 3; $resultat = $a % $b;`
  - Opérateur `%` = modulo (reste de la division euclidienne).
  - $10 = 3 \times 3 + 1$ .
  - Résultat : 1 (type `int`).
- `$x = 5; $y = "5"; var_dump($x === $y);`
  - Opérateur `===` = identité (valeur ET type identiques).
  - `$x` est de type `integer`, `$y` est de type `string`.
  - Résultat : `bool(false)`.
  - Note : `$x == $y` (égalité valeur seule, avec type juggling) aurait donné `true`.
- `$p = true; $q = false; $r = $p && $q;`
  - Opérateur `&&` = ET logique (AND). Vrai seulement si les deux opérandes sont vraies. Court-circuite (évalue `$q` seulement si `$p` est `true`).
  - `true && false = false`.
  - Résultat : `false` (type `bool`).
- `$montant = 150000; $tva = 19.25; $total = $montant * (1 + $tva / 100);`
  - Priorités : division / avant addition +, puis multiplication \*.
  - $\frac{19.25}{100} = 0,1925$ .
  - $1 + 0,1925 = 1,1925$ .
  - $150000 \times 1,1925 = 178875$ .
  - Résultat : 178875 (type `float` car `$tva` est `float` → promotion de type).

## CORRIGÉ

## Exercice 5 — Script PHP complet : Affichage et variables

```
<?php
// 1. Déclaration de la constante (convention : MAJUSCULES_SNAKE_CASE)
define('TAUX_CONVERSION', 655.957);

// 2. Déclaration de la variable
$montantFCFA = 50000;

// 3. Calcul de l'équivalent en euros
$montantEUR = $montantFCFA / TAUX_CONVERSION;

// 4. Affichage formaté avec concaténation (plus lisible pour les appels de fonction)
// number_format(valeur, décimales, séparateur_décimal, séparateur_milliers)
// Standard FR : 2 décimales, virgule, espace insécable (ou espace simple)
echo "<p>" . number_format($montantFCFA, 0, ',', ' ') . " FCFA = "
    . number_format($montantEUR, 2, ',', ' ') . " EUR</p>";

/*
Sortie attendue :
<p>50 000 FCFA = 76,22 EUR</p>
(50000 / 655.957 = 76.224... -> arrondi 76,22)
*/
?>
```

## À RETENIR

## Points clés

- `define('NOM', valeur)` pour une constante globale (insensible à la casse par défaut, mais convention majuscules).
- `const NOM = valeur;` (au niveau supérieur) pour une constante de classe ou namespace (sensible à la casse).
- `number_format` est indispensable pour l'affichage monétaire (arrondi correct + formatage local).
- L'interpolation dans les guillemets doubles "... \$var ..." fonctionne, mais pour des appels de fonction complexes comme `number_format`, la concaténation `.` est plus lisible et évite les ambiguïtés de parsing.

## CORRIGÉ

## Exercice 6 — Tableaux et boucles : Bulletin simplifié

```
<?php
// Tableau associatif multidimensionnel (liste d'enregistrements)
$eleves = [
    ['nom' => 'Mbarga', 'prenom' => 'Jean', 'moyenne' => 14.5],
```

```
[ 'nom' => 'Kouam', 'prenom' => 'Marie', 'moyenne' => 9.2],  
[ 'nom' => 'Njoya', 'prenom' => 'Paul', 'moyenne' => 12.0],  
[ 'nom' => 'Abena', 'prenom' => 'Grace', 'moyenne' => 8.5]  
];  
  
// Débogage : visualiser la structure (à commenter/supprimer en production)  
// print_r($eleves);  
  
// Début du tableau HTML  
echo "<table border='1' style='border-collapse : collapse; width : 50%;>";  
echo "<thead><tr><th>Nom</th><th>Prénom</th><th>Moyenne</th></tr></thead>";  
echo "<tbody>";  
  
foreach ($eleves as $eleve) {  
    // Condition : moyenne >= 10 (admis)  
    if ($eleve['moyenne'] >= 10) {  
        echo "<tr>";  
        // htmlspecialchars() : OBLIGATOIRE pour toute sortie HTML (prévention XSS)  
        echo "<td>". htmlspecialchars($eleve['nom']). "</td>";  
        echo "<td>". htmlspecialchars($eleve['prenom']). "</td>";  
        // number_format pour forcer 2 décimales (14,50 au lieu de 14,5)  
        echo "<td>". number_format($eleve['moyenne'], 2, ',', ' '). "</td>";  
        echo "</tr>";  
    }  
}  
  
echo "</tbody></table>";  
?>
```

Sortie attendue (HTML rendu) :

| Nom    | Prénom | Moyenne |
|--------|--------|---------|
| Mbarga | Jean   | 14,50   |
| Njoya  | Paul   | 12,00   |

#### ATTENTION Bonnes pratiques illustrées

- htmlspecialchars() systématique sur toute donnée dynamique affichée en HTML (réflexe sécurité XSS).
- number\_format pour l’affichage numérique standardisé (2 décimales fixes).
- La condition `>= 10` inclut la moyenne 10,0 (Paul Njoya admis).
- print\_r / var\_dump sont des outils de débogage, à retirer en production.

#### CORRIGÉ Exercice 7 — Fonction utilisateur : Calcul de remise

```
<?php
/**
 * Calcule une remise si le montant dépasse un seuil.
 * @param float $montant Montant total de l'achat
 * @param float $seuil Seuil de déclenchement (défaut : 100 000)
 * @param float $taux Taux de remise (défaut : 0.10 = 10%)
 * @return float Montant de la remise (0 si seuil non atteint)
 */
function calculerRemise($montant, $seuil = 100000, $taux = 0.10) {
    if ($montant > $seuil) {
        return $montant * $taux;
    }
    return 0;
}

// Tests
$remise1 = calculerRemise(150000); // > 100000 -> 150000 * 0.10 = 15000
$remise2 = calculerRemise(50000); // <= 100000 -> 0

// Affichage dans une liste HTML
echo "<ul>";
echo "<li>Montant 150 000 FCFA : Remise = " . number_format($remise1, 0, ',', '') . " FCFA</li>";
echo "<li>Montant 50 000 FCFA : Remise = " . number_format($remise2, 0, ',', '') . " FCFA</li>";
echo "</ul>";
?>
```

Sortie attendue :

- Montant 150 000 FCFA : Remise = 15 000 FCFA
- Montant 50 000 FCFA : Remise = 0 FCFA

#### À RETENIR Points clés fonction

- Les paramètres par défaut (\$seuil = 100000, \$taux = 0.10) permettent d'appeler la fonction avec 1, 2 ou 3 arguments (polymorphisme ad hoc).
- La condition est strictement supérieur (>), pas >= (selon énoncé "dépasse").
- La fonction retourne une valeur (ne fait pas d'echo elle-même), ce qui la rend réutilisable, testable et respectueuse de la séparation responsabilités.

#### CORRIGÉ Exercice 8 — Formulaire HTML + Traitement PHP : Conversion FCFA/Euro

Fichier 1 : formulaire.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
```

```

<title>Conversion FCFA vers Euro</title>
<style>
  body { font-family : sans-serif ; max-width : 400px ; margin : 2rem auto ; }
  .form-group { margin-bottom : 1rem ; }
  label { display : block ; margin-bottom : 0.5rem ; font-weight : bold ; }
  input[type="number"] { width : 100% ; padding : 0.5rem ; box-sizing : border-box ; }
  button { padding : 0.7rem 1.5rem ; background : #0066cc ; color : white ; border : none ; cursor :
  pointer ; }
  button :hover { background : #004499 ; }
</style>
</head>
<body>
  <h1>Convertisseur FCFA \rightarrow EUR</h1>
  <form action="traitement.php" method="POST">
    <div class="form-group">
      <label for="montant_fcfa">Montant en FCFA :</label>
      <input type="number" id="montant_fcfa" name="montant_fcfa"
        step="1" min="0" required placeholder="Ex : 50000">
    </div>
    <button type="submit">Convertir</button>
  </form>
</body>
</html>

```

## Fichier 2 : traitement.php

```

<?php
// Constante de conversion (taux BCEAO fixe)
define('TAUX', 655.957);

// 1. Vérification : la requête est-elle POST et le champ existe-t-il ?
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['montant_fcfa'])) {

  $valeurBrute = $_POST['montant_fcfa'];

  // 2. Validation : est-ce numérique et positif ?
  // is_numeric accepte "123", "12.3", " 123 ", "1e2", "0xFF" (hex).
  // Pour une saisie utilisateur stricte, filter_var est plus robuste (voir note).
  if (is_numeric($valeurBrute) && $valeurBrute >= 0) {
    $montantFCFA = (float)$valeurBrute ; // Cast explicite vers float

    // 3. Calcul
    $montantEUR = $montantFCFA / TAUX ;

    // 4. Affichage résultat (page HTML complète)
    echo "<!DOCTYPE html><html lang='fr'><head><meta charset='UTF-8'><title>Résultat</title></
    head><body>";
    echo "<h1>Résultat de la conversion</h1>";
    echo "<p><strong>". number_format($montantFCFA, 0, ',', '') . " FCFA</strong> = ";
    echo "<strong>". number_format($montantEUR, 2, ',', '') . " EUR</strong></p>";
    echo "<p><a href='formulaire.html'>Nouvelle conversion</a></p>";
    echo "</body></html>";

  } else {
    // Erreur : valeur non numérique ou négative
    echo "<!DOCTYPE html><html lang='fr'><head><meta charset='UTF-8'><title>Erreur</title></
    head><body>";
  }
}

```

```
    echo "<h1>Erreur de saisie</h1>";
    // htmlspecialchars sur la valeur réaffichée (protection XSS réflexe)
    echo "<p class='erreur' style='color:red;'>La valeur saisie (<em>".htmlspecialchars($valeurBrute)
    . "</em>) n'est pas un montant valide (nombre positif attendu).</p>";
    echo "<p><a href='formulaire.html'>Revenir au formulaire</a></p>";
    echo "</body></html>";
}

} else {
    // Accès direct au script sans passer par le formulaire (ou méthode GET)
    echo "<!DOCTYPE html><html lang='fr'><head><meta charset='UTF-8'><title>Accès interdit</title>
    <\/head><body>";
    echo "<h1>Accès non autorisé</h1>";
    echo "<p>Veuillez utiliser le formulaire de saisie.</p>";
    echo "<p><a href='formulaire.html'>Aller au formulaire</a></p>";
    echo "</body></html>";
}
?>
```

**ATTENTION Points de vigilance (Bac)**

- `isset()` avant d'accéder à `$_POST` (évite `Warning: Undefined array key`).
- `is_numeric()` valide le format numérique mais accepte l'hexadécimal (0xFF) et la notation scientifique (1e2). Pour une robustesse pro : `filter_var($val, FILTER_VALIDATE_FLOAT, ['options' => ['min_range' => 0]])`.
- `htmlspecialchars()` sur la valeur réaffichée en cas d'erreur (protection XSS réflexe).
- Vérification `$_SERVER['REQUEST_METHOD'] === 'POST'` pour éviter le traitement direct via URL (GET).
- Structure HTML complète dans la réponse PHP (doctype, head, body) pour une page valide.

**CORRIGÉ Exercice 9 — Sujet Bac Type : QCM/QR et Analyse de code**

Question 1 (QCM) : Différence `include` vs `require` Réponse : En cas d'échec (fichier introuvable), `include` émet un `Warning` et le script continue son exécution. `require` émet une `Fatal Error` et arrête immédiatement le script.

Question 2 (QCM) : Superglobale pour paramètre d'URL Réponse : `$_GET`. L'URL `admis.php?matricule=TI2024001` place `matricule` dans `$_GET['matricule']`.

Question 3 (Analyse) : Correction des 4 erreurs

1. Erreur 1 (Syntaxe/Logique) : Point-virgule après le `if`.

```
// LIGNE INCORRECTE :  
if ($eleve['note'] >= 10); {  
// LIGNE CORRIGÉE (supprimer le ;) :  
if ($eleve['note'] >= 10) {
```

Explication : Le ; termine l'instruction `if`. Le bloc `{ ... }` devient un bloc indépendant exécuté systématiquement pour chaque élève, ignorant la condition. Tous les élèves s'affichent.

## 2. Erreur 2 (Syntaxe) : Clé de tableau sans guillemets.

```
// LIGNE INCORRECTE :  
echo "<td>" . $eleve[note] . "</td>";  
// LIGNE CORRIGÉE (guillemets autour de la clé string) :  
echo "<td>" . $eleve['note'] . "</td>";  
// Ou avec interpolation (guillemets doubles) : echo "<td>{$eleve['note']}</td>";
```

Explication : `note` sans guillemets est interprété comme une constante. Si la constante n'est pas définie, PHP émet un Warning "Use of undefined constant note" et utilise la chaîne "note" comme fallback. Cela fonctionne "par chance" mais est une erreur grave (performance, risque de collision).

## 3. Erreur 3 (Logique) : Comptage total au lieu des admis.

```
// LIGNE INCORRECTE (après la boucle) :  
$nb_admis = count($eleves);  
// CORRECTION : Initialiser AVANT la boucle, incrémenter DANS le if.
```

Explication : `count($eleves)` compte tous les élèves (3 ici), pas seulement les admis (2 ici : Fouda et Ngono).

## 4. Erreur 4 (Syntaxe/Affichage) : Guillemets simples / Interpolation incorrecte.

```
// LIGNE INCORRECTE (guillemets simples -> pas d'interpolation) :  
echo '<p>Total admis : $nb_admis</p>';  
// LIGNE CORRIGÉE (concaténation propre ou accolades) :  
echo "<p>Total admis : " . $nb_admis . "</p>";  
// Ou : echo "<p>Total admis : {$nb_admis}</p>";
```

Explication : Dans les guillemets simples, les variables ne sont pas interpolées. La correction "propre" (convention PSR) privilégie la concaténation . ou la syntaxe `{ $var }` pour lisibilité.

Script corrigé complet :

```
<?php  
$eleves = [  
    ['matricule' => 'TI2024001', 'nom' => 'FOUDA', 'note' => 12.5],  
    ['matricule' => 'TI2024002', 'nom' => 'MESSI', 'note' => 9.8],  
    ['matricule' => 'TI2024003', 'nom' => 'NGONO', 'note' => 10.0],
```

```
];

echo "<h1>Liste des admis</h1>";
echo "<table border='1'>";
echo "<tr><th>Matricule</th><th>Nom</th><th>Note</th></tr>";

// Initialisation du compteur AVANT la boucle
$nb_admis = 0;

foreach($eleves as $eleve){
    // Erreur 1 corrigée : pas de point-virgule
    if ($eleve['note'] >= 10){
        echo "<tr>";
        echo "<td>". $eleve['matricule']. "</td>";
        echo "<td>". $eleve['nom']. "</td>";
        // Erreur 2 corrigée : guillemets sur 'note'
        echo "<td>". $eleve['note']. "</td>";
        echo "</tr>";
        // Incréméntation du compteur DANS le if (Erreur 3 corrigée)
        $nb_admis++;
    }
}
echo "</table>";

// Erreur 4 corrigée : concaténation propre
echo "<p>Total admis : ". $nb_admis. "</p>";
?>
```

#### Question 4 (Algorithme) : Pseudo-code pour \$nb\_admis

```
// Initialisation
nb_admis <- 0

POUR CHAQUE eleve DANS eleves FAIRE
    SI eleve['note'] >= 10 ALORS
        // Affichage ligne tableau (omis ici)
        nb_admis <- nb_admis + 1
    FIN SI
FIN POUR

AFFICHER "Total admis : ", nb_admis
```

## À RETENIR

## Barème indicatif (sur 6 pts typiques Bac)

- Q1 : 1 pt (distinction Warning/Fatal Error + continuation/arrêt).
- Q2 : 0,5 pt (\$\_GET).
- Q3 : 2,5 pts (0,5 pt par erreur identifiée + 0,5 pt par correction correcte).
- Q4 : 1 pt (initialisation à 0 avant boucle, incrémentation dans le SI, affichage après boucle).
- Présentation / rigueur syntaxique : 1 pt.

## CORRIGÉ

## Exercice 10 — Sujet Bac Type : Développement complet – Gestion de bibliothèque

```
<?php
// --- DONNÉES INITIALES (Modèle) ---
$livres = [
    ['isbn' => '978-2-7460-1234-5', 'titre' => 'Algorithmique et Programmation', 'auteur' => 'M. TCHOUA',
     'annee' => 2022, 'dispo' => true],
    ['isbn' => '978-2-212-12345-6', 'titre' => 'Réseaux Informatiques', 'auteur' => 'P. KAMGA', 'annee' =>
     2021, 'dispo' => false],
    ['isbn' => '978-2-100-78901-2', 'titre' => 'Bases de Données SQL', 'auteur' => 'A. MBARGA', 'annee'
     => 2023, 'dispo' => true],
];
define('NOM_BIBLIO', 'Bibliothèque Centrale – Lycée Bilingue Ebolowa');

// --- TRAITEMENT (Contrôleur) ---
$resultats = $livres; // Par défaut, on affiche tout
$termeRecherche = '';
$rechercheEffectuee = false;

if ($_SERVER['REQUEST_METHOD'] === 'GET' && isset($_GET['recherche_titre'])) {
    $termeRecherche = trim($_GET['recherche_titre']); // Nettoyage espaces
    $rechercheEffectuee = true;

    if ($termeRecherche !== '') {
        // Filtrage insensible à la casse avec strpos
        $resultats = array_filter($livres, function($livre) use ($termeRecherche) {
            // strpos retourne la position (entier >= 0) ou false
            // On utilise !== false pour tester l'existence (0 == false en loose comparison !)
            return strpos($livre['titre'], $termeRecherche) !== false;
        });
        // Réindexer le tableau (array_filter conserve les clés originales)
        $resultats = array_values($resultats);
    } else {
        // Terme vide -> on affiche tout
        $resultats = $livres;
    }
}
```

```

    }
  }
  ?>
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Recherche Livre – <? = htmlspecialchars(NOM_BIBLIO) ?></title>
  <style>
    body { font-family : Arial, sans-serif; margin : 2rem; max-width : 900px; }
    header { background : #2c3e50; color : white; padding : 1rem; text-align : center; margin-bottom : 2rem; border-radius : 5px; }
    form { background : #f8f9fa; padding : 1.5rem; border-radius : 5px; margin-bottom : 2rem; border : 1px solid #ddd; }
    .form-group { display : flex; gap : 1rem; align-items : center; flex-wrap : wrap; }
    label { font-weight : bold; }
    input[type="text"] { flex-grow : 1; min-width : 200px; padding : 0.5rem; border : 1px solid #ccc; border-radius : 3px; }
    button { padding : 0.5rem 1.5rem; background : #2c3e50; color : white; border : none; border-radius : 3px; cursor : pointer; }
    button:hover { background : #1a252f; }
    table { width : 100%; border-collapse : collapse; margin-top : 1rem; }
    th, td { border : 1px solid #ddd; padding : 0.75rem; text-align : left; }
    th { background : #34495e; color : white; }
    tr:nth-child(even) { background : #f2f2f2; }
    .dispo-oui { color : green; font-weight : bold; }
    .dispo-non { color : red; font-weight : bold; }
    .erreur { color : #c0392b; background : #fadbdb; padding : 1rem; border-radius : 5px; border : 1px solid #c0392b; }
    .info { color : #2980b9; font-style : italic; }
  </style>
</head>
<body>

<header>
  <h1><? = htmlspecialchars(NOM_BIBLIO) ?></h1>
</header>

<!-- FORMULAIRE (Vue) – Auto-soumission (action vide) -->
<form method="GET" action="">
  <div class="form-group">
    <label for="recherche_titre">Rechercher par titre :</label>
    <input type="text"
      id="recherche_titre"
      name="recherche_titre"
      value="<? = htmlspecialchars($termeRecherche) ?>"
      placeholder="Ex : Algorithmique, Réseaux, SQL...">
    <button type="submit">Chercher</button>
    <?php if ($rechercheEffectuee && $termeRecherche !== '') : ?>
      <a href="recherche.php" style="margin-left : 1rem; text-decoration : none; color : #666;">
        Réinitialiser</a>
    <?php endif; ?>
  </div>
</form>

<!-- AFFICHAGE RÉSULTATS -->
<?php if ($rechercheEffectuee && $termeRecherche !== '' && empty($resultats)) : ?>

```

```

<p class="erreur">Aucun livre trouvé pour "<strong><?= htmlspecialchars($termeRecherche) ?></strong>"</p>

<?php elseif (!empty($resultats)) : ?>
<p class="info"><?= count($resultats) ?> livre(s) trouvé(s)<?= $rechercheEffectuee ? ' pour ' .
htmlspecialchars($termeRecherche) . ' ' : ' ' ?></p>
<table>
  <thead>
    <tr>
      <th>ISBN</th>
      <th>Titre</th>
      <th>Auteur</th>
      <th>Année</th>
      <th>Disponibilité</th>
    </tr>
  </thead>
  <tbody>
    <?php foreach ($resultats as $livre) : ?>
      <tr>
        <td><?= htmlspecialchars($livre['isbn']) ?></td>
        <td><?= htmlspecialchars($livre['titre']) ?></td>
        <td><?= htmlspecialchars($livre['auteur']) ?></td>
        <td><?= htmlspecialchars($livre['annee']) ?></td>
        <td class="<?= $livre['dispo'] ? 'dispo-oui' : 'dispo-non' ?>">
          <?= $livre['dispo'] ? 'Oui' : 'Non' ?>
        </td>
      </tr>
    <?php endforeach; ?>
  </tbody>
</table>

<?php else : ?>
<!-- Cas initial (pas de recherche) ou terme vide --> liste complète -->
<p class="info">Liste complète des ouvrages (<?= count($livres) ?>)</p>
<table>
  <thead>
    <tr><th>ISBN</th><th>Titre</th><th>Auteur</th><th>Année</th><th>Disponibilité</th></tr>
  </thead>
  <tbody>
    <?php foreach ($livres as $livre) : ?>
      <tr>
        <td><?= htmlspecialchars($livre['isbn']) ?></td>
        <td><?= htmlspecialchars($livre['titre']) ?></td>
        <td><?= htmlspecialchars($livre['auteur']) ?></td>
        <td><?= htmlspecialchars($livre['annee']) ?></td>
        <td class="<?= $livre['dispo'] ? 'dispo-oui' : 'dispo-non' ?>"><?= $livre['dispo'] ? 'Oui'
          : 'Non' ?></td>
      </tr>
    <?php endforeach; ?>
  </tbody>
</table>
<?php endif; ?>

</body>
</html>

```

**À RETENIR****Points de vigilance (Barème Bac 10–12 pts)**

- Architecture MVC simplifiée : Données (haut), Traitement (milieu), Vue (bas HTML). Séparation claire des responsabilités.
- Auto-soumission : `action=""` (ou `$_SERVER['PHP_SELF']` sécurisé par `htmlspecialchars()`) + `method="GET"` (idempotent, bookmarkable).
- Récupération sécurisée : `isset($_GET['...']), trim(), htmlspecialchars()` en réaffichage (valeur du input).
- Filtrage : `array_filter` + fonction anonyme (closure) + `use ($termeRecherche)` + `stripos(...)` `!= false`. `array_values` pour réindexer (clés numériques continues).
- Affichage conditionnel : 3 cas distincts : (1) Recherche + 0 résultat → message erreur. (2) Résultats trouvés → tableau. (3) Pas de recherche / terme vide → liste complète.
- Sécurité XSS : `htmlspecialchars()` sur Toutes les sorties dynamiques (constante, variable recherche, données livres).
- Constante : `NOM_BIBLIO` affichée dans `<header>` et `<title>`.
- HTML valide : Structure complète, `<thead>/<tbody>`, attributs `id/for` pour accessibilité.
- Syntaxe alternative : `<?php if ... : ?> ... <?php endif; ?>` et `<?php foreach ... : ?> ... <?php endforeach; ?>` idéales dans les templates HTML.

**CORRIGÉ****Exercice 11 — Sujet Bac Type : Architecture Client–Serveur et Sécurité**

Question 1 : Pourquoi le double-clic ne fonctionne pas

**PROPRIÉTÉ Explication**

Lorsqu'on double-clique sur un fichier `.php` dans l'explorateur de fichiers, le système d'exploitation (via l'association de fichiers) l'ouvre directement dans le navigateur via le protocole `file:///` (ex : `file:///C:/Users/.../inscription.php`). Le navigateur ne fait que lire le fichier comme du texte statique. Il ne sait pas exécuter du PHP. Il affiche le contenu brut du fichier (balises `<?php ... ?>` comprises) car il ne reconnaît pas cette syntaxe comme du HTML valide à rendre.

Rôle du Serveur Web (Apache/Nginx) + Module PHP (PHP-FPM/mod\_php) :

1. Le Serveur Web écoute sur le port 80 (HTTP) ou 443 (HTTPS). Il reçoit les requêtes HTTP (GET, POST...).
2. Si l'URL cible un fichier `.php`, le serveur web ne l'envoie pas tel quel. Il le transmet au Module PHP (via `mod_php` intégré ou PHP-FPM via FastCGI).
3. L'Interpréteur PHP analyse le code, l'exécute (accès BDD, calculs, logique), génère une sortie (généralement du HTML).
4. Le Serveur Web récupère cette sortie, construit la Réponse HTTP (en-têtes + corps HTML) et l'envoie au navigateur.
5. Le Navigateur rend (rendu) le HTML reçu.

Conclusion : Sans serveur web actif (WampServer, XAMPP, Laragon, Docker, ou serveur distant) accédé via `http://localhost/...` ou `http://domaine.tld/...`, le code PHP n'est jamais exécuté.

**Question 2 : Chaîne complète de traitement d'une requête POST (Inscription)**

1. Client (Navigateur) : L'utilisateur remplit le formulaire HTML (`<form method="POST" action="inscription.php">`) et clique sur "Valider".
2. Client (Navigateur) : Construction de la Requête HTTP POST.
  - Ligne de requête : `POST /inscription.php HTTP/1.1`
  - En-têtes : `Host: www.lycee.cm, Content-Type: application/x-www-form-urlencoded, Content-Length: ..., Cookie: PHPSESSID=...`
  - Corps : `pseudo=Jean&email=jean%40exemple.com&mdp=secret` (données encodées URL).
3. Réseau / DNS : Résolution du nom de domaine `www.lycee.cm` en adresse IP (ex : `192.168.1.50` ou IP publique) via DNS. Routage IP/TCP vers le serveur.
4. Serveur Web (Apache/Nginx) : Réception de la requête TCP sur le port 443 (HTTPS) ou 80. Déchiffrement TLS (si HTTPS). Analyse de l'URL (`/inscription.php`). Détection de l'extension `.php`.

5. Serveur Web → Interpréteur PHP (PHP-FPM/mod\_php) : Transmission du script à exécuter + variables d'environnement (`$_SERVER`, `$_POST`, `$_GET`, `$_COOKIE`, `$_FILES`).

6. Interpréteur PHP :

- Exécution du script `inscription.php`.
- Récupération données : `$pseudo = $_POST['pseudo'] ?? '';`
- Traitement métier : Validation, hachage mot de passe (`password_hash`), vérification unicité...
- Accès Base de Données (Moteur MySQL/MariaDB/PostgreSQL) : Connexion (PDO), Requête `INSERT`, `Commit/Rollback`.
- Génération de la sortie (HTML de confirmation ou redirection `header('Location: ...')`).

7. Serveur Web : Réception de la sortie PHP. Construction de la Réponse HTTP.

- Ligne de statut : `HTTP/1.1 200 OK` (ou `302 Found` si redirection).
- En-têtes : `Content-Type: text/html; charset=UTF-8, Set-Cookie: ..., Content-Length: ....`
- Corps : Code HTML de la page de confirmation.

8. Réseau : Acheminement de la réponse vers le client.

9. Client (Navigateur) : Réception. Analyse en-têtes (gestion cookies, cache). Rendu HTML (construction DOM, application CSS, exécution JS). Affichage à l'écran : "Inscription réussie!".

Question 3 : Faille de sécurité critique

- Faille : XSS (Cross-Site Scripting) – de type Réfléchi (Reflected) ou Stocké (Stored) selon l'usage ultérieur.
- Explication : `echo $pseudo;` insère la donnée brute dans le flux HTML. Si `$pseudo` contient `<script>alert('hack')</script>` ou `<img src=x onerror=stealCookies()>`, le navigateur de la victime exécute ce code JavaScript dans le contexte du site. Conséquences : Vol de session (`session_id`), keylogging, défiguration, redirection vers phishing.
- Protection native PHP : `htmlspecialchars($pseudo, ENT_QUOTES, 'UTF-8')`.
  - Convertit `&` en `&amp;`, `"` en `&quot;`, `'` en `&#039;` (avec `ENT_QUOTES`), `<` en `&lt;`, `>` en `&gt;`.
  - Le navigateur affiche le texte littéral au lieu de l'interpréter comme du HTML/JS.
- Bonne pratique : Appliquer systématiquement à toute variable dynamique affichée dans un contexte HTML (`echo htmlspecialchars($var);`). Utiliser des moteurs de templates (Twig, Blade, Latte) qui l'échappent par défaut.

Question 4 : Bonnes pratiques – Fichier `config.php` + `require_once`

1. Séparation des responsabilités (SoC) / DRY (Don't Repeat Yourself) : Les identifiants BDD (hôte, nom, user, mot de passe), constantes d'application (nom site, taux TVA, chemins) sont centralisés en un seul endroit. Si le mot de passe BDD change, on modifie un seul fichier au lieu de 50 scripts.
2. Sécurité (Hors DocumentRoot) : On peut placer `config.php` en dehors du dossier public (`public_html`, `www`, `htdocs`). Ex : `/var/www/projet/config/config.php` vs `/var/www/projet/public/index.php`. Même si le serveur web est mal configuré (ex : module PHP désactivé), le fichier `config.php` n'est jamais accessible via URL (pas de fuite d'identifiants dans le navigateur).
3. `require_once` vs `include` :
  - `require` : Arrêt fatal si fichier manquant (config indispensable → l'appli ne peut pas tourner sans).
  - `_once` : Évite les erreurs "Cannot redeclare constant/function/class" si le script principal inclut un fichier qui inclut déjà `config.php` (dépendances croisées).
4. Gestion d'environnements : Facilite le passage Dev/Prod (fichier `config.local.php` ignoré par Git, `config.prod.php` sur serveur).

**À RETENIR** Barème indicatif (sur 8–10 pts)

- Q1 : 2 pts (Explication claire distinction fichier local vs HTTP + rôle Apache/PHP-FPM).
- Q2 : 3 pts (Étapes ordonnées, vocabulaire précis : Requête HTTP, DNS, Serveur Web, Interpréteur, Moteur BDD, Réponse HTTP, Rendu).
- Q3 : 2 pts (Nom XSS + `htmlspecialchars` avec bons paramètres `ENT_QUOTES`, UTF-8).
- Q4 : 2–3 pts (Centralisation, Sécurité hors WebRoot, `require_once` vs `include`).

## 13 FICHE BILAN



Carte mentale de la leçon : la programmation dynamique côté serveur avec PHP

### FICHE BILAN L'essentiel à connaître pour l'examen

#### Définitions clés

- **PHP** : langage de script exécuté côté serveur ; le navigateur ne reçoit que du HTML, jamais le code source PHP.
- **Script PHP** : bloc de code délimité par `<?php` et `?>`, insérable à tout endroit d'un docu-

ment HTML (fichier d'extension .php).

- **Variable superglobale** : tableau prédéfini accessible partout (\$\_GET, \$\_POST, \$\_FILES, \$\_SERVER).
- **Inclusion** : insertion d'un fichier PHP dans un autre via include, require (et leurs variantes \_once).

Syntaxe à connaître par cœur

| Notion             | Syntaxe                                                 |
|--------------------|---------------------------------------------------------|
| Affichage          | echo "Bonjour"; print "Salut"; print_r(\$tab);          |
| Variable           | \$nom = "Amina"; (commence par \$, pas de type déclaré) |
| Constante          | define("TVA", 0.1925); ou const TVA = 0.1925;           |
| Tableau indexé     | \$notes = array(12, 15, 9); ou \$notes = [12, 15, 9];   |
| Tableau associatif | \$eleve = ["nom"=>"Amina", "age"=>17];                  |
| Fonction           | function moyenne(\$a, \$b) { return (\$a+\$b)/2; }      |
| Commentaires       | // ligne # ligne /* bloc */                             |
| Inclusion          | require_once "config.php";                              |

Opérateurs

| Catégorie     | Opérateurs                                                |
|---------------|-----------------------------------------------------------|
| Arithmétiques | + - * / %(modulo)                                         |
| Comparaison   | == (égal), === (identique : valeur et type), != < > <= >= |
| Logiques      | && (ET),    (OU), ! (NON)                                 |
| Assignation   | = += -= *= /= .= (concaténation avec .)                   |

Récupération d'un formulaire

- method="get" ⇒ données dans l'URL, lues via \$\_GET["champ"].
- method="post" ⇒ données dans le corps de la requête, lues via \$\_POST["champ"].
- Fichier envoyé : \$\_FILES["f"]["name"], ["tmp\_name"], ["size"], ["error"] + enctype="multipart/form-data".
- Variables serveur utiles : \$\_SERVER["REQUEST\_METHOD"], \$\_SERVER["REMOTE\_ADDR"], \$\_SERVER["HTTP\_HOST"].

Méthodes express

1. Tester un script : placer le fichier dans www (WampServer), lancer Apache, ouvrir http://localhost/monfichier.php — jamais en double-cliquant sur le fichier.
2. Sécuriser une donnée reçue : vérifier isset(\$\_POST["x"]) puis afficher avec htmlspecialchars(\$\_POST["x"]).
3. Déboguer un tableau : echo "<pre>"; print\_r(\$tab); echo "</pre>";.

## À RETENIR

## Checklist avant le Bac

- ☐ Je sais expliquer la différence entre exécution côté client et côté serveur.
- ☐ Je sais insérer un bloc `<?php ... ?>` dans une page HTML.
- ☐ Je maîtrise `echo`, `print` et `print_r`.
- ☐ Je déclare correctement une variable (`$`), une constante (`define`) et les deux types de tableaux.
- ☐ J'écris et j'appelle une fonction PHP avec paramètres et `return`.
- ☐ Je distingue `==` de `===` et j'utilise `&&`, `||`, `!`.
- ☐ Je sais récupérer un champ de formulaire avec `$_GET` ou `$_POST`.
- ☐ Je connais la structure de `$_FILES` et l'attribut `enctype` obligatoire.
- ☐ Je choisis entre `include` et `require` (erreur fatale ou non).
- ☐ Je vérifie toujours `isset()` avant d'utiliser une donnée reçue.
- ☐ Je protège mes affichages avec `htmlspecialchars()` contre les injections XSS.
- ☐ Je sais tester mon script via `http://localhost/` avec WampServer.



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+